

Python-Inhalte aus V01–V20 Skripten und Praxisnotebooks

Datei: lessons/V01-Binaeres-Zahlensystem/V01-Binaeres-Zahlensystem_skript.md

[!TIP] Beispiel: Das erste Python-Programm

```
# Das klassische "Hello, World!" Programm  
print("Hello, World!")
```

[!TIP] Beispiel: Verschiedene Verwendungen von print()

```
# Text ausgeben  
print("Willkommen zur Informatik-Vorlesung!")  
  
# Zahlen ausgeben  
print(42)  
print(3.14159)  
  
# Mehrere Werte ausgeben (automatisch durch Leerzeichen getrennt)  
print("Die Antwort ist", 42)  
  
# Berechnungen direkt ausgeben  
print("5 + 3 =", 5 + 3)  
  
# Leere Zeile ausgeben  
print()
```

[!WARNING] Syntaxfehler bei fehlenden Anführungszeichen:
Wenn du Text ohne Anführungszeichen verwendest, interpretiert Python dies als Variablennamen oder Schlüsselwort. Dies führt zu einem Fehler:

```
print(Hello) # Fehler: NameError: name 'Hello' is not defined  
print("Hello") # Korrekt
```

[!TIP] Beispiel: Variablen erstellen und verwenden

```
# Variable mit Zahl erstellen  
alter = 25  
print("Ich bin", alter, "Jahre alt")  
  
# Variable mit Text erstellen  
name = "Anna"  
print("Mein Name ist", name)  
  
# Variable mit Kommazahl erstellen  
pi = 3.14159
```

```
print("Pi ist ungefähr", pi)

# Variablen in Berechnungen verwenden
radius = 5
umfang = 2 * pi * radius
print("Der Umfang eines Kreises mit Radius", radius, "ist", umfang)
```

[!TIP] Beispiel: Gültige und ungültige Variablennamen

```
# Gültige Namen
alter = 25
max_geschwindigkeit = 120
temperatur_celsius = 22.5
istGueutig = True
_internal_value = 42

# Ungültige Namen
# 1alter = 25           # Fehler: beginnt mit Ziffer
# max-geschwindigkeit = 120 # Fehler: Bindestrich nicht erlaubt
# for = 5               # Fehler: 'for' ist ein Schlüsselwort
# max geschwindigkeit = 120 # Fehler: Leerzeichen nicht erlaubt
```

[!WARNING] Überschreiben von Built-in-Namen: Python hat vordefinierte Funktionen wie `print`, `len`, `sum` etc. Verwende diese Namen nicht als Variablen, da du sonst die ursprüngliche Funktion überschreibst:

```
# Schlecht: Überschreibt die print-Funktion
print = 42
print("Hello") # Fehler: 'int' object is not callable
```

[!TIP> Beispiel: Variablen aktualisieren

```
# Variable erstellen
zaehler = 0
print("Zaehler:", zaehler)

# Wert ändern
zaehler = 5
print("Zaehler:", zaehler)

# Variable in ihrer eigenen Berechnung verwenden
zaehler = zaehler + 1
print("Zaehler:", zaehler)

# Kurzschreibweise für zaehler = zaehler + 1
zaehler += 1
print("Zaehler:", zaehler)
```

[!TIP] Beispiel: Einfache Benutzereingabe

```
# Benutzer nach seinem Namen fragen
name = input("Wie ist dein Name? ")
print("Hallo", name, "!")

# Benutzer nach seinem Alter fragen
alter = input("Wie alt bist du? ")
print("Du bist", alter, "Jahre alt.")
```

[!WARNING] `input()` gibt immer einen String zurück: Auch wenn der Benutzer eine Zahl eingibt, wird diese als String (Text) zurückgegeben. Für Berechnungen muss der String in eine Zahl konvertiert werden:

```
# Falsch: Versucht, Strings zu addieren
zahl1 = input("Erste Zahl: ") # Benutzer gibt "5" ein
zahl2 = input("Zweite Zahl: ") # Benutzer gibt "3" ein
summe = zahl1 + zahl2
print("Summe:", summe) # Ausgabe: "53" statt 8!
```

```
# Richtig: Konvertierung in Ganzzahl
zahl1 = int(input("Erste Zahl: ")) # "5" wird zu 5
zahl2 = int(input("Zweite Zahl: ")) # "3" wird zu 3
summe = zahl1 + zahl2
print("Summe:", summe) # Ausgabe: 8
```

[!TIP] Beispiel: Praktisches Programm mit Eingabe und Berechnung

```
# Programm zur Berechnung der Kreisfläche
print("=== Kreisflächen-Rechner ===")

# Radius vom Benutzer einlesen
radius_text = input("Gib den Radius in cm ein: ")
radius = float(radius_text) # String in Fließkommazahl umwandeln

# Fläche berechnen
pi = 3.14159
flaeche = pi * radius * radius

# Ergebnis ausgeben
print("Ein Kreis mit Radius", radius, "cm hat eine Fläche von", flaeche, "cm²")
```

[!TIP] Beispiel: Verschiedene Arten von Kommentaren

```
# Dies ist ein einzeliger Kommentar

# Kommentare können verwendet werden, um Code zu erklären
```

```

geschwindigkeit = 100 # in km/h

# Kommentare können auch Code temporär deaktivieren
# print("Diese Zeile wird nicht ausgeführt")

# Mehrzeiliger Kommentar (Docstring)
"""
Dies ist ein mehrzeiliger Kommentar.
Er kann sich über mehrere Zeilen erstrecken.
Wird häufig für Dokumentation am Anfang von Dateien oder Funktionen verwendet.
"""

# Praktisches Beispiel mit Kommentaren
# Berechnung der Beschleunigungszeit
v_end = 100      # Endgeschwindigkeit in km/h
v_start = 0      # Startgeschwindigkeit in km/h
a = 3.5          # Beschleunigung in m/s2

# Umrechnung km/h in m/s: durch 3.6 teilen
v_end_ms = v_end / 3.6
v_start_ms = v_start / 3.6

# Zeit berechnen:  $t = (v_{\text{end}} - v_{\text{start}}) / a$ 
zeit = (v_end_ms - v_start_ms) / a

print("Beschleunigungszeit:", zeit, "Sekunden")

[!WARNING] Fehler 1: Fehlende Anführungszeichen bei
Strings

# Falsch
print(Hallo Welt) # NameError: name 'Hallo' is not defined

# Richtig
print("Hallo Welt")

[!WARNING] Fehler 3: Vergessen, input() in Zahl zu kon-
vertieren

# Falsch
alter = input("Alter: ") # Benutzer gibt "25" ein
in_fuenf_jahren = alter + 5 # TypeError: can only concatenate str (not "int") to str

# Richtig
alter = int(input("Alter: "))
in_fuenf_jahren = alter + 5
print("In 5 Jahren bist du", in_fuenf_jahren, "Jahre alt")

```

[!WARNING] Fehler 4: Variablen vor der Zuweisung verwenden

```
# Falsch
print(ergebnis) # NameError: name 'ergebnis' is not defined
ergebnis = 42

# Richtig
ergebnis = 42
print(ergebnis)
```

[!WARNING] Fehler 5: Tippfehler bei Variablennamen

```
# Falsch
geschwindigkeit = 100
print(geschwindigkiet) # NameError: name 'geschwindigkiet' is not defined

# Richtig
geschwindigkeit = 100
print(geschwindigkeit)
```

Datei: lessons/V02-Fliesskommazahlen/V02-Fliesskommazahlen_skript.md

[!TIP] Grundlegende f-String Verwendung:

```
name = "Ada"
alter = 25
print(f"Hallo, ich bin {name} und {alter} Jahre alt.")
# Ausgabe: Hallo, ich bin Ada und 25 Jahre alt.
```

Ausdrücke in f-Strings: Innerhalb der geschweiften Klammern können beliebige Python-Ausdrücke stehen:

```
x = 10
y = 20
print(f"Die Summe von {x} und {y} ist {x + y}.")
# Ausgabe: Die Summe von 10 und 20 ist 30.
```

Dezimalstellen bei Fließkommazahlen

```
pi = 3.14159265359

# Auf 2 Dezimalstellen runden
print(f"Pi gerundet: {pi:.2f}")
# Ausgabe: Pi gerundet: 3.14

# Auf 4 Dezimalstellen
```

```
print(f"Pi präziser: {pi:.4f}")
# Ausgabe: Pi präziser: 3.1416
```

Für sehr große oder sehr kleine Zahlen ist die **wissenschaftliche Notation** (Exponentialschreibweise) praktisch:

```
grosse_zahl = 123456789.123456
kleine_zahl = 0.000000123456
```

```
print(f"Große Zahl: {grosse_zahl:.3e}")
# Ausgabe: Große Zahl: 1.235e+08
```

```
print(f"Kleine Zahl: {kleine_zahl:.3e}")
# Ausgabe: Kleine Zahl: 1.235e-07
```

Oft möchte man Ausgaben in Spalten formatieren. Die Breite eines Feldes wird vor dem Dezimalpunkt angegeben:

```
zahlen = [1.5, 12.75, 123.456, 0.001]
```

```
print("Rechtsbündig (Breite 10):")
for z in zahlen:
    print(f"{z:10.2f}")
# Ausgabe:
#      1.50
#     12.75
#    123.46
#      0.00
```

```
print("\nLinksbündig (Breite 10):")
for z in zahlen:
    print(f"{z:<10.2f}")
# Ausgabe:
# 1.50
# 12.75
# 123.46
# 0.00
```

```
print("\nZentriert (Breite 10):")
for z in zahlen:
    print(f"{z:^10.2f}")
# Ausgabe:
#   1.50
#  12.75
# 123.46
#   0.00
```

Für bessere Lesbarkeit großer Zahlen:

```
million = 1000000
milliarde = 1000000000

print(f"Eine Million: {million:,}")
# Ausgabe: Eine Million: 1,000,000

print(f"Eine Milliarde: {milliarde:_}")
# Ausgabe: Eine Milliarde: 1_000_000_000
```

Prozentangaben

```
anteil = 0.8642

print(f"Anteil: {anteil:.1%}")
# Ausgabe: Anteil: 86.4%

print(f"Anteil: {anteil:.2%}")
# Ausgabe: Anteil: 86.42%
```

Grundlegende Verwendung:

```
name = "Ada"
alter = 25
nachricht = "Hallo, ich bin {} und {} Jahre alt.".format(name, alter)
print(nachricht)
# Ausgabe: Hallo, ich bin Ada und 25 Jahre alt.
```

Mit Positionsangabe:

```
nachricht = "Hallo, ich bin {0} und {1} Jahre alt. {0} ist mein Name.".format("Ada", 25)
print(nachricht)
# Ausgabe: Hallo, ich bin Ada und 25 Jahre alt. Ada ist mein Name.
```

Mit benannten Platzhaltern:

```
nachricht = "Hallo, ich bin {name} und {alter} Jahre alt.".format(name="Ada", alter=25)
print(nachricht)
# Ausgabe: Hallo, ich bin Ada und 25 Jahre alt.
```

Formatierung wie bei f-Strings:

```
pi = 3.14159265359
print("Pi gerundet: {:.2f}".format(pi))
# Ausgabe: Pi gerundet: 3.14

print("Pi präziser: {:.4f}".format(pi))
# Ausgabe: Pi präziser: 3.1416
```

[!NOTE] Der %-Operator ist eine veraltete Methode zur String-Formatierung, die Platzhalter wie %s, %d, %f verwendet und Werte

aus einem Tupel einsetzt.

```
name = "Ada"
alter = 25
print("Hallo, ich bin %s und %d Jahre alt." % (name, alter))
# Ausgabe: Hallo, ich bin Ada und 25 Jahre alt.
```

Wichtige Platzhalter: - %s – String - %d – Integer (decimal) - %f – Float - %.2f – Float mit 2 Dezimalstellen

```
pi = 3.14159265359
print("Pi: %.2f" % pi)
# Ausgabe: Pi: 3.14
```

Standardmäßig trennt `print()` mehrere Argumente durch ein Leerzeichen. Mit `sep` kann dies angepasst werden:

```
print("A", "B", "C")
# Ausgabe: A B C
```

```
print("A", "B", "C", sep="-")
# Ausgabe: A-B-C
```

```
print("A", "B", "C", sep="")
# Ausgabe: ABC
```

```
print(2026, 1, 1, sep="/")
# Ausgabe: 2026/1/1
```

Standardmäßig fügt `print()` am Ende einen Zeilenumbruch `\n` hinzu. Mit `end` kann dies geändert werden:

```
print("Zeile 1")
print("Zeile 2")
# Ausgabe:
# Zeile 1
# Zeile 2
```

```
print("Zeile 1", end=" ")
print("Zeile 2")
# Ausgabe: Zeile 1 Zeile 2
```

```
print("Lade", end="")
print(".", end="")
print(".", end="")
print(".", end="")
print(" Fertig!")
# Ausgabe: Lade... Fertig!
```

Praktisches Beispiel – Fortschrittsbalken:


```
import time
```

```
for i in range(10):  
    print("#", end="", flush=True)  
    time.sleep(0.2)  
print(" Komplet!")  
# Ausgabe (nach und nach): ##### Komplet!
```

Normalerweise werden Ausgaben **gepuffert** (buffered), d.h. sie werden erst geschrieben, wenn der Puffer voll ist oder ein Zeilenumbruch kommt. Mit `flush=True` wird die Ausgabe sofort geschrieben:

```
print("Warte 5 Sekunden", end="", flush=True)  
time.sleep(5)  
print(" Fertig!")
```

[!TIP] `print()`-Parameter kombinieren:

```
print("Status", "OK", "2026", sep=" | ", end=" [DONE]\n")  
# Ausgabe: Status | OK | 2026 [DONE]
```

[!TIP] Escape-Sequenzen in Aktion:

```
print("Name:\tAlter:\tStadt:")  
print("Ada\t25\tBerlin")  
print("Bob\t30\tMünchen")  
# Ausgabe:  
# Name: Alter: Stadt:  
# Ada 25 Berlin  
# Bob 30 München
```

```
print("Zeile 1\nZeile 2\nZeile 3")  
# Ausgabe:  
# Zeile 1  
# Zeile 2  
# Zeile 3
```

```
print("Pfad: C:\\Users\\Ada\\Documents")  
# Ausgabe: Pfad: C:\Users\Ada\Documents
```

Wenn Sie viele Backslashes in einem String haben (z.B. bei Windows-Pfaden oder regulären Ausdrücken), können Sie **Raw Strings** verwenden, die durch ein `r` vor dem Anführungszeichen gekennzeichnet sind. In Raw Strings werden Escape-Sequenzen **nicht** interpretiert:

```
# Ohne Raw String (umständlich)  
pfad = "C:\\Users\\Ada\\Documents\\file.txt"  
print(pfad)  
# Ausgabe: C:\Users\Ada\Documents\file.txt
```

```

# Mit Raw String (einfacher)
pfad = r"C:\Users\Ada\Documents\file.txt"
print(pfad)
# Ausgabe: C:\Users\Ada\Documents\file.txt

```

Für längere Texte über mehrere Zeilen können **triple quotes** (dreifache Anführungszeichen) verwendet werden:

```

text = """Dies ist ein
mehrzeiliger String.
Er kann über mehrere
Zeilen gehen."""

```

```

print(text)
# Ausgabe:
# Dies ist ein
# mehrzeiliger String.
# Er kann über mehrere
# Zeilen gehen.

```

Mehrzeilige Strings respektieren alle Zeilenumbrüche und Einrückungen:

```

gedicht = """Rosen sind rot,
Veilchen sind blau,
Python ist toll,
und das weißt du genau!"""

```

```

print(gedicht)

```

[!TIP] **Mehrzeilige f-Strings:**

f-Strings können auch mehrzeilig sein:

```

name = "Ada"
alter = 25

```

```

info = f"""Name: {name}
Alter: {alter}
Status: Aktiv"""

```

```

print(info)
# Ausgabe:
# Name: Ada
# Alter: 25
# Status: Aktiv

```

Dateien lesen

```
# Datei im Lesemodus öffnen
datei = open("beispiel.txt", "r")
```

```
# Gesamten Inhalt lesen
inhalt = datei.read()
print(inhalt)
```

```
# Datei schließen
datei.close()
```

Zeile für Zeile lesen:

```
datei = open("beispiel.txt", "r")

for zeile in datei:
    print(zeile, end="") # end="" vermeidet doppelte Zeilenumbrüche

datei.close()
```

Alle Zeilen als Liste lesen:

```
datei = open("beispiel.txt", "r")

zeilen = datei.readlines() # Gibt Liste von Strings zurück
print(zeilen)
# ['Hallo, dies ist die erste Zeile.\n', 'Und dies ist die zweite Zeile.\n']

datei.close()
```

Das manuelle Öffnen und Schließen von Dateien ist fehleranfällig (was passiert bei einem Fehler vor `.close()`?). Python bietet das **with-Statement**, das sicherstellt, dass die Datei automatisch geschlossen wird:

```
# Schreiben mit context manager
with open("beispiel.txt", "w") as datei:
    datei.write("Diese Datei wird automatisch geschlossen.\n")
    datei.write("Auch bei Fehlern!\n")
# Datei ist hier automatisch geschlossen
```

```
# Lesen mit context manager
with open("beispiel.txt", "r") as datei:
    inhalt = datei.read()
    print(inhalt)
# Datei ist hier automatisch geschlossen
```

[!TIP] **Best Practice: Immer with verwenden!**

Das **with-Statement** ist die empfohlene Methode zum Arbeiten mit Dateien:

```

# Gut: mit context manager
with open("daten.txt", "r") as f:
    daten = f.read()

# Schlecht: manuelles Öffnen/Schließen
f = open("daten.txt", "r")
daten = f.read()
f.close() # Wird bei Fehler möglicherweise nicht erreicht!

```

Praktisches Beispiel: Messdaten speichern

```

# Messdaten generieren und in Datei speichern
with open("messdaten.txt", "w") as datei:
    datei.write("Zeit (s)\tTemperatur (°C)\n")
    datei.write("-" * 30 + "\n")

    for t in range(0, 11):
        temp = 20 + t * 0.5 # Simulierte Temperatur
        datei.write(f"{t}\t\t{temp:.1f}\n")

print("Messdaten gespeichert!")

# Messdaten wieder einlesen
with open("messdaten.txt", "r") as datei:
    print("\nInhalt der Datei:")
    print(datei.read())

[!WARNING] Fehler 2: Dezimalstellen bei print() statt in
Format-String

Problem:

pi = 3.14159
print("Pi:", pi) # Gibt alle Dezimalstellen aus

```

Lösung: Verwenden Sie f-Strings zur Kontrolle:

```
print(f"Pi: {pi:.2f}") # Gibt 3.14 aus
```

[!WARNING] Fehler 4: Vergessener Mode-Parameter beim Anhängen

Problem: Wollen an Datei anhängen, verwenden aber "w" statt "a" und löschen versehentlich alle Daten.

Lösung:

```

# Anhängen ohne Überschreiben
with open("logfile.txt", "a") as datei:
    datei.write("Neuer Eintrag\n")

```

Datei: lessons/V03-Boolsche-Algebra-Teil1/V03-Boolsche-Algebra-Teil1_skript.md

[!TIP]

```
# Integer-Beispiele
alter = 25
jahr = 2026
temperatur_celsius = -5
sehr_grosse_zahl = 123456789012345678901234567890
```

```
print(f"Alter: {alter}, Typ: {type(alter)}")
# Ausgabe: Alter: 25, Typ: <class 'int'>
```

[!TIP]

```
# Float-Beispiele
pi = 3.14159
temperatur = -12.5
wissenschaftliche_notation = 6.022e23 # Avogadro-Konstante
sehr_klein = 1.5e-10
```

```
print(f"Pi: {pi}, Typ: {type(pi)}")
# Ausgabe: Pi: 3.14159, Typ: <class 'float'>
```

[!TIP]

```
# String-Beispiele
name = "Ada Lovelace"
einzeilig = 'Python ist toll'
mehrzeilig = """Dies ist ein
mehrzeiliger String
über mehrere Zeilen."""
```

```
print(f"Name: {name}, Typ: {type(name)}")
# Ausgabe: Name: Ada Lovelace, Typ: <class 'str'>
```

[!TIP]

```
# Boolean-Beispiele
ist_student = True
ist_abgeschlossen = False
ergebnis = (5 > 3) # True, da 5 größer als 3
```

```
print(f"Student: {ist_student}, Typ: {type(ist_student)}")
# Ausgabe: Student: True, Typ: <class 'bool'>
```

[!TIP]

```

# String zu Integer
alter_str = "25"
alter_int = int(alter_str)
print(f"{alter_int} + 1 = {alter_int + 1}") # 26

# String zu Float
temperatur_str = "23.5"
temperatur_float = float(temperatur_str)
print(f"Temperatur: {temperatur_float}°C") # 23.5°C

# Mit Benutzereingabe
eingabe = input("Gib eine Zahl ein: ") # z.B. "42"
zahl = int(eingabe) # Konvertierung notwendig!
print(f"Doppelt: {zahl * 2}")

[!WARNING] ValueError bei ungültiger Konvertierung:
Wenn der String keine gültige Zahl enthält, wirft Python einen
ValueError:

int("abc") # ValueError: invalid literal for int() with base 10: 'abc'
float("3.14.15") # ValueError: could not convert string to float

[!TIP]

# Zahlen zu String
alter = 25
jahr = 2026

# Mit str() für Konkatination
nachricht = "Ich bin " + str(alter) + " Jahre alt."
print(nachricht) # Ich bin 25 Jahre alt.

# Besser: f-Strings (automatische Konvertierung)
nachricht = f"Ich bin {alter} Jahre alt."
print(nachricht) # Ich bin 25 Jahre alt.

[!TIP]

# Float zu Integer (Abschneiden, nicht Runden!)
print(int(3.7)) # 3 (nicht 4!)
print(int(3.2)) # 3
print(int(-2.9)) # -2 (nicht -3!)

# Für echtes Runden: round()
print(round(3.7)) # 4
print(round(3.2)) # 3
print(round(2.5)) # 2 (Banker's Rounding bei .5)

[!TIP]

```

```

# Falsy Values (ergeben False)
print(bool(0))      # False
print(bool(0.0))    # False
print(bool(""))     # False (leerer String)
print(bool(None))   # False

# Truthy Values (ergeben True)
print(bool(42))      # True (jede Zahl != 0)
print(bool(-1))      # True (auch negative Zahlen)
print(bool("Hallo")) # True (nicht-leerer String)
print(bool(" "))     # True (String mit Leerzeichen ist nicht leer!)

[!TIP]

# Type Checking mit type()
x = 42
y = 3.14
z = "Hallo"

print(type(x)) # <class 'int'>
print(type(y)) # <class 'float'>
print(type(z)) # <class 'str'>

# Vergleich mit type()
if type(x) == int:
    print("x ist ein Integer")

# Direkter Vergleich
print(type(x) == type(42)) # True

[!TIP]

# Type Checking mit isinstance()
x = 42
y = 3.14
z = True # bool ist Subtyp von int!

print(isinstance(x, int)) # True
print(isinstance(y, float)) # True
print(isinstance(z, int)) # True! bool erbt von int
print(isinstance(z, bool)) # True

# Mehrere Typen gleichzeitig prüfen (Tupel)
eingabe = "Hallo"
if isinstance(eingabe, (int, float)):
    print("eingabe ist eine Zahl")
else:
    print("eingabe ist keine Zahl")

```

```

[!TIP]

# Strings sind unveränderlich
text = "Hallo"
print(id(text)) # Speicheradresse, z.B. 140234567890123

text = text + " Welt" # Erzeugt NEUEN String
print(id(text)) # Andere Speicheradresse!

# Integers sind unveränderlich
x = 10
print(id(x))
x = x + 1 # Erzeugt NEUEN Integer
print(id(x)) # Andere Speicheradresse

[!TIP]

# Listen sind veränderlich
zahlen = [1, 2, 3]
print(id(zahlen)) # Speicheradresse

zahlen.append(4) # Modifiziert DIESELBE Liste
print(id(zahlen)) # GLEICHE Speicheradresse!
print(zahlen)    # [1, 2, 3, 4]

[!TIP]

# Globale Variable
sprache = "Python"

def zeige_sprache():
    print(f"Ich programmiere in {sprache}") # Zugriff auf globale Variable

zeige_sprache() # Ich programmiere in Python
print(sprache)  # Python (auch außerhalb der Funktion sichtbar)

[!TIP]

def berechne_summe():
    # Lokale Variablen
    a = 10
    b = 20
    summe = a + b
    return summe

ergebnis = berechne_summe() # 30
print(ergebnis)

# print(a) # NameError! a existiert außerhalb der Funktion nicht

```


[!WARNING] **Namenskonflikte:** Wenn eine lokale Variable denselben Namen hat wie eine globale, **überdeckt** die lokale Variable die globale innerhalb der Funktion:

```
x = "global"

def funktion():
    x = "lokal"  # Neue lokale Variable, überdeckt globales x
    print(x)    # lokal

funktion()
print(x)  # global (globales x ist unverändert)

[!TIP]

zaehler = 0  # Globale Variable

def inkrementiere():
    global zaehler  # Referenz auf globale Variable
    zaehler += 1

print(zaehler)  # 0
inkrementiere()
print(zaehler)  # 1
inkrementiere()
print(zaehler)  # 2

[!TIP]

# Mehrere Variablen mit demselben Wert
x = y = z = 0
print(x, y, z)  # 0 0 0

# Nützlich für Initialisierung
summe = anzahl = 0

[!WARNING] Vorsicht bei mutable Typen: Bei veränderlichen
Typen (Listen) zeigen alle Variablen auf dasselbe Objekt:

a = b = []  # Beide zeigen auf DIESELBE Liste!
a.append(1)
print(b)  # [1] - auch b ist betroffen!

# Besser: Separate Listen erzeugen
a = []
b = []

[!TIP]

# Mehrere Variablen gleichzeitig zuweisen
```

```

x, y, z = 1, 2, 3
print(x) # 1
print(y) # 2
print(z) # 3

# Variablen tauschen (ohne temporäre Variable!)
a, b = 10, 20
print(f"Vorher: a={a}, b={b}") # a=10, b=20

a, b = b, a # Swap!
print(f"Nachher: a={a}, b={b}") # a=20, b=10

# Funktionsrückgaben unpacking
def min_max(zahlen):
    return min(zahlen), max(zahlen)

minimum, maximum = min_max([5, 2, 8, 1, 9])
print(f"Min: {minimum}, Max: {maximum}") # Min: 1, Max: 9

```

Lösung: Prüfe den Nenner vor der Division:

```

nenner = 0
if nenner != 0:
    ergebnis = 10 / nenner
else:
    print("Division durch Null nicht möglich!")

[!WARNING] Fehler: Annahme, dass String-Methoden den originalen String ändern:

text = "hallo"
text.upper() # Erzeugt neuen String, ändert text nicht!
print(text) # hallo (unverändert)

```

Lösung: Weise das Ergebnis der Variable zu:

```

text = "hallo"
text = text.upper() # Zuweisung nötig!
print(text) # HALLO

[!WARNING] Fehler: Zuweisung (=) statt Vergleich (==) in Bedingungen:

```

```

x = 5
if x = 10: # SyntaxError: invalid syntax
    print("x ist 10")

```

Lösung: Verwende == für Vergleiche:

```

x = 5
if x == 10: # Korrekt

```

```
print("x ist 10")
```

Datei: lessons/V04-Boolsche-Algebra-Teil2/V04-Boolsche-Algebra-Teil2__skript.md

[!TIP]

```
# Gleichheit testen
```

```
x = 5
```

```
y = 5
```

```
z = 10
```

```
print(x == y) # True - beide haben denselben Wert
```

```
print(x == z) # False - unterschiedliche Werte
```

```
print(x != z) # True - sie sind ungleich
```

```
# Funktioniert auch mit Strings
```

```
name1 = "Python"
```

```
name2 = "Python"
```

```
name3 = "python" # Kleinschreibung!
```

```
print(name1 == name2) # True - identische Strings
```

```
print(name1 == name3) # False - Groß-/Kleinschreibung zählt!
```

```
print(name1 != name3) # True - sie sind unterschiedlich
```

[!WARNING] **Häufiger Fehler: == vs. =**

Anfänger verwechseln oft den Zuweisungsoperator = mit dem Vergleichsoperator ==: - $x = 5 \rightarrow$ **Zuweisung**: Variable x bekommt den Wert 5 - $x == 5 \rightarrow$ **Vergleich**: Prüft, ob x den Wert 5 hat

```
# Falsch (führt zu SyntaxError in Bedingungen):
```

```
if x = 5: # SyntaxError: invalid syntax  
    pass
```

```
# Richtig:
```

```
if x == 5:  
    pass
```

[!TIP]

```
# Zahlenvergleiche
```

```
alter = 25
```

```
print(alter < 30) # True
```

```
print(alter > 20) # True
```

```
print(alter <= 25) # True (gleich ist inkludiert!)
```

```

print(alter >= 26)  # False

# String-Vergleiche (lexikographisch)
print("Apple" < "Banana")  # True - A kommt vor B im Alphabet
print("abc" < "abd")      # True - c kommt vor d
print("Python" < "python") # True - Großbuchstaben vor Kleinbuchstaben (ASCII)

# Float-Vergleiche
temperatur = 20.5
print(temperatur >= 20.0)  # True
print(temperatur < 21.0)  # True

[!WARNING] Float-Vergleiche mit Vorsicht!

Aufgrund von Rundungsfehlern bei Fließkommazahlen (siehe V02)
sollte man Floats nicht direkt mit == vergleichen:

# Problematisch:
result = 0.1 + 0.2
print(result == 0.3)  # False! (wegen Rundungsfehlern)
print(result)         # 0.30000000000000004

# Richtig mit Toleranz (bereits in V03 eingeführt: abs()):
tolerance = 1e-10
print(abs(result - 0.3) < tolerance)  # True

[!TIP]

# AND-Verknüpfungen
a = True
b = True
c = False

print(a and b)  # True - beide sind wahr
print(a and c)  # False - c ist falsch
print(c and c)  # False - beide sind falsch

# Praktisches Beispiel - Bereichsprüfung
alter = 25
hat_fuehrerschein = True

# Darf Auto mieten? (18+ UND Führerschein)
kann_auto_mieten = alter >= 18 and hat_fuehrerschein
print(kann_auto_mieten)  # True

# Mehrfache Verknüpfung
temperatur = 22
luftfeuchtigkeit = 45

```

```

# Komfortzone: 20-25°C UND 40-60% Luftfeuchtigkeit
ist_komfortabel = (20 <= temperatur <= 25) and (40 <= luftfeuchtigkeit <= 60)
print(ist_komfortabel) # True

[!TIP]

# OR-Verknüpfungen
a = True
b = False
c = False

print(a or b) # True - a ist wahr
print(b or c) # False - beide sind falsch
print(a or c) # True - a ist wahr

# Praktisches Beispiel - Zutrittsberechtigung
ist_admin = False
ist_mitarbeiter = True
ist_gast = False

# Hat Zugang? (Admin ODER Mitarbeiter ODER Gast)
hat_zugang = ist_admin or ist_mitarbeiter or ist_gast
print(hat_zugang) # True

# Wochenende prüfen
tag = "Samstag"
ist_wochenende = (tag == "Samstag") or (tag == "Sonntag")
print(ist_wochenende) # True

[!TIP]

# NOT-Negation
a = True
b = False

print(not a) # False - kehrt True um
print(not b) # True - kehrt False um

# Praktisches Beispiel - Fehlererkennung
fehler_gefunden = False

if not fehler_gefunden:
    print("Programm läuft fehlerfrei")
# Ausgabe: Programm läuft fehlerfrei

# Doppelte Negation (hebt sich auf)
x = True

```

```

print(not not x)  # True - doppelte Negation

# Kombination mit anderen Operatoren
ist_fertig = False
ist_abgebrochen = False

noch_aktiv = not ist_fertig and not ist_abgebrochen
print(noch_aktiv)  # True

[!TIP]

# Klassische Bereichsprüfung mit and
alter = 25
if alter >= 18 and alter < 65:
    print("Erwachsen, nicht Senior")

# Eleganter mit verketteten Vergleichen
if 18 <= alter < 65:
    print("Erwachsen, nicht Senior")
# Ausgabe: Erwachsen, nicht Senior

# Weitere Beispiele
x = 15

# Prüft: 0 < x <= 100
if 0 < x <= 100:
    print(f"{x} liegt im Bereich (0, 100]")
# Ausgabe: 15 liegt im Bereich (0, 100]

# Prüft Gleichheit aller drei Variablen
a = 5
b = 5
c = 5
if a == b == c:
    print("Alle drei sind gleich")
# Ausgabe: Alle drei sind gleich

# Komplexere Kette
temperatur = 22.5
if 15 < temperatur < 25:
    print("Angenehme Temperatur")
# Ausgabe: Angenehme Temperatur

```

[!WARNING] **Vorsicht bei ungewöhnlichen Ketten**

Verkettete Vergleiche funktionieren bei allen Vergleichsoperatoren, aber nicht alle Kombinationen ergeben semantisch Sinn:

```

# Sinnvoll:
x = 10
if 5 < x < 15: # x liegt zwischen 5 und 15
    pass

# Syntaktisch korrekt, aber semantisch seltsam:
if 5 < x > 3: # x ist größer als 5 UND größer als 3 (redundant)
    pass

# Mathematisch korrekt, aber unüblich in Code:
if x < 15 > 10: # x < 15 AND 15 > 10 (15 > 10 ist immer wahr!)
    pass # Entspricht einfach x < 15

[!TIP]

# Demonstration der Kurzschlussauswertung
def teuer_check():
    print("Teure Berechnung wird ausgeführt!")
    return True

x = False

# Der rechte Teil wird NICHT ausgeführt
result = x and teuer_check()
print(result) # False
# Keine Ausgabe von "Teure Berechnung..."!

# Praktisches Beispiel - Division-durch-Null vermeiden
x = 0
y = 10

# Sicher: Prüft erst, ob x != 0, bevor dividiert wird
if x != 0 and y / x > 5:
    print("y/x ist größer als 5")
else:
    print("Division nicht möglich oder Bedingung nicht erfüllt")
# Ausgabe: Division nicht möglich oder Bedingung nicht erfüllt
# Kein ZeroDivisionError, weil y/x nicht ausgewertet wird!

# Ohne Kurzschlussauswertung würde das crashen:
# if y / x > 5 and x != 0: # ZeroDivisionError!

[!TIP]

# Demonstration der Kurzschlussauswertung bei OR
def teuer_check():
    print("Teure Berechnung wird ausgeführt!")
    return False

```

```

x = True

# Der rechte Teil wird NICHT ausgeführt
result = x or teuer_check()
print(result) # True
# Keine Ausgabe von "Teure Berechnung..."!

# Praktisches Beispiel - Default-Werte
benutzername = "" # Leerer String

# Wenn benutzername leer, verwende "Gast"
anzeigename = benutzername or "Gast"
print(anzeigename) # Gast

benutzername = "Alice"
anzeigename = benutzername or "Gast"
print(anzeigename) # Alice

```

[!WARNING] Performance-Tipp: Reihenfolge bei AND

Setze "billige" Checks vor "teure" Checks bei and:

```

# Gut: Einfacher Check zuerst
if benutzer_eingeloggt and komplexe_berechtigung_pruefen():
    pass
# Wenn benutzer_eingeloggt False ist, wird die teure Funktion nicht aufgerufen!

# Schlecht: Teure Operation zuerst
if komplexe_berechtigung_pruefen() and benutzer_eingeloggt:
    pass
# Teure Funktion wird immer aufgerufen, auch wenn benutzer_eingeloggt False ist

[!TIP]

# Beispiel 1: not hat höhere Priorität als and
x = True
y = False

result = not x and y
# Wird interpretiert als: (not x) and y
# = False and False
# = False
print(result) # False

# Beispiel 2: and hat höhere Priorität als or
a = True
b = False

```



```

c = True

result = a or b and c
# Wird interpretiert als: a or (b and c)
# = True or (False and True)
# = True or False
# = True
print(result) # True

# Beispiel 3: Vergleiche vor logischen Operatoren
x = 5
result = x > 3 and x < 10
# Wird interpretiert als: (x > 3) and (x < 10)
# = True and True
# = True
print(result) # True

['TIP']

# Zahlen
print(bool(0))      # False - Null ist falsy
print(bool(0.0))    # False - Auch Fließkomma-Null
print(bool(42))     # True - Alle anderen Zahlen sind truthy
print(bool(-5))     # True - Auch negative Zahlen!

# Strings
print(bool(""))     # False - Leerer String ist falsy
print(bool("0"))     # True - String "0" ist truthy (nicht leer!)
print(bool("Hallo")) # True - Nicht-leere Strings sind truthy

# None
print(bool(None))    # False - None ist falsy

# Collections (werden in V08 behandelt, hier zur Demonstration)
print(bool([]))      # False - Leere Liste ist falsy
print(bool([1, 2]))  # True - Nicht-leere Liste ist truthy
print(bool({}))      # False - Leeres Dictionary ist falsy

['TIP']

# Liste auf Leerheit prüfen
zahlen = []

# Unpythonisch (funktioniert, aber umständlich):
if len(zahlen) > 0:
    print("Liste enthält Elemente")
else:
    print("Liste ist leer")

```

```

# Pythonisch (idiomatisch):
if zahlen:
    print("Liste enthält Elemente")
else:
    print("Liste ist leer")
# Ausgabe: Liste ist leer

# String auf Leerheit prüfen
name = ""
if not name:
    print("Kein Name angegeben")
# Ausgabe: Kein Name angegeben

# Default-Werte mit or
eingabe = ""
wert = eingabe or "Standard"
print(wert) # Standard (weil eingabe falsy ist)

eingabe = "Benutzer"
wert = eingabe or "Standard"
print(wert) # Benutzer

[!WARNING] Vorsicht: if x: vs. if x is not None:
Diese beiden Checks sind nicht identisch!

# Fall 1: x ist 0
x = 0

if x: # False, weil 0 falsy ist
    print("x hat einen Wert")
else:
    print("x ist falsy")
# Ausgabe: x ist falsy

if x is not None: # True, weil 0 != None
    print("x ist nicht None")
# Ausgabe: x ist nicht None

# Fall 2: x ist None
x = None

if x: # False
    print("x hat einen Wert")

if x is not None: # False

```

```

    print("x ist nicht None")

# Fall 3: x ist leerer String
x = ""

if x: # False, weil "" falsy ist
    print("x hat einen Wert")

if x is not None: # True, weil "" != None
    print("x ist nicht None")
# Ausgabe: x ist nicht None

[!TIP]

# Beispiel 1: Passwortvalidierung
passwort = "Sicher123!"

# Anforderungen prüfen
hat_mindestlaenge = len(passwort) >= 8
hat_ziffer = any(c.isdigit() for c in passwort) # Vorschau auf VO8, any() wurde in VO4 erw
hat_grossbuchstaben = any(c.isupper() for c in passwort)

ist_gueltig = hat_mindestlaenge and hat_ziffer and hat_grossbuchstaben
print(f"Passwort gültig: {ist_gueltig}") # True

# Beispiel 2: Zugriffskontrolle mit mehreren Bedingungen
ist_admin = False
ist_moderator = True
ist_autor = False
beitrag_alter_tage = 3

# Darf bearbeiten, wenn: Admin ODER (Moderator UND Beitrag < 7 Tage) ODER (Autor UND Beitrag
darf_bearbeiten = (
    ist_admin or
    (ist_moderator and beitrags_alter_tage < 7) or
    (ist_autor and beitrags_alter_tage < 1)
)
print(f"Bearbeitungsrecht: {darf_bearbeiten}") # True

# Beispiel 3: Sensorvalidierung
temperatur = 85.0
druck = 1.5
vibration = 0.8

# Warnung, wenn Temperatur zu hoch ODER Druck zu niedrig ODER Vibration zu hoch
temperatur_kritisch = temperatur > 80
druck_kritisch = druck < 1.0

```

```

vibration_kritisch = vibration > 1.0

warnung_ausgeben = temperatur_kritisch or druck_kritisch or vibration_kritisch

if warnung_ausgeben:
    print(" Warnung: Kritische Betriebsbedingungen!")
    if temperatur_kritisch:
        print(f" - Temperatur zu hoch: {temperatur}°C")
    if druck_kritisch:
        print(f" - Druck zu niedrig: {druck} bar")
    if vibration_kritisch:
        print(f" - Vibration zu hoch: {vibration} mm/s")
# Ausgabe: Warnung: Kritische Betriebsbedingungen!
#          - Temperatur zu hoch: 85.0°C

```

[!WARNING] Häufige Fehler bei komplexen Bedingungen

Fehler 1: Mehrfachvergleich falsch formuliert

```

x = 5

# FALSCH (funktioniert nicht wie erwartet):
if x == 3 or 4 or 5:
    print("x ist 3, 4 oder 5")
# Wird interpretiert als: if (x == 3) or (4) or (5)
# (4) und (5) sind truthy, daher immer True!

# RICHTIG:
if x == 3 or x == 4 or x == 5:
    print("x ist 3, 4 oder 5")

# ALTERNATIV (eleganter, wird in V08 behandelt):
if x in [3, 4, 5]:
    print("x ist 3, 4 oder 5")

```

[!WARNING] **Fehler 1:** == vs. = verwechseln

Problem: Zuweisungsoperator statt Vergleichsoperator verwendet

```

# Falsch:
x = 5
if x = 10: # SyntaxError: invalid syntax
    print("x ist 10")

# Richtig:
if x == 10:
    print("x ist 10")

```

[!WARNING] **Fehler 2:** Float-Vergleiche ohne Toleranz

Problem: Direkter Vergleich von Floats schlägt wegen Rundungsfehlern fehl

```
# Problematisch:
result = 0.1 + 0.2
if result == 0.3: # False!
    print("Gleich")

# Richtig mit Toleranz:
if abs(result - 0.3) < 1e-10:
    print("Gleich")
```

[!WARNING] Fehler 3: Truthy/Falsy missverstehen

Problem: String "0" wird als truthy interpretiert (weil nicht leer!)

```
eingabe = "0"

# Falsch (denkt, "0" ist falsy):
if eingabe:
    print("Eingabe vorhanden") # Wird ausgeführt!
# Ausgabe: Eingabe vorhanden (nicht wie erwartet!)

# Richtig (expliziter Check):
if eingabe and eingabe != "0":
    print("Gültige Eingabe")

# Oder für Zahlen:
if int(eingabe) != 0:
    print("Nicht Null")
```

[!WARNING] Fehler 4: Mehrfachvergleich mit or falsch

Problem: `x == 1 or 2` funktioniert nicht wie erwartet

```
x = 5

# Falsch:
if x == 1 or 2: # Immer True!
    print("x ist 1 oder 2")
# Wird interpretiert als: if (x == 1) or (2)
# (2) ist truthy, also immer True!

# Richtig:
if x == 1 or x == 2:
    print("x ist 1 oder 2")
```

Datei: lessons/V05-Programm-Ablauf-Plaene-Teil1/V05-Programm-Ablauf-Plaene-Teil1_skript.md

Syntax:

```
if Bedingung:
    # Dieser Code wird ausgeführt, wenn Bedingung True ist
    Anweisung1
    Anweisung2
    # ...
# Programm setzt hier fort (unabhängig von Bedingung)

[!TIP]

# Beispiel: Zugriffsprüfung
alter = 20

if alter >= 18:
    print("Zugriff erlaubt")
    print("Willkommen!")

print("Programmende")
```

Syntax:

```
if Bedingung:
    # Wird ausgeführt, wenn Bedingung True
    Anweisungen_wenn_wahr
else:
    # Wird ausgeführt, wenn Bedingung False
    Anweisungen_wenn_falsch
# Programm setzt hier fort

[!TIP]

# Beispiel: Gerade oder ungerade Zahl
zahl = int(input("Gib eine Zahl ein: "))

if zahl % 2 == 0:
    print(f"{zahl} ist gerade")
else:
    print(f"{zahl} ist ungerade")

print("Prüfung abgeschlossen")
```

Syntax:

```
if Bedingung1:
    # Wird ausgeführt, wenn Bedingung1 True
    Anweisungen1
```

```

elif Bedingung2:
    # Wird ausgeführt, wenn Bedingung1 False und Bedingung2 True
    Anweisungen2
elif Bedingung3:
    # Wird ausgeführt, wenn Bedingung1 und Bedingung2 False, aber Bedingung3 True
    Anweisungen3
else:
    # Wird ausgeführt, wenn keine der Bedingungen True ist
    Anweisungen_sonst
# Programm setzt hier fort

[!TIP]

# Beispiel: Notenbewertung
punkte = int(input("Erreichte Punktzahl (0-100): "))

if punkte >= 90:
    note = "Sehr gut (1)"
elif punkte >= 80:
    note = "Gut (2)"
elif punkte >= 70:
    note = "Befriedigend (3)"
elif punkte >= 60:
    note = "Ausreichend (4)"
else:
    note = "Nicht bestanden (5)"

print(f"Deine Note: {note}")

[!TIP]

# Beispiel: Rabattberechnung
kunde_typ = "Premium"
bestellwert = 150

if kunde_typ == "Premium":
    if bestellwert >= 100:
        rabatt = 0.20 # 20% Rabatt
        print("20% Premium-Rabatt angewendet")
    else:
        rabatt = 0.10 # 10% Rabatt
        print("10% Premium-Rabatt angewendet")
else:
    if bestellwert >= 100:
        rabatt = 0.05 # 5% Rabatt
        print("5% Standard-Rabatt angewendet")
    else:
        rabatt = 0.0 # Kein Rabatt

```

```

        print("Kein Rabatt")

endpreis = bestellwert * (1 - rabatt)
print(f"Endpreis: {endpreis:.2f} EUR")

Alternative mit logischen Operatoren:

# Besser lesbar: Kombination von Bedingungen
if kunde_typ == "Premium" and bestellwert >= 100:
    rabatt = 0.20
elif kunde_typ == "Premium" and bestellwert < 100:
    rabatt = 0.10
elif kunde_typ != "Premium" and bestellwert >= 100:
    rabatt = 0.05
else:
    rabatt = 0.0

```

Syntax:

```

wert_wenn_wahr if Bedingung else wert_wenn_falsch

[!TIP]

```

```

# Beispiel: Absolute Betrag ohne abs()-Funktion
x = -7

```

```

# Normale if-else-Schreibweise:
if x >= 0:
    absolut = x
else:
    absolut = -x

```

```

# Ternäre Operator-Schreibweise:
absolut = x if x >= 0 else -x

```

```

print(f"Der Absolutbetrag von {x} ist {absolut}")

```

Weitere Beispiele:

```

# Status-Message basierend auf Login
ist_eingeloggt = True
status = "Willkommen zurück!" if ist_eingeloggt else "Bitte einloggen"
print(status) # Ausgabe: Willkommen zurück!

```

```

# Maximum von zwei Zahlen
a, b = 10, 20
maximum = a if a > b else b
print(f"Größere Zahl: {maximum}") # Ausgabe: Größere Zahl: 20

```

```

# Inline-Verwendung in print

```



```

alter = 25
print(f"Du bist {'volljährig' if alter >= 18 else 'minderjährig'}")
# Ausgabe: Du bist volljährig

```

[!WARNING] **Lesbarkeit vs. Kompaktheit:** Der ternäre Operator sollte nur für **einfache, einzeilige Entscheidungen** verwendet werden. Bei komplexen Bedingungen oder mehreren Anweisungen pro Zweig wird der Code schnell unlesbar. In solchen Fällen ist klassisches if-else vorzuziehen.

Schlecht: Unleserlich

```

ergebnis = (x + y) * 2 if (a > b and c < d) or (e == f and g != h) else (x - y) / 3 if z > 0

```

Gut: Klar strukturiert

```

if (a > b and c < d) or (e == f and g != h):
    ergebnis = (x + y) * 2
elif z > 0:
    ergebnis = (x - y) / 3
else:
    ergebnis = 0

```

[!TIP]

Beispiel: Gerüst für spätere Implementierung

```

benutzer_rolle = "Admin"

```

```

if benutzer_rolle == "Admin":
    pass # TODO: Admin-Funktionen implementieren
elif benutzer_rolle == "Moderator":
    pass # TODO: Moderator-Funktionen implementieren
else:
    print("Normaler Benutzer - eingeschränkter Zugriff")

```

[!WARNING] **Fehler 1: Vergessener Doppelpunkt nach if/elif/else**

Falsch:

```

if x > 10
    print("Groß")

```

Fehler: SyntaxError: invalid syntax

Lösung: Doppelpunkt nach der Bedingung nicht vergessen:

Richtig:

```

if x > 10:
    print("Groß")

```

[!WARNING] **Fehler 2: Fehlende oder inkonsistente Einrückung**

```
# Falsch:
if x > 10:
    print("Groß") # Nicht eingerückt!
```

Fehler: IndentationError: expected an indented block

Lösung: Alle Zeilen im if-Block müssen gleich eingerückt sein (4 Leerzeichen):

```
# Richtig:
if x > 10:
    print("Groß")
    print("Sehr groß sogar")
```

[!WARNING] **Fehler 3: Verwechslung von = und ==**

```
# Falsch:
if x = 10: # Zuweisung statt Vergleich!
    print("x ist 10")
```

Fehler: SyntaxError: invalid syntax

Lösung: Verwende == für Vergleiche, = nur für Zuweisungen:

```
# Richtig:
if x == 10:
    print("x ist 10")
```

[!WARNING] **Fehler 4: Falsche Reihenfolge bei if-elif-else**

```
# Problematisch:
punkte = 95
if punkte >= 50:
    print("Bestanden")
elif punkte >= 90:
    print("Sehr gut") # Wird nie erreicht!
```

Problem: Die erste Bedingung `punkte >= 50` ist bereits bei 95 erfüllt, also wird "Bestanden" ausgegeben und die zweite Bedingung nie geprüft.

Lösung: Spezifischere Bedingungen zuerst prüfen:

```
# Richtig:
if punkte >= 90:
    print("Sehr gut")
elif punkte >= 50:
    print("Bestanden")
```

[!WARNING] **Fehler 5: Mehrere separate if statt if-elif-else**

```
# Ineffizient:
if temperatur < 0:
```

```

    print("Gefroren")
if temperatur >= 0 and temperatur < 20:
    print("Kühl")
if temperatur >= 20:
    print("Warm")

```

Problem: Alle drei Bedingungen werden immer geprüft, auch wenn die erste bereits zutrifft.

Lösung: Verwende if-elif-else für sich gegenseitig ausschließende Fälle:

```

# Effizienter:
if temperatur < 0:
    print("Gefroren")
elif temperatur < 20:
    print("Kühl")
else:
    print("Warm")

```

Datei: lessons/V06-Programm-Ablauf-Plaene-Teil2/V06-Programm-Ablauf-Plaene-Teil2_skript.md

Syntax der for-Schleife:

```

for variable in sequenz:
    # Schleifenkörper
    # wird für jedes Element ausgeführt

```

1. `range(stop)`: Erzeugt Zahlen von 0 bis `stop - 1`

```

for i in range(5):
    print(i)  # Gibt 0, 1, 2, 3, 4 aus

```

2. `range(start, stop)`: Erzeugt Zahlen von `start` bis `stop - 1`

```

for i in range(2, 6):
    print(i)  # Gibt 2, 3, 4, 5 aus

```

3. `range(start, stop, step)`: Erzeugt Zahlen von `start` bis `stop - 1` mit Schrittweite `step`

```

for i in range(0, 10, 2):
    print(i)  # Gibt 0, 2, 4, 6, 8 aus

```

[!TIP] Beispiel: Summe der Zahlen von 1 bis 100 berechnen

```

summe = 0
for i in range(1, 101):  # Von 1 bis 100 (101 ist exklusiv)

```

```

    summe += i
print(f"Summe: {summe}") # Ausgabe: Summe: 5050

```

Negative Schrittweiten: Mit negativem `step` kann rückwärts gezählt werden.

```

for i in range(10, 0, -1):
    print(i) # Gibt 10, 9, 8, ..., 1 aus (Countdown)

```

Die `for`-Schleife kann über jedes Iterable iterieren, nicht nur über Zahlenfolgen. Strings sind in Python Sequenzen von Zeichen, über die wir direkt iterieren können.

```

text = "Python"
for buchstabe in text:
    print(buchstabe)
# Ausgabe:
# P
# y
# t
# h
# o
# n

```

[!TIP] Beispiel: Vokale in einem String zählen

```

text = "Informatik Grundlagen"
vokale = "aeiouAEIOU"
anzahl = 0

for zeichen in text:
    if zeichen in vokale:
        anzahl += 1

print(f"Anzahl Vokale: {anzahl}") # Ausgabe: Anzahl Vokale: 8

```

Syntax der while-Schleife:

```

while bedingung:
    # Schleifenkörper
    # wird ausgeführt, solange bedingung True ist

```

[!TIP] Beispiel: Countdown mit `while`

```

countdown = 5
while countdown > 0:
    print(countdown)
    countdown -= 1 # Zähler dekrementieren
print("Start!")
# Ausgabe:
# 5
# 4

```

```
# 3
# 2
# 1
# Start!
```

Beispiel für eine Endlos-Schleife (FEHLER!):

```
# FEHLER: Endlos-Schleife
x = 0
while x < 10:
    print(x)
    # x wird nie verändert → Bedingung bleibt immer True
```

Korrigierte Version:

```
# KORREKT
x = 0
while x < 10:
    print(x)
    x += 1 # x wird erhöht, Bedingung wird irgendwann False
```

[!TIP] **Beispiel: Dieselbe Aufgabe mit for und while**

Aufgabe: Zahlen von 1 bis 5 ausgeben

Mit for:

```
for i in range(1, 6):
    print(i)
```

Mit while:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

[!TIP] **Beispiel: Eingabe einer Zahl zwischen 1 und 10**

```
zahl = 0
while zahl < 1 or zahl > 10:
    eingabe = input("Bitte eine Zahl zwischen 1 und 10 eingeben: ")
    if eingabe.isdigit():
        zahl = int(eingabe)
        if zahl < 1 or zahl > 10:
            print("Fehler: Zahl muss zwischen 1 und 10 liegen.")
    else:
        print("Fehler: Keine gültige Zahl.")

print(f"Danke! Sie haben {zahl} eingegeben.")
```

1. Summation:

```

summe = 0
for zahl in range(1, 11):
    summe += zahl
print(summe)  # 55

```

2. Produkt:

```

produkt = 1
for zahl in range(1, 6):
    produkt *= zahl
print(produkt)  # 120 (Fakultät von 5)

```

3. String-Konkatenation:

```

ergebnis = ""
for i in range(5):
    ergebnis += str(i) + " "
print(ergebnis)  # "0 1 2 3 4 "

```

[TIP] Beispiel: Gerade Zahlen von 1 bis 100 zählen

```

anzahl_gerade = 0
for zahl in range(1, 101):
    if zahl % 2 == 0:
        anzahl_gerade += 1
print(f"Anzahl gerader Zahlen: {anzahl_gerade}")  # 50

```

Obwohl Listen erst in V08 ausführlich behandelt werden, können wir bereits jetzt über einfache Listen iterieren, da Listen ebenfalls Iterables sind.

```

fruechte = ["Apfel", "Banane", "Kirsche"]
for frucht in fruechte:
    print(f"Ich mag {frucht}")
# Ausgabe:
# Ich mag Apfel
# Ich mag Banane
# Ich mag Kirsche

```

[!NOTE] `enumerate(iterable, start=0)` (Built-in) gibt ein Iterable zurück, das Tupel aus Index und Wert erzeugt. Der Parameter `start` legt fest, bei welchem Index die Zählung beginnt (Standard: 0).

```

fruechte = ["Apfel", "Banane", "Kirsche"]
for index, frucht in enumerate(fruechte):
    print(f"{index}: {frucht}")
# Ausgabe:
# 0: Apfel
# 1: Banane
# 2: Kirsche

```

Mit `start=1` kann die Zählung bei 1 beginnen:

```
for nummer, frucht in enumerate(fruechte, start=1):
    print(f"{nummer}. {frucht}")
# Ausgabe:
# 1. Apfel
# 2. Banane
# 3. Kirsche
```

[!WARNING] Fehler 1: Modifikation der Schleifenvariable in `for` wirkt sich nicht auf die Iteration aus

```
# FEHLER: i wird nur lokal geändert
for i in range(5):
    i = 10 # Hat keinen Einfluss auf die nächste Iteration
    print(i) # Gibt immer 10 aus
```

[!WARNING] Fehler 2: `range()` erzeugt keine Liste, sondern ein Range-Objekt

```
zahlen = range(5)
print(zahlen) # Ausgabe: range(0, 5), nicht [0, 1, 2, 3, 4]
```

[!WARNING] Fehler 3: Vergessen, die Schleifenvariable in `while` zu aktualisieren

```
# FEHLER: Endlos-Schleife
i = 0
while i < 5:
    print(i)
    # i wird nicht erhöht → Endlos-Schleife!
```

Datei: lessons/V07-Software-Engineering-KISS-DRY-SRP/V07-Software-Engineering-KISS-DRY-SRP_skript.md

[!TIP]

```
# Schlecht: Unnötig komplex
def check_status(x):
    if x > 0:
        if x < 100:
            if x % 2 == 0:
                return "even_valid"
            else:
                return "odd_valid"
        else:
            return "too_large"
    else:
        return "too_large"
```

```

        return "invalid"

# Gut: Einfach und klar
def check_status(x):
    if x <= 0:
        return "invalid"
    if x >= 100:
        return "too_large"
    return "even_valid" if x % 2 == 0 else "odd_valid"

[!TIP]

# Schlecht: Code-Duplikation
temperatur_celsius_1 = (temperatur_fahrenheit_1 - 32) * 5/9
temperatur_celsius_2 = (temperatur_fahrenheit_2 - 32) * 5/9
temperatur_celsius_3 = (temperatur_fahrenheit_3 - 32) * 5/9

# Gut: Funktion zur Wiederverwendung
def fahrenheit_zu_celsius(fahrenheit):
    """Konvertiert Fahrenheit in Celsius."""
    return (fahrenheit - 32) * 5 / 9

temperatur_celsius_1 = fahrenheit_zu_celsius(temperatur_fahrenheit_1)
temperatur_celsius_2 = fahrenheit_zu_celsius(temperatur_fahrenheit_2)
temperatur_celsius_3 = fahrenheit_zu_celsius(temperatur_fahrenheit_3)

# Noch besser: Mit Schleife
temperaturen_fahrenheit = [temperatur_fahrenheit_1, temperatur_fahrenheit_2, temperatur_fahrenheit_3]
temperaturen_celsius = [fahrenheit_zu_celsius(temp) for temp in temperaturen_fahrenheit]

[!TIP]

# Schlecht: Mehrere Verantwortlichkeiten
def verarbeite_bestellung(bestellung_id):
    # Daten aus Datenbank laden
    bestellung = lade_aus_db(bestellung_id)

    # Validierung
    if bestellung.betrag < 0:
        raise ValueError("Negativer Betrag")

    # Berechnung
    steuern = bestellung.betrag * 0.19
    gesamt = bestellung.betrag + steuern

    # In Datenbank speichern
    speichere_in_db(bestellung_id, gesamt)

```



```

# E-Mail senden
sende_email(bestellung.kunde_email, f"Rechnung: {gesamt}€")

# Logging
schreibe_log(f"Bestellung {bestellung_id} verarbeitet")

# Gut: Aufgeteilt nach Verantwortlichkeiten
def lade_bestellung(bestellung_id):
    """Lädt Bestellung aus Datenbank."""
    return lade_aus_db(bestellung_id)

def validiere_bestellung(bestellung):
    """Validiert Bestellungsdaten."""
    if bestellung.betrag < 0:
        raise ValueError("Negativer Betrag")

def berechne_gesamtbetrag(betrag, steuersatz=0.19):
    """Berechnet Gesamtbetrag inkl. Steuern."""
    steuern = betrag * steuersatz
    return betrag + steuern

def speichere_bestellung(bestellung_id, betrag):
    """Speichert Bestellung in Datenbank."""
    speichere_in_db(bestellung_id, betrag)

def benachrichtige_kunde(kunde_email, betrag):
    """Sendet Bestätigungsmail an Kunden."""
    sende_email(kunde_email, f"Rechnung: {betrag}€")

def protokolliere_verarbeitung(bestellung_id):
    """Protokolliert Verarbeitung."""
    schreibe_log(f"Bestellung {bestellung_id} verarbeitet")

# Koordinierende Funktion (orchestriert die Schritte)
def verarbeite_bestellung(bestellung_id):
    """Koordiniert die Verarbeitung einer Bestellung."""
    bestellung = lade_bestellung(bestellung_id)
    validiere_bestellung(bestellung)
    gesamt = berechne_gesamtbetrag(bestellung.betrag)
    speichere_bestellung(bestellung_id, gesamt)
    benachrichtige_kunde(bestellung.kunde_email, gesamt)
    protokolliere_verarbeitung(bestellung_id)

['TIP']

# Beispiel: Suche nach einem Element
zahlen = [3, 7, 2, 9, 4, 11, 5]

```

```

gesucht = 9

for i, zahl in enumerate(zahlen):
    if zahl == gesucht:
        print(f"Gefunden an Index {i}")
        break # Schleife beenden, da gefunden
    else:
        print("Nicht gefunden") # Wird nur ausgeführt, wenn break NICHT aufgerufen wurde

# Beispiel: Eingabevalidierung mit while
while True:
    eingabe = input("Gib eine positive Zahl ein: ")
    if eingabe.isdigit():
        zahl = int(eingabe)
        if zahl > 0:
            print(f"Danke! Du hast {zahl} eingegeben.")
            break # Gültige Eingabe, Schleife beenden
        else:
            print("Die Zahl muss positiv sein!")
    else:
        print("Das ist keine gültige Zahl!")

['TIP']

# break in verschachtelten Schleifen
for i in range(3):
    print(f"Äußere Schleife: i = {i}")
    for j in range(5):
        if j == 3:
            print(f"Break bei j = {j}")
            break # Beendet nur die innere Schleife
        print(f"  Innere Schleife: j = {j}")
    print("Innere Schleife beendet")

# Ausgabe:
# Äußere Schleife: i = 0
#   Innere Schleife: j = 0
#   Innere Schleife: j = 1
#   Innere Schleife: j = 2
#   Break bei j = 3
# Innere Schleife beendet
# Äußere Schleife: i = 1
# ...

['TIP']

# Beispiel: Nur gerade Zahlen verarbeiten
zahlen = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

```

```

for zahl in zahlen:
    if zahl % 2 != 0:
        continue # Ungerade Zahlen überspringen
    print(f"{zahl} ist gerade")

# Ausgabe:
# 2 ist gerade
# 4 ist gerade
# 6 ist gerade
# 8 ist gerade
# 10 ist gerade

# Beispiel: Eingaben verarbeiten und ungültige überspringen
eingaben = ["12", "abc", "45", "", "78", "xyz"]

for eingabe in eingaben:
    if not eingabe.isdigit():
        print(f"Überspringe ungültige Eingabe: '{eingabe}'")
        continue

    # Nur gültige Zahlen werden hier verarbeitet
    zahl = int(eingabe)
    quadrat = zahl ** 2
    print(f"{zahl}² = {quadrat}")

    ['TIP']

# Demonstration des Unterschieds
print("Mit break:")
for i in range(10):
    if i == 5:
        break
    print(i, end=" ")
print("\nSchleife beendet")
# Ausgabe: 0 1 2 3 4
# Schleife beendet

print("\nMit continue:")
for i in range(10):
    if i == 5:
        continue
    print(i, end=" ")
print("\nSchleife beendet")
# Ausgabe: 0 1 2 3 4 6 7 8 9
# Schleife beendet

```

[!WARNING] Häufige Fehler mit break und continue:

Fehler 1: continue in Kombination mit Inkrement-Operationen bei while-Schleifen vergessen

```
# Problematisch: Endlos-Schleife
i = 0
while i < 10:
    if i % 2 == 0:
        continue # i wird nie inkrementiert!
    print(i)
    i += 1
```

```
# Richtig: Inkrement vor continue
i = 0
while i < 10:
    i += 1
    if i % 2 == 0:
        continue
    print(i)
```

Fehler 2: break verwenden, wenn continue gemeint war (oder umgekehrt)

```
# Falsch: Beendet Schleife beim ersten ungeraden Element
for zahl in [2, 4, 5, 6, 8]:
    if zahl % 2 != 0:
        break # Sollte continue sein
    print(zahl)
# Gibt nur "2 4" aus, statt alle geraden
```

[!TIP]

```
# Beispiel: Primzahl-Test mit else
def ist_primzahl(n):
    """Prüft, ob n eine Primzahl ist."""
    if n < 2:
        return False

    for teiler in range(2, int(n ** 0.5) + 1):
        if n % teiler == 0:
            print(f"{n} ist durch {teiler} teilbar")
            break # Teiler gefunden, keine Primzahl
    else:
        # Wird nur ausgeführt, wenn break NICHT aufgerufen wurde
        print(f"{n} ist eine Primzahl")
        return True

    return False
```

```

ist_primzahl(17)  # 17 ist eine Primzahl
ist_primzahl(18)  # 18 ist durch 2 teilbar

[!TIP]

# Beispiel: Suche in einer Liste
namen = ["Alice", "Bob", "Charlie", "David"]
gesucht = "Eve"

for name in namen:
    if name == gesucht:
        print(f"Gefunden: {name}")
        break
    else:
        print(f"{gesucht} wurde nicht gefunden")

# Beispiel: Validierung aller Elemente
zahlen = [2, 4, 6, 8, 10]

for zahl in zahlen:
    if zahl % 2 != 0:
        print(f"{zahl} ist ungerade - Validierung fehlgeschlagen")
        break
    else:
        print("Alle Zahlen sind gerade - Validierung erfolgreich")

# Beispiel: while mit else
versuche = 0
max_versuche = 3

while versuche < max_versuche:
    passwort = input("Passwort: ")
    versuche += 1

    if passwort == "geheim123":
        print("Zugriff gewährt!")
        break
    else:
        # Wird nur ausgeführt, wenn alle Versuche aufgebraucht sind
        print("Maximale Anzahl an Versuchen erreicht. Zugriff verweigert.")

[!TIP]

# Ohne else-Klausel: Flag-basierter Ansatz
namen = ["Alice", "Bob", "Charlie", "David"]
gesucht = "Eve"
gefunden = False

```

```

for name in namen:
    if name == gesucht:
        print(f"Gefunden: {name}")
        gefunden = True
        break

if not gefunden:
    print(f"{gesucht} wurde nicht gefunden")

# Mit else-Klausel: Eleganter
for name in namen:
    if name == gesucht:
        print(f"Gefunden: {name}")
        break
else:
    print(f"{gesucht} wurde nicht gefunden")

[!WARNING] Häufiger Fehler: Die else-Klausel wird nicht als
"falls Schleife nicht durchlaufen" interpretiert (was falsch ist), son-
dern als "falls break nicht aufgerufen wurde" (was richtig ist). Die
Schleife kann vollständig durchlaufen werden, und else wird trotz-
dem ausgeführt – solange kein break vorkam.

# Missverständnis: else wird ausgeführt!
for i in range(5):
    print(i)
else:
    print("Schleife vollständig durchlaufen") # Wird ausgeführt

[!TIP]

# Einfaches Beispiel: Multiplikationstabelle
for i in range(1, 6):
    for j in range(1, 6):
        produkt = i * j
        print(f"{i} × {j} = {produkt:2d}", end=" ")
    print() # Zeilenumbruch nach jeder Zeile

# Ausgabe:
# 1 × 1 = 1  1 × 2 = 2  1 × 3 = 3  1 × 4 = 4  1 × 5 = 5
# 2 × 1 = 2  2 × 2 = 4  2 × 3 = 6  2 × 4 = 8  2 × 5 = 10
# ...

[!TIP]

# Demonstration der Laufzeit-Explosion
import time

```

```

n = 100

# Einfache Schleife:  $O(n)$ 
start = time.time()
for i in range(n):
    pass
print(f"Einfache Schleife: {time.time() - start:.6f}s")

# Doppelt verschachtelt:  $O(n^2)$ 
start = time.time()
for i in range(n):
    for j in range(n):
        pass
print(f"Doppelt verschachtelt: {time.time() - start:.6f}s")

# Dreifach verschachtelt:  $O(n^3)$ 
start = time.time()
for i in range(n):
    for j in range(n):
        for k in range(n):
            pass
print(f"Dreifach verschachtelt: {time.time() - start:.6f}s")

[!TIP]

# Beispiel: Rechteck aus Sternen zeichnen
hoehe = 5
breite = 8

for zeile in range(hoehe):
    for spalte in range(breite):
        print("*", end="")
    print()

# Ausgabe:
# *****
# *****
# *****
# *****
# *****

# Beispiel: Dreieck aus Zahlen
hoehe = 5

for zeile in range(1, hoehe + 1):
    for zahl in range(1, zeile + 1):
        print(zahl, end="")

```

```

    print()

# Ausgabe:
# 1
# 12
# 123
# 1234
# 12345

# Beispiel: Alle Paare aus zwei Listen
farben = ["Rot", "Grün", "Blau"]
groessen = ["S", "M", "L"]

for farbe in farben:
    for groesse in groessen:
        print(f"{farbe}-{groesse}")

# Ausgabe:
# Rot-S
# Rot-M
# Rot-L
# Grün-S
# ...

[!TIP]

# break bricht nur innere Schleife ab
for i in range(3):
    print(f"Äußere: {i}")
    for j in range(5):
        if j == 2:
            break
        print(f"    Innere: {j}")
# Äußere Schleife läuft weiter

# Um beide Schleifen abubrechen: Flag verwenden
abbruch = False
for i in range(3):
    if abbruch:
        break
    for j in range(5):
        if i == 1 and j == 2:
            abbruch = True
            break
        print(f"({i}, {j})")

# Alternative: Exception (nicht empfohlen für Kontrollfluss)

```



```

class LoopBreak(Exception):
    pass

try:
    for i in range(3):
        for j in range(5):
            if i == 1 and j == 2:
                raise LoopBreak
            print(f"({i}, {j})")
except LoopBreak:
    pass

# Beste Lösung: In Funktion auslagern und return verwenden
def verarbeite_matrix():
    for i in range(3):
        for j in range(5):
            if i == 1 and j == 2:
                return # Beendet beide Schleifen
            print(f"({i}, {j})")

verarbeite_matrix()

```

Die grundlegende Syntax einer List Comprehension ist: `[ausdruck for element in iterable]`. Dies ist äquivalent zu einer for-Schleife, die eine Liste aufbaut:

```

# Mit Schleife
ergebnis = []
for element in iterable:
    ergebnis.append(ausdruck)

# Mit List Comprehension
ergebnis = [ausdruck for element in iterable]

[!TIP]

# Beispiel: Quadratzahlen erstellen
zahlen = [1, 2, 3, 4, 5]

# Mit Schleife
quadrate = []
for zahl in zahlen:
    quadrate.append(zahl ** 2)
print(quadrate) # [1, 4, 9, 16, 25]

# Mit List Comprehension
quadrate = [zahl ** 2 for zahl in zahlen]
print(quadrate) # [1, 4, 9, 16, 25]

```

```

# Direkt mit range()
quadrade = [x ** 2 for x in range(1, 6)]
print(quadrade) # [1, 4, 9, 16, 25]

[!TIP]

# Beispiel: Nur gerade Zahlen
zahlen = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Mit Schleife
gerade = []
for zahl in zahlen:
    if zahl % 2 == 0:
        gerade.append(zahl)
print(gerade) # [2, 4, 6, 8, 10]

# Mit List Comprehension
gerade = [zahl for zahl in zahlen if zahl % 2 == 0]
print(gerade) # [2, 4, 6, 8, 10]

# Beispiel: Quadrate gerader Zahlen
quadrade_gerade = [zahl ** 2 for zahl in zahlen if zahl % 2 == 0]
print(quadrade_gerade) # [4, 16, 36, 64, 100]

# Beispiel: Wörter mit mehr als 3 Buchstaben
woerter = ["hi", "Hallo", "Welt", "zu", "Python"]
lange_woerter = [wort for wort in woerter if len(wort) > 3]
print(lange_woerter) # ['Hallo', 'Welt', 'Python']

[ausdruck for outer in outer_iterable for inner in inner_iterable]
entspricht:

for outer in outer_iterable:
    for inner in inner_iterable:
        ausdruck

[!TIP]

# Beispiel: Alle Kombinationen
buchstaben = ['A', 'B', 'C']
zahlen = [1, 2, 3]

# Mit Schleifen
kombinationen = []
for buchstabe in buchstaben:
    for zahl in zahlen:
        kombinationen.append(f"{buchstabe}{zahl}")
print(kombinationen)

```

```

# ['A1', 'A2', 'A3', 'B1', 'B2', 'B3', 'C1', 'C2', 'C3']

# Mit List Comprehension
kombinationen = [f"{buchstabe}{zahl}" for buchstabe in buchstaben for zahl in zahlen]
print(kombinationen)
# ['A1', 'A2', 'A3', 'B1', 'B2', 'B3', 'C1', 'C2', 'C3']

# Beispiel: Matrix transponieren
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Transponierte Matrix (Zeilen und Spalten tauschen)
transponiert = [[zeile[i] for zeile in matrix] for i in range(len(matrix[0]))]
print(transponiert)
# [[1, 4, 7], [2, 5, 8], [3, 6, 9]]

[!WARNING] Lesbarkeit beachten: Verschachtelte List Comprehensions können schnell unlesbar werden. Als Faustregel gilt: Wenn eine List Comprehension mehr als zwei Ebenen der Verschachtelung hat oder nicht auf eine Zeile passt, ist eine explizite Schleife wahrscheinlich lesbarer.

# Zu komplex: Schwer lesbar
ergebnis = [[x * y for x in range(3) if x > 0] for y in range(5) if y % 2 == 0]

# Besser: Explizite Schleifen
ergebnis = []
for y in range(5):
    if y % 2 == 0:
        zeile = []
        for x in range(3):
            if x > 0:
                zeile.append(x * y)
        ergebnis.append(zeile)

[!TIP]

# Beispiel: Gerade/Ungerade kennzeichnen
zahlen = [1, 2, 3, 4, 5, 6]

kennzeichnung = ["gerade" if zahl % 2 == 0 else "ungerade" for zahl in zahlen]
print(kennzeichnung)
# ['ungerade', 'gerade', 'ungerade', 'gerade', 'ungerade', 'gerade']

# Beispiel: Werte begrenzen (Clipping)

```

```

werte = [-5, 10, 3, 25, -2, 15]
maximum = 20
minimum = 0

begrenzt = [max(minimum, min(wert, maximum)) for wert in werte]
print(begrenzt)
# [0, 10, 3, 20, 0, 15]

# Mit ternärem Operator für klarere Logik
begrenzt = [wert if 0 <= wert <= 20 else (0 if wert < 0 else 20) for wert in werte]
print(begrenzt)
# [0, 10, 3, 20, 0, 15]

[!WARNING] Häufiger Fehler: List Comprehensions für Seiteneffekte verwenden

# Schlecht: List Comprehension nur für Seiteneffekt
zahlen = [1, 2, 3, 4, 5]
[print(zahl) for zahl in zahlen] # Erstellt nutzlose Liste von None-Werten

# Gut: Normale Schleife für Seiteneffekte
for zahl in zahlen:
    print(zahl)

[!TIP]

# Set Comprehension (erzeugt Set statt Liste)
zahlen = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
eindeutige_quadrate = {zahl ** 2 for zahl in zahlen}
print(eindeutige_quadrate) # {1, 4, 9, 16} (automatisch eindeutig)

# Dictionary Comprehension (erzeugt Dictionary)
woerter = ["Hallo", "Welt", "Python"]
laengen = {wort: len(wort) for wort in woerter}
print(laengen) # {'Hallo': 5, 'Welt': 4, 'Python': 6}

[!WARNING] Fehler 1: Schleifenvariable in List Comprehension außerhalb verwenden

# Problem: Schleifenvariable "leakt" nach List Comprehension
quadrate = [x ** 2 for x in range(5)]
print(x) # Überraschung: x existiert und hat Wert 4 (letzter Wert)
# In Python 3 ist dies behoben für List Comprehensions

# Bei normalen Schleifen existiert die Variable weiterhin
for i in range(5):
    pass
print(i) # 4 (existiert weiterhin)

```

[!WARNING] **Fehler 2:** Modifikation der iterierten Liste während der Iteration

```
# Gefährlich: Liste während Iteration ändern
zahlen = [1, 2, 3, 4, 5]
for zahl in zahlen:
    if zahl % 2 == 0:
        zahlen.remove(zahl) # Ändert Liste während Iteration!
print(zahlen) # Unvorhersehbares Ergebnis: [1, 3, 5] oder [1, 2, 3, 5]?
```

Lösung: Iteriere über eine Kopie oder erstelle eine neue Liste mit den gewünschten Elementen.

```
# Richtig: Über Kopie iterieren
zahlen = [1, 2, 3, 4, 5]
for zahl in zahlen[:]: #[:] erstellt flache Kopie
    if zahl % 2 == 0:
        zahlen.remove(zahl)
print(zahlen) # [1, 3, 5]
```

```
# Oder: Neue Liste mit List Comprehension
zahlen = [1, 2, 3, 4, 5]
zahlen = [zahl for zahl in zahlen if zahl % 2 != 0]
print(zahlen) # [1, 3, 5]
```

[!WARNING] **Fehler 3:** Unendliche Schleifen mit `while True` ohne `break`

```
# Gefährlich: Keine Abbruchbedingung
while True:
    print("Das läuft ewig...")
# Kein break!
```

Lösung: Stelle immer sicher, dass eine Abbruchbedingung existiert.

```
# Richtig: break bei Bedingung
zaehler = 0
while True:
    print(f"Durchlauf {zaehler}")
    zaehler += 1
    if zaehler >= 10:
        break
```

```
# Oder: Explizite Bedingung
zaehler = 0
while zaehler < 10:
    print(f"Durchlauf {zaehler}")
    zaehler += 1
```

Datei: lessons/V08-Listen-und-Datenstrukturen-Teil1/V08-Listen-und-Datenstrukturen-Teil1_skript.md

Struktur eines Knotens:

```
class Node:
    def __init__(self, data):
        self.data = data # Datenwert
        self.next = None # Verweis auf nächsten Knoten
```

Struktur eines Knotens:

```
class DNode:
    def __init__(self, data):
        self.data = data # Datenwert
        self.next = None # Verweis vorwärts
        self.prev = None # Verweis rückwärts
```

[!TIP]

Leere Liste erstellen

```
leere_liste = []
```

```
auch_leer = list()
```

Liste mit Elementen

```
zahlen = [1, 2, 3, 4, 5]
```

```
fruechte = ["Apfel", "Banane", "Kirsche"]
```

Gemischte Typen (möglich, aber nicht empfohlen)

```
gemischt = [42, "Hallo", 3.14, True, [1, 2]]
```

Liste aus String erstellen

```
buchstaben = list("Python") # ['P', 'y', 't', 'h', 'o', 'n']
```

Liste aus range erstellen

```
zahlenfolge = list(range(10)) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

[!TIP]

```
fruechte = ["Apfel", "Banane", "Kirsche", "Datteln"]
```

Positiver Index (von vorne)

```
print(fruechte[0]) # "Apfel" (erstes Element)
```

```
print(fruechte[2]) # "Kirsche"
```

Negativer Index (von hinten)

```
print(fruechte[-1]) # "Datteln" (letztes Element)
```

```
print(fruechte[-2]) # "Kirsche"
```

```

# Index außerhalb des Bereichs → IndexError
# print(fruechte[10]) # IndexError: list index out of range

[!WARNING] IndexError vermeiden: Prüfe vor Zugriff, ob der
Index gültig ist:

if 0 <= index < len(liste):
    element = liste[index]

[!TIP]

fruechte = ["Apfel", "Banane"]

fruechte.append("Kirsche")
print(fruechte) # ['Apfel', 'Banane', 'Kirsche']

# Auch Listen können hinzugefügt werden (als einzelnes Element!)
fruechte.append(["Erdbeere", "Himbeere"])
print(fruechte) # ['Apfel', 'Banane', 'Kirsche', ['Erdbeere', 'Himbeere']]

[!TIP]

zahlen = [1, 2, 4, 5]

# Füge 3 an Index 2 ein
zahlen.insert(2, 3)
print(zahlen) # [1, 2, 3, 4, 5]

# Am Anfang einfügen
zahlen.insert(0, 0)
print(zahlen) # [0, 1, 2, 3, 4, 5]

# Index größer als Länge → fügt am Ende ein
zahlen.insert(100, 99)
print(zahlen) # [0, 1, 2, 3, 4, 5, 99]

[!TIP]

liste1 = [1, 2, 3]
liste2 = [4, 5, 6]

# extend() fügt Elemente einzeln hinzu
liste1.extend(liste2)
print(liste1) # [1, 2, 3, 4, 5, 6]

# Vergleich mit append():
liste3 = [1, 2, 3]
liste3.append(liste2)
print(liste3) # [1, 2, 3, [4, 5, 6]] (Liste als Element!)

```

```

# extend() mit anderen Iterables
liste1.extend("AB")
print(liste1)  # [1, 2, 3, 4, 5, 6, 'A', 'B']

[!TIP]

liste1 = [1, 2, 3]
liste2 = [4, 5, 6]

# Verkettung erzeugt neue Liste
neue_liste = liste1 + liste2
print(neue_liste)  # [1, 2, 3, 4, 5, 6]
print(liste1)      # [1, 2, 3] (unverändert)

# Mehrfache Verkettung
result = [1] + [2, 3] + [4, 5]
print(result)  # [1, 2, 3, 4, 5]

[!TIP]

fruechte = ["Apfel", "Banane", "Kirsche", "Banane"]

fruechte.remove("Banane")
print(fruechte)  # ['Apfel', 'Kirsche', 'Banane'] (nur erste Banane entfernt)

# Entfernen eines nicht vorhandenen Elements → ValueError
# fruechte.remove("Orange")  # ValueError: list.remove(x): x not in list

[!WARNING] ValueError abfangen: Prüfe vorher, ob das Element existiert:

if "Orange" in fruechte:
    fruechte.remove("Orange")

[!TIP]

zahlen = [10, 20, 30, 40, 50]

# Letztes Element entfernen
letztes = zahlen.pop()
print(letztes)  # 50
print(zahlen)  # [10, 20, 30, 40]

# Element an Index 1 entfernen
element = zahlen.pop(1)
print(element)  # 20
print(zahlen)  # [10, 30, 40]

# Stack-Verhalten simulieren
stack = [1, 2, 3]

```



```

stack.append(4)  # Push
top = stack.pop()  # Pop → 4

[!TIP]

zahlen = [1, 2, 3, 4, 5]
zahlen.clear()
print(zahlen)  # []
print(len(zahlen))  # 0

# Alternative: Neuzuweisung
zahlen = []  # Erzeugt neue leere Liste

[!TIP]

zahlen = [0, 1, 2, 3, 4, 5]

# Einzelnes Element löschen
del zahlen[0]
print(zahlen)  # [1, 2, 3, 4, 5]

# Slice löschen
del zahlen[1:3]
print(zahlen)  # [1, 4, 5]

# Gesamte Variable löschen (Variable existiert danach nicht mehr)
del zahlen
# print(zahlen)  # NameError: name 'zahlen' is not defined

[!TIP]

fruechte = ["Apfel", "Banane", "Kirsche", "Banane"]

index = fruechte.index("Banane")
print(index)  # 1 (erstes Vorkommen)

# Suche ab Index 2
index2 = fruechte.index("Banane", 2)
print(index2)  # 3 (zweites Vorkommen)

# Element nicht vorhanden → ValueError
# fruechte.index("Orange")  # ValueError

[!TIP]

zahlen = [1, 2, 3, 2, 4, 2, 5]

anzahl = zahlen.count(2)
print(anzahl)  # 3

```

```

# Element nicht vorhanden
print(zahlen.count(99)) # 0

[!TIP]

fruechte = ["Apfel", "Banane", "Kirsche"]

if "Banane" in fruechte:
    print("Banane ist vorhanden")

if "Orange" not in fruechte:
    print("Orange ist nicht vorhanden")

# Auch für Bedingungen in Comprehensions
ergebnis = [f for f in fruechte if "a" in f]
print(ergebnis) # ['Banane']

[!TIP]

zahlen = [3, 1, 4, 1, 5, 9, 2]

# Aufsteigend sortieren
zahlen.sort()
print(zahlen) # [1, 1, 2, 3, 4, 5, 9]

# Absteigend sortieren
zahlen.sort(reverse=True)
print(zahlen) # [9, 5, 4, 3, 2, 1, 1]

# Strings sortieren (lexikographisch)
woerter = ["Zebra", "Apfel", "Banane"]
woerter.sort()
print(woerter) # ['Apfel', 'Banane', 'Zebra']

# Nach Länge sortieren (key-Funktion)
woerter.sort(key=len)
print(woerter) # ['Apfel', 'Zebra', 'Banane']

[!TIP]

zahlen = [3, 1, 4, 1, 5]

sortiert = sorted(zahlen)
print(sortiert) # [1, 1, 3, 4, 5]
print(zahlen) # [3, 1, 4, 1, 5] (Original unverändert)

# String sortieren (gibt Liste zurück!)
buchstaben = sorted("python")
print(buchstaben) # ['h', 'n', 'o', 'p', 't', 'y']

```

```

# Zurück zu String
sortiert_string = ''.join(sorted("python"))
print(sortiert_string) # "hnopty"

[!TIP]

zahlen = [1, 2, 3, 4, 5]

zahlen.reverse()
print(zahlen) # [5, 4, 3, 2, 1]

# Alternative mit Slicing (erzeugt neue Liste)
zahlen2 = [1, 2, 3, 4, 5]
umgekehrt = zahlen2[::-1]
print(umgekehrt) # [5, 4, 3, 2, 1]
print(zahlen2) # [1, 2, 3, 4, 5] (Original unverändert)

[!TIP]

zahlen = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Grundlegendes Slicing
print(zahlen[2:5]) # [2, 3, 4] (Index 2 bis 4)
print(zahlen[:5]) # [0, 1, 2, 3, 4] (Anfang bis 4)
print(zahlen[5:]) # [5, 6, 7, 8, 9] (Ab 5 bis Ende)
print(zahlen[:]) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] (Komplette Liste kopieren)

# Mit Schrittweite
print(zahlen[::2]) # [0, 2, 4, 6, 8] (Jedes zweite Element)
print(zahlen[1::2]) # [1, 3, 5, 7, 9] (Jedes zweite, ab Index 1)
print(zahlen[::3]) # [0, 3, 6, 9] (Jedes dritte)

# Negative Schrittweite (rückwärts)
print(zahlen[::-1]) # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] (Liste umkehren)
print(zahlen[7:2:-1]) # [7, 6, 5, 4, 3] (Von 7 bis 3, rückwärts)

# Negative Indizes
print(zahlen[-3:]) # [7, 8, 9] (Letzte 3 Elemente)
print(zahlen[:-3]) # [0, 1, 2, 3, 4, 5, 6] (Alle außer letzte 3)

[!WARNING] Slicing erstellt Shallow Copy: Bei Listen mit
veränderbaren Objekten werden nur die Referenzen kopiert:

original = [[1, 2], [3, 4]]
kopie = original[:]

kopie[0][0] = 99 # Ändert auch das Original!
print(original) # [[99, 2], [3, 4]]

```

```

[!TIP]

zahlen = [0, 1, 2, 3, 4, 5]

# Bereich ersetzen
zahlen[1:4] = [10, 20, 30]
print(zahlen)  # [0, 10, 20, 30, 4, 5]

# Bereich mit weniger Elementen ersetzen (Liste schrumpft)
zahlen[1:4] = [99]
print(zahlen)  # [0, 99, 4, 5]

# Bereich löschen
zahlen[1:3] = []
print(zahlen)  # [0, 5]

# Einfügen ohne Löschen (leerer Slice)
zahlen[1:1] = [1, 2, 3, 4]
print(zahlen)  # [0, 1, 2, 3, 4, 5]

[!TIP]

# Tupel erstellen
punkt = (10, 20)
farben = ("Rot", "Grün", "Blau")

# Ohne Klammern (Tuple Packing)
koordinaten = 5, 10, 15
print(type(koordinaten))  # <class 'tuple'>

# Ein-Element-Tupel (Komma erforderlich!)
single = (42,)  # Tupel
kein_tupel = 42  # Integer!
print(type(single))  # <class 'tuple'>
print(type(kein_tupel))  # <class 'int'>

# Leeres Tupel
leer = ()
auch_leer = tuple()

# Zugriff wie bei Listen
print(punkt[0])  # 10
print(farben[-1])  # "Blau"

# Slicing funktioniert
print(farben[1:])  # ('Grün', 'Blau')

[!TIP]

```

```

# Tupel als Dictionary-Keys
positionen = {
    (0, 0): "Start",
    (10, 5): "Checkpoint",
    (20, 20): "Ziel"
}

# Multiple Return Values
def min_max(zahlen):
    return min(zahlen), max(zahlen)

minimum, maximum = min_max([3, 1, 4, 1, 5])
print(minimum, maximum) # 1 5

[!TIP]

# Einfaches Unpacking
punkt = (10, 20)
x, y = punkt
print(x, y) # 10 20

# Liste unpacken
farben = ["Rot", "Grün", "Blau"]
rot, gruen, blau = farben

# Variablen tauschen (sehr pythonisch!)
a, b = 5, 10
a, b = b, a # Tausch ohne temporäre Variable
print(a, b) # 10 5

# Unpacking mit * (Rest-Elemente sammeln)
zahlen = [1, 2, 3, 4, 5]
erste, *rest, letzte = zahlen
print(erste) # 1
print(rest) # [2, 3, 4]
print(letzte) # 5

# Nur interessante Elemente extrahieren
_, zweite, *_ , vorletzte, _ = [1, 2, 3, 4, 5, 6]
print(zweite, vorletzte) # 2 5

[!TIP]

liste1 = [1, 2, 3]
liste2 = liste1 # Aliasing, keine Kopie!

liste2.append(4)
print(liste1) # [1, 2, 3, 4] (auch liste1 ist geändert!)

```

```

print(liste2)  # [1, 2, 3, 4]

# Prüfen, ob selbes Objekt
print(liste1 is liste2)  # True
print(id(liste1) == id(liste2))  # True

[!TIP]

original = [1, 2, 3]

# Methode 1: Slicing
kopie1 = original[:]

# Methode 2: list()-Konstruktor
kopie2 = list(original)

# Methode 3: .copy()-Methode
kopie3 = original.copy()

# Alle drei sind unabhängige Kopien
kopie1.append(4)
print(original)  # [1, 2, 3] (unverändert)
print(kopie1)    # [1, 2, 3, 4]

# Prüfen: Verschiedene Objekte
print(original is kopie1)  # False

[!WARNING] Shallow Copy vs. Deep Copy: Die oben genannten
Methoden erstellen nur Shallow Copies. Bei Listen mit veränder-
baren Objekten werden nur die Referenzen kopiert:

original = [[1, 2], [3, 4]]
kopie = original[:]

kopie[0][0] = 99
print(original)  # [[99, 2], [3, 4]] (auch original geändert!)

Für vollständige Kopien (Deep Copy):

import copy
kopie = copy.deepcopy(original)

kopie[0][0] = 99
print(original)  # [[1, 2], [3, 4]] (unverändert)

[!TIP]

# Quadratzahlen
quadrates = [x**2 for x in range(10)]
print(quadrates)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

```

```

# Mit Bedingung (Filter)
gerade = [x for x in range(20) if x % 2 == 0]
print(gerade) # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# String-Verarbeitung
woerter = ["Hallo", "Welt", "Python"]
grossbuchstaben = [w.upper() for w in woerter]
print(grossbuchstaben) # ['HALLO', 'WELT', 'PYTHON']

# Verschachtelt
matrix = [[i*j for j in range(3)] for i in range(3)]
print(matrix) # [[0, 0, 0], [0, 1, 2], [0, 2, 4]]

# Komplexere Ausdrücke
werte = [1, 2, 3, 4, 5]
ergebnis = [x*2 if x % 2 == 0 else x for x in werte]
print(ergebnis) # [1, 4, 3, 8, 5]

['TIP']

zahlen = [1, 2, 3, 4, 5]
print(len(zahlen)) # 5

# Leere Liste
print(len([])) # 0

['TIP']

zahlen = [1, 2, 3, 4, 5]
print(sum(zahlen)) # 15

# Mit Startwert
print(sum(zahlen, 10)) # 25

# Durchschnitt berechnen
durchschnitt = sum(zahlen) / len(zahlen)
print(durchschnitt) # 3.0

['TIP']

zahlen = [3, 1, 4, 1, 5, 9, 2]

print(min(zahlen)) # 1
print(max(zahlen)) # 9

# Mit Strings (lexikographisch)
woerter = ["Zebra", "Apfel", "Banane"]
print(min(woerter)) # "Apfel"

```

```

print(max(woerter)) # "Zebra"

# Mit key-Funktion (nach Länge)
print(min(woerter, key=len)) # "Zebra" (oder "Apfel", beide 5 Zeichen)
print(max(woerter, key=len)) # "Banane" (6 Zeichen)

[!TIP]

# all() - Alle müssen True sein
zahlen = [2, 4, 6, 8]
print(all(x % 2 == 0 for x in zahlen)) # True (alle gerade)

zahlen2 = [2, 3, 4]
print(all(x % 2 == 0 for x in zahlen2)) # False (3 ist ungerade)

# any() - Mindestens eines muss True sein
zahlen3 = [1, 3, 5, 6]
print(any(x % 2 == 0 for x in zahlen3)) # True (6 ist gerade)

zahlen4 = [1, 3, 5]
print(any(x % 2 == 0 for x in zahlen4)) # False (alle ungerade)

# Praktisches Beispiel: Passwort-Validierung
passwort = "Abc123!"
hat_grossbuchstabe = any(c.isupper() for c in passwort)
hat_ziffer = any(c.isdigit() for c in passwort)
lang_genug = len(passwort) >= 8

gueltig = all([hat_grossbuchstabe, hat_ziffer, lang_genug])
print(gueltig) # False (zu kurz)

[!TIP]

namen = ["Alice", "Bob", "Charlie"]
alter = [25, 30, 35]
staedte = ["Berlin", "Hamburg", "München"]

# Paralleles Durchlaufen
for name, age, stadt in zip(namen, alter, staedte):
    print(f"{name} ({age}) aus {stadt}")
# Ausgabe:
# Alice (25) aus Berlin
# Bob (30) aus Hamburg
# Charlie (35) aus München

# zip() zu Liste konvertieren
paare = list(zip(namen, alter))
print(paare) # [('Alice', 25), ('Bob', 30), ('Charlie', 35)]

```



```

# Unterschiedliche Längen (stoppt bei kürzestem)
liste1 = [1, 2, 3]
liste2 = ['a', 'b']
print(list(zip(liste1, liste2))) # [(1, 'a'), (2, 'b')]

# "Unzip" mit zip(...)
paare = [(1, 'a'), (2, 'b'), (3, 'c')]
zahlen, buchstaben = zip(*paare)
print(zahlen) # (1, 2, 3)
print(buchstaben) # ('a', 'b', 'c')

[!TIP]

# Stack mit append() und pop()
stack = []

# Push-Operationen
stack.append(1)
stack.append(2)
stack.append(3)
print(stack) # [1, 2, 3]

# Pop-Operationen (LIFO)
top = stack.pop()
print(top) # 3
print(stack) # [1, 2]

# Peek (ohne Entfernen)
if stack:
    top = stack[-1]
    print(top) # 2

# Prüfen, ob leer
is_empty = len(stack) == 0
# oder (idiomatischer):
is_empty = not stack

[!WARNING] Performance-Problem: .pop(0) ist  $O(n)$ , da
alle Elemente nach links verschoben werden müssen. Für ef-
fiziente Queues verwende collections.deque (wird in späteren
Vorlesungen behandelt):

from collections import deque

queue = deque()
queue.append(1) # Enqueue:  $O(1)$ 
first = queue.popleft() # Dequeue:  $O(1)$ 

```

[!TIP]

Queue mit Listen (NICHT empfohlen für große Datenmengen!)

```
queue = []
```

Enqueue-Operationen

```
queue.append(1)
```

```
queue.append(2)
```

```
queue.append(3)
```

```
print(queue) # [1, 2, 3]
```

Dequeue-Operationen (FIFO)

```
first = queue.pop(0) # Ineffizient:  $O(n)$ !
```

```
print(first) # 1
```

```
print(queue) # [2, 3]
```

Peek (ohne Entfernen)

```
if queue:
```

```
    front = queue[0]
```

```
    print(front) # 2
```

[!WARNING] **Fehler 2: Liste während Iteration modifizieren**

Niemals eine Liste modifizieren, während man über sie iteriert!

Falsch:

```
zahlen = [1, 2, 3, 4, 5]
```

```
for zahl in zahlen:
```

```
    if zahl % 2 == 0:
```

```
        zahlen.remove(zahl) # Überspringt Elemente!
```

Richtig (Methode 1: Über Kopie iterieren):

```
zahlen = [1, 2, 3, 4, 5]
```

```
for zahl in zahlen[:]: # Iteriere über Kopie
```

```
    if zahl % 2 == 0:
```

```
        zahlen.remove(zahl)
```

Richtig (Methode 2: List Comprehension):

```
zahlen = [1, 2, 3, 4, 5]
```

```
zahlen = [z for z in zahlen if z % 2 != 0]
```

[!WARNING] **Fehler 3: Aliasing statt Kopieren**

Zuweisung erstellt keine Kopie, sondern nur eine Referenz.

Falsch:

```
liste1 = [1, 2, 3]
```

```
liste2 = liste1 # Nur Referenz!
```

```
liste2.append(4)
print(liste1)  # [1, 2, 3, 4] (auch liste1 geändert!)
```

Richtig:

```
liste1 = [1, 2, 3]
liste2 = liste1[:]  # oder list(liste1) oder liste1.copy()
liste2.append(4)
print(liste1)  # [1, 2, 3] (unverändert)
```

[!WARNING] **Fehler 4: Index außerhalb des Bereichs**

Lösung: Prüfe Länge oder verwende `.get()` bei Dictionaries (V08, Teil 2):

```
if index < len(liste):
    element = liste[index]
```

[!WARNING] **Fehler 5: Verwechslung append() vs. extend()**

```
liste = [1, 2, 3]
liste.append([4, 5])
print(liste)  # [1, 2, 3, [4, 5]] (Liste als Element!)
```

```
liste = [1, 2, 3]
liste.extend([4, 5])
print(liste)  # [1, 2, 3, 4, 5] (Elemente einzeln hinzugefügt)
```

Datei: lessons/V09-Listen-und-Datenstrukturen-Teil2/V09-Listen-und-Datenstrukturen-Teil2__skript.md

[!TIP]

```
# ValueError-Beispiel:
zahl = int("abc")  # ValueError: invalid literal for int()
```

[!TIP]

```
# FileNotFoundError-Beispiel:
with open("nicht_vorhanden.txt") as datei:
    inhalt = datei.read()
# FileNotFoundError: [Errno 2] No such file or directory
```

[!NOTE] **try-except-Block:** Ein Kontrollstrukt, das es ermöglicht, potenziell fehlerhafte Code-Abschnitte zu überwachen und auf Fehler zu reagieren, ohne dass das Programm abstürzt.

Syntax:

```
try:
    # Code, der eine Exception werfen könnte
```

```

        risikoreicher_code()
    except ExceptionType:
        # Code, der ausgeführt wird, wenn ExceptionType auftritt
        fehlerbehandlung()

        ['TIP']

try:
    zahl = int(input("Gib eine Zahl ein: "))
    ergebnis = 100 / zahl
    print(f"Ergebnis: {ergebnis}")
except ValueError:
    print("Fehler: Keine gültige Zahl eingegeben!")
except ZeroDivisionError:
    print("Fehler: Division durch Null ist nicht erlaubt!")

    ['TIP']

def sicheres_liste_zugriff(liste, index):
    """Greift sicher auf ein Listenelement zu."""
    try:
        return liste[index]
    except IndexError:
        print(f"Fehler: Index {index} existiert nicht.")
        return None
    except TypeError:
        print("Fehler: Index muss eine Ganzzahl sein.")
        return None

# Anwendung:
zahlen = [10, 20, 30]
print(sicheres_liste_zugriff(zahlen, 1))      # 20
print(sicheres_liste_zugriff(zahlen, 10))    # Fehler: Index 10 existiert nicht. / None
print(sicheres_liste_zugriff(zahlen, "a"))    # Fehler: Index muss eine Ganzzahl sein. / None

    ['TIP']

try:
    wert = int(input("Zahl: "))
    ergebnis = 100 / wert
except (ValueError, ZeroDivisionError):
    print("Ungültige Eingabe oder Division durch Null!")

    ['TIP']

try:
    datei = open("config.txt")
    inhalt = datei.read()
except FileNotFoundError as e:
    print(f"Datei konnte nicht geöffnet werden: {e}")

```

```

    print(f"Dateipfad: {e.filename}")
except PermissionError as e:
    print(f"Keine Berechtigung zum Zugriff: {e}")

[!NOTE] else-Klausel: Ein optionaler Block nach allen except-
Klauseln, der nur ausgeführt wird, wenn keine Exception aufgetreten
ist.

```

Syntax:

```

try:
    code()
except ExceptionType:
    fehlerbehandlung()
else:
    erfolgs_code()  # Nur wenn keine Exception auftrat

[!TIP]

def datei_verarbeiten(dateiname):
    """Verarbeitet eine Datei mit explizitem Erfolgs-Handling."""
    try:
        datei = open(dateiname, 'r')
    except FileNotFoundError:
        print(f"Fehler: Datei '{dateiname}' nicht gefunden.")
    else:
        # Dieser Block wird nur ausgeführt, wenn open() erfolgreich war
        inhalt = datei.read()
        print(f"Datei erfolgreich gelesen: {len(inhalt)} Zeichen")
        datei.close()

```

```
datei_verarbeiten("test.txt")
```

[!NOTE] **finally-Klausel:** Ein optionaler Block, der **garantiert** ausgeführt wird, egal ob eine Exception auftrat oder nicht. Selbst wenn ein **return**-Statement im **try** oder **except** steht, wird **finally** vorher ausgeführt.

Syntax:

```

try:
    code()
except ExceptionType:
    fehlerbehandlung()
finally:
    aufräumen()  # Wird IMMER ausgeführt

[!TIP]

def datei_sicher_lesen(dateiname):
    """Liest eine Datei und stellt sicher, dass sie geschlossen wird."""

```

```

datei = None
try:
    datei = open(dateiname, 'r')
    inhalt = datei.read()
    return inhalt
except FileNotFoundError:
    print("Datei nicht gefunden.")
    return None
finally:
    # Wird immer ausgeführt, auch wenn return im try steht
    if datei:
        datei.close()
        print("Datei wurde geschlossen.")

ergebnis = datei_sicher_lesen("daten.txt")

[!TIP]

def sichere_division(a, b):
    """Führt eine Division mit vollständiger Fehlerbehandlung durch."""
    print("Starte Division...")
    try:
        ergebnis = a / b
    except ZeroDivisionError:
        print("Fehler: Division durch Null!")
        return None
    except TypeError:
        print("Fehler: Ungültige Datentypen!")
        return None
    else:
        print(f"Division erfolgreich: {a} / {b} = {ergebnis}")
        return ergebnis
    finally:
        print("Division-Operation beendet.")

# Tests:
print(sichere_division(10, 2))
# Output:
# Starte Division...
# Division erfolgreich: 10 / 2 = 5.0
# Division-Operation beendet.
# 5.0

print(sichere_division(10, 0))
# Output:
# Starte Division...
# Fehler: Division durch Null!

```

```

# Division-Operation beendet.
# None

[!TIP]

def setze_alter(alter):
    """Setzt das Alter, wirft ValueError bei ungültiger Eingabe."""
    if not isinstance(alter, int):
        raise TypeError("Alter muss eine Ganzzahl sein!")
    if alter < 0:
        raise ValueError("Alter kann nicht negativ sein!")
    if alter > 150:
        raise ValueError("Alter ist unrealistisch hoch!")

    print(f"Alter gesetzt: {alter}")
    return alter

# Anwendung:
try:
    setze_alter(25)          # OK
    setze_alter(-5)         # ValueError
except ValueError as e:
    print(f"Ungültiges Alter: {e}")

[!TIP]

def verarbeite_daten(datei):
    """Verarbeitet Daten, loggt Fehler, aber wirft sie erneut."""
    try:
        inhalt = open(datei).read()
        return inhalt
    except FileNotFoundError as e:
        print(f"LOG: Datei {datei} nicht gefunden.")
        raise # Wirft die Exception erneut

try:
    verarbeite_daten("nicht_vorhanden.txt")
except FileNotFoundError:
    print("Höhere Ebene behandelt den Fehler.")

[!NOTE] Benutzerdefinierte Exception: Eine eigene Exception-
Klasse, die von Exception oder einem anderen Exception-Typ erbt.
Sie kann zusätzliche Attribute und Methoden enthalten.

Syntax:

class MeinFehler(Exception):
    """Beschreibung des Fehlers."""
    pass

```

[!TIP]

```
class UngueltigeEmailError(Exception):
    """Exception für ungültige E-Mail-Adressen."""
    def __init__(self, email, nachricht="E-Mail-Format ungültig"):
        self.email = email
        self.nachricht = nachricht
        super().__init__(f"{nachricht}: {email}")

def validiere_email(email):
    """Validiert eine E-Mail-Adresse (vereinfacht)."""
    if "@" not in email or "." not in email:
        raise UngueltigeEmailError(email)
    return True

# Anwendung:
try:
    validiere_email("benutzer@example.com") # OK
    validiere_email("ungueltig")            # UngueltigeEmailError
except UngueltigeEmailError as e:
    print(f"Fehler: {e}")
    print(f"Problematische E-Mail: {e.email}")
```

[!WARNING] Fehler 1: Zu breite except-Klauseln

Problem:

```
try:
    code()
except:
    print("Ein Fehler ist aufgetreten.")
```

Warum problematisch: Fängt alle Exceptions ab, auch KeyboardInterrupt (Strg+C) und SystemExit. Verhindert kontrolliertes Beenden und erschwert Debugging.

Lösung: Verwende spezifische Exception-Typen oder mindestens `except Exception::`

```
try:
    code()
except Exception as e:
    print(f"Fehler: {e}")
```

[!WARNING] Fehler 2: Exceptions verschlucken

Problem:

```
try:
    kritischer_code()
```



```
except ValueError:
    pass # Fehler wird ignoriert
```

Warum problematisch: Der Fehler wird komplett unterdrückt.
Niemand erfährt, dass etwas schiefgelaufen ist.

Lösung: Mindestens loggen, oder den Benutzer informieren:

```
try:
    kritischer_code()
except ValueError as e:
    print(f"Warnung: {e}")
    # Oder: logging.error(f"Fehler: {e}")

[!WARNING] Fehler 3: Ausnahmen für Kontrollfluss miss-
brauchen
```

Problem:

```
try:
    wert = dictionary[key]
except KeyError:
    wert = default_wert
```

Datei: lessons/V10-Laufzeitanalyse-und-Algorithmik/V10-
Laufzeitanalyse-und-Algorithmik_skript.md

Beispiel 1: Einfache Schleife

```
def summe_bis_n(n):
    total = 0           # 1 Operation
    for i in range(n): # n Iterationen
        total += i      # 2 Operationen pro Iteration (Addition, Zuweisung)
    return total        # 1 Operation
```

Beispiel 2: Verschachtelte Schleifen

```
def summe_matrix(matrix):
    total = 0
    for zeile in matrix:      # n Iterationen (wenn Matrix n×n)
        for element in zeile: # n Iterationen pro Zeile
            total += element   # 2 Operationen
    return total
```

Beispiel 3: Binäre Suche (Halbierungsstrategie)

```
def binaere_suche(sortierte_liste, ziel):
    links, rechts = 0, len(sortierte_liste) - 1
```

```

while links <= rechts:
    mitte = (links + rechts) // 2
    if sortierte_liste[mitte] == ziel:
        return mitte
    elif sortierte_liste[mitte] < ziel:
        links = mitte + 1
    else:
        rechts = mitte - 1
return -1

```

Beispiel: Fibonacci-Zahlen (naiv rekursiv)

```

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

```

Bubble Sort vergleicht benachbarte Elemente und vertauscht sie, wenn sie in falscher Reihenfolge sind. Dieser Prozess wird wiederholt, bis keine Vertauschungen mehr nötig sind.

```

def bubble_sort(liste):
    n = len(liste)
    for i in range(n):
        for j in range(0, n-i-1):
            if liste[j] > liste[j+1]:
                liste[j], liste[j+1] = liste[j+1], liste[j]

```

Quick Sort wählt ein Pivot-Element und partitioniert die Liste in Elemente kleiner und größer als das Pivot. Anschließend wird rekursiv auf beide Teilbereiche Quick Sort angewendet.

```

def quick_sort(liste):
    if len(liste) <= 1:
        return liste
    pivot = liste[len(liste) // 2]
    links = [x for x in liste if x < pivot]
    mitte = [x for x in liste if x == pivot]
    rechts = [x for x in liste if x > pivot]
    return quick_sort(links) + mitte + quick_sort(rechts)

```

Merge Sort teilt die Liste rekursiv in zwei Hälften, sortiert diese und verschmilzt (merged) sie anschließend.

```

def merge_sort(liste):
    if len(liste) <= 1:
        return liste

    mitte = len(liste) // 2

```

```

links = merge_sort(liste[:mitte])
rechts = merge_sort(liste[mitte:])

return merge(links, rechts)

def merge(links, rechts):
    ergebnis = []
    i = j = 0
    while i < len(links) and j < len(rechts):
        if links[i] < rechts[j]:
            ergebnis.append(links[i])
            i += 1
        else:
            ergebnis.append(rechts[j])
            j += 1
    ergebnis.extend(links[i:])
    ergebnis.extend(rechts[j:])
    return ergebnis

```

In Python werden Funktionen mit dem Schlüsselwort **def** (define) definiert. Die grundlegende Syntax lautet:

```

def funktionsname(parameter1, parameter2):
    """Docstring: Beschreibt, was die Funktion macht."""
    # Funktionskörper
    anweisung1
    anweisung2
    return ergebnis

```

[!TIP] **Beispiel: Einfache Funktion ohne Parameter**

```

def gruesse():
    """Gibt eine Begrüßung aus."""
    print("Hallo, willkommen in der Vorlesung!")

```

Funktionsaufruf

```
gruesse() # Ausgabe: Hallo, willkommen in der Vorlesung!
```

[!TIP] **Beispiel: Funktion mit Parametern**

```

def berechne_flaeche(laenge, breite):
    """Berechnet die Fläche eines Rechtecks."""
    flaeche = laenge * breite
    return flaeche

```

Funktionsaufruf

```

ergebnis = berechne_flaeche(5, 3)
print(f"Fläche: {ergebnis}") # Ausgabe: Fläche: 15

```

[!WARNING] **Häufiger Fehler:** Funktionsdefinition ohne Doppelpunkt führt zu `SyntaxError`:

```
def meine_funktion()  # Falsch: Doppelpunkt fehlt!
    print("Test")
```

Parameter sind Variablen in der Funktionsdefinition, die Eingabewerte entgegennehmen. **Argumente** sind die konkreten Werte, die beim Funktionsaufruf übergeben werden.

```
def addiere(a, b):  # a und b sind Parameter
    return a + b
```

```
ergebnis = addiere(5, 3)  # 5 und 3 sind Argumente
```

Standardmäßig werden Argumente in der Reihenfolge zugeordnet, in der sie übergeben werden:

```
def beschreibe_person(name, alter, beruf):
    print(f"{name} ist {alter} Jahre alt und arbeitet als {beruf}.")
```

```
beschreibe_person("Alice", 30, "Ingenieurin")
# Ausgabe: Alice ist 30 Jahre alt und arbeitet als Ingenieurin.
```

[!TIP] **Beispiel: Funktion mit Rückgabewert**

```
def quadrat(x):
    """Berechnet das Quadrat einer Zahl."""
    return x ** 2
```

```
ergebnis = quadrat(5)
print(ergebnis)  # Ausgabe: 25
```

Python ermöglicht es, mehrere Werte gleichzeitig zurückzugeben (als `Tupel`):

```
def kreisberechnung(radius):
    """Berechnet Umfang und Fläche eines Kreises."""
    import math
    umfang = 2 * math.pi * radius
    flaeche = math.pi * radius ** 2
    return umfang, flaeche  # Gibt Tupel zurück
```

```
u, f = kreisberechnung(5)
print(f"Umfang: {u:.2f}, Fläche: {f:.2f}")
# Ausgabe: Umfang: 31.42, Fläche: 78.54
```

[!TIP] **Unpacking bei mehreren Rückgabewerten:**

```
# Variante 1: Unpacking in separate Variablen
umfang, flaeche = kreisberechnung(5)
```

```

# Variante 2: Als Tupel empfangen
ergebnis = kreisberechnung(5)
print(ergebnis) # (31.41592653589793, 78.53981633974483)
print(ergebnis[0]) # Umfang
print(ergebnis[1]) # Fläche

```

Wenn eine Funktion kein `return`-Statement hat oder `return` ohne Wert aufgerufen wird, gibt sie implizit `None` zurück:

```

def sage_hallo(name):
    print(f"Hallo, {name}!")
    # Kein return-Statement

```

```

ergebnis = sage_hallo("Alice")
print(ergebnis) # Ausgabe: None

```

[!WARNING] Häufiger Fehler: `print()` ist kein `return`!

```

def addiere_falsch(a, b):
    print(a + b) # Gibt nichts zurück, nur Ausgabe!

```

```

ergebnis = addiere_falsch(3, 4)
# Ausgabe: 7
print(ergebnis) # None (Funktion gibt nichts zurück!)

```

```

# Richtig:
def addiere_richtig(a, b):
    return a + b

```

```

ergebnis = addiere_richtig(3, 4)
print(ergebnis) # 7

```

[!TIP] Beispiel: Funktion mit Default-Parametern

```

def gruesse(name, grusskarte="Hallo"):
    """Begrüßt eine Person mit anpassbarer Grußformel."""
    print(f"{grusskarte}, {name}!")

```

```

# Aufruf mit Default-Wert
gruesse("Alice") # Ausgabe: Hallo, Alice!

```

```

# Aufruf mit eigenem Wert
gruesse("Bob", "Guten Tag") # Ausgabe: Guten Tag, Bob!

```

```

# Aufruf mit Keyword Argument
gruesse("Charlie", grusskarte="Moin") # Ausgabe: Moin, Charlie!

```

Wichtige Regeln: 1. Parameter mit Default-Werten müssen nach Parametern ohne Default stehen:

```

# Richtig:
def funktion(a, b, c=10, d=20):
    pass

# Falsch:
def funktion(a, c=10, b, d=20): # SyntaxError!
    pass

```

2. Default-Werte werden nur einmal bei Funktionsdefinition ausgewertet:

```

def funktion(x=[]): # Vorsicht: Liste wird nur einmal erstellt!
    x.append(1)
    return x

```

```

print(funktion()) # [1]
print(funktion()) # [1, 1] - Dieselbe Liste!

```

[!WARNING] **Mutable Default-Werte vermeiden:** Verwende niemals mutable Objekte (Listen, Dictionaries) als Default-Werte:

```

# Falsch:
def fuege_element_hinzu(element, liste=[]):
    liste.append(element)
    return liste

print(fuege_element_hinzu(1)) # [1]
print(fuege_element_hinzu(2)) # [1, 2] - Unerwartetes Verhalten!

```

```

# Richtig:
def fuege_element_hinzu(element, liste=None):
    if liste is None:
        liste = []
    liste.append(element)
    return liste

print(fuege_element_hinzu(1)) # [1]
print(fuege_element_hinzu(2)) # [2] - Neue Liste jedes Mal

```

Praktisches Beispiel: Funktion mit mehreren Default-Parametern

```

def berechne_zinseszins(kapital, zinssatz=0.05, jahre=10):
    """
    Berechnet das Endkapital nach Zinseszins.

    kapital: Anfangskapital in Euro
    zinssatz: Jährlicher Zinssatz (Standard: 5% = 0.05)
    jahre: Anlagedauer in Jahren (Standard: 10)
    """

```

```

    """
    endkapital = kapital * (1 + zinssatz) ** jahre
    return endkapital

# Verschiedene Aufrufvarianten:
print(berechne_zinseszins(1000)) # Standard: 5%, 10 Jahre
# Ausgabe: 1628.89

print(berechne_zinseszins(1000, 0.03)) # 3%, 10 Jahre
# Ausgabe: 1343.92

print(berechne_zinseszins(1000, zinssatz=0.07, jahre=5)) # 7%, 5 Jahre
# Ausgabe: 1402.55

```

1. Funktionen können Variablen zugewiesen werden:

```

def addiere(a, b):
    return a + b

# Funktion einer Variable zuweisen
rechne = addiere
print(rechne(3, 4)) # Ausgabe: 7

```

2. Funktionen können als Argumente übergeben werden:

```

def wende_operation_an(funktion, a, b):
    """Wendet eine übergebene Funktion auf zwei Werte an."""
    return funktion(a, b)

def multipliziere(x, y):
    return x * y

ergebnis = wende_operation_an(multipliziere, 5, 3)
print(ergebnis) # Ausgabe: 15

```

3. Funktionen können als Rückgabewerte zurückgegeben werden:

```

def erzeuge_multiplikator(faktor):
    """Gibt eine Funktion zurück, die mit 'faktor' multipliziert."""
    def multipliziere(x):
        return x * faktor
    return multipliziere

verdopple = erzeuge_multiplikator(2)
verdreifache = erzeuge_multiplikator(3)

print(verdopple(10)) # Ausgabe: 20
print(verdreifache(10)) # Ausgabe: 30

```

[!TIP] **Praktische Anwendung – Sortierung mit key-Parameter:** Die `sorted()`-Funktion und `.sort()`-Methode akzeptieren einen `key`-Parameter, der eine Funktion ist:

```
# Sortiere nach Länge der Strings
woerter = ["Python", "ist", "eine", "tolle", "Sprache"]
sortiert = sorted(woerter, key=len)
print(sortiert) # ['ist', 'eine', 'tolle', 'Python', 'Sprache']
```

```
# Sortiere Tupel nach zweitem Element
punkte = [(1, 5), (3, 2), (2, 8)]
sortiert = sorted(punkte, key=lambda x: x[1])
print(sortiert) # [(3, 2), (1, 5), (2, 8)]
```

[!TIP] **Beispiel: Lokale vs. Globale Variablen**

```
x = 10 # Globale Variable

def meine_funktion():
    x = 5 # Lokale Variable (überdeckt die globale)
    print(f"Innerhalb der Funktion: x = {x}")

meine_funktion() # Ausgabe: Innerhalb der Funktion: x = 5
print(f"Außerhalb der Funktion: x = {x}") # Ausgabe: Außerhalb der Funktion: x = 10
```

[!WARNING] **global-Keyword sparsam verwenden:** Wie in V03 erwähnt, sollten globale Variablen vermieden werden. Bevorzuge Parameter und Rückgabewerte:

```
# Nicht empfohlen:
counter = 0

def increment():
    global counter
    counter += 1

# Besser:
def increment(counter):
    return counter + 1

counter = 0
counter = increment(counter)
```

[!TIP] **Beispiel: Gut dokumentierte Funktion**

```
def berechne_bmi(gewicht, groesse):
    """
    Berechnet den Body-Mass-Index (BMI).
    """
```



```

Parameter:
    gewicht (float): Körpergewicht in Kilogramm
    groesse (float): Körpergröße in Metern

Rückgabewert:
    float: BMI-Wert (Gewicht / Größe²)

Beispiel:
>>> berechne_bmi(70, 1.75)
22.86
"""
return gewicht / (groesse ** 2)

# Docstring abrufen:
print(berechne_bmi.__doc__)
help(berechne_bmi)

```

Bubble Sort als Funktion

```

def bubble_sort(liste):
    """
    Sortiert eine Liste mit dem Bubble-Sort-Algorithmus.

    Parameter:
        liste (list): Liste mit vergleichbaren Elementen

    Rückgabewert:
        list: Sortierte Kopie der Liste

    Zeitkomplexität:  $O(n^2)$ 
    """
    # Kopie erstellen, um Original nicht zu verändern
    sortierte_liste = liste.copy()
    n = len(sortierte_liste)

    for i in range(n):
        # Flag für Optimierung (frühzeitiger Abbruch)
        getauscht = False
        for j in range(0, n-i-1):
            if sortierte_liste[j] > sortierte_liste[j+1]:
                # Tausche benachbarte Elemente
                sortierte_liste[j], sortierte_liste[j+1] = sortierte_liste[j+1], sortierte_liste[j]
                getauscht = True
        # Wenn keine Tauschung stattfand, ist Liste sortiert
        if not getauscht:
            break

```

```

        return sortierte_liste

# Test:
zahlen = [64, 34, 25, 12, 22, 11, 90]
sortiert = bubble_sort(zahlen)
print(f"Original: {zahlen}")
print(f"Sortiert: {sortiert}")

```

Binäre Suche als Funktion

```

def binaere_suche(sortierte_liste, ziel):
    """
    Sucht ein Element in einer sortierten Liste mit binärer Suche.

    Parameter:
        sortierte_liste (list): Sortierte Liste mit vergleichbaren Elementen
        ziel: Das gesuchte Element

    Rückgabewert:
        int: Index des Elements, oder -1 wenn nicht gefunden

    Zeitkomplexität:  $O(\log n)$ 
    Voraussetzung: Liste muss sortiert sein!
    """
    links, rechts = 0, len(sortierte_liste) - 1

    while links <= rechts:
        mitte = (links + rechts) // 2

        if sortierte_liste[mitte] == ziel:
            return mitte # Gefunden!
        elif sortierte_liste[mitte] < ziel:
            links = mitte + 1 # Suche in rechter Hälfte
        else:
            rechts = mitte - 1 # Suche in linker Hälfte

    return -1 # Nicht gefunden

# Test:
zahlen = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
index = binaere_suche(zahlen, 11)
print(f"Element 11 gefunden an Index: {index}") # 5

index = binaere_suche(zahlen, 8)
print(f"Element 8 gefunden an Index: {index}") # -1 (nicht vorhanden)

```

[!WARNING] Fehler 1: Funktion aufrufen, bevor sie definiert ist

```
# Falsch:  
ergebnis = addiere(3, 4) # NameError: name 'addiere' is not defined
```

```
def addiere(a, b):  
    return a + b
```

[!WARNING] Fehler 2: return vergessen

```
def quadrat(x):  
    x ** 2 # Fehlt: return!
```

```
ergebnis = quadrat(5)  
print(ergebnis) # None
```

[!WARNING] Fehler 3: Funktion mit und ohne return verwechseln

```
# Funktion mit Seiteneffekt (print):
```

```
def zeige_ergebnis(x):  
    print(x ** 2)
```

```
# Funktion mit Rückgabewert:
```

```
def berechne_ergebnis(x):  
    return x ** 2
```

```
# Verwechslung:
```

```
ergebnis = zeige_ergebnis(5) # Gibt 25 aus, aber ergebnis ist None!  
print(ergebnis * 2) # TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'
```

[!WARNING] Fehler 4: Mutable Default-Werte

```
def fuege_hinzu(element, liste=[]):  
    liste.append(element)  
    return liste
```

```
print(fuege_hinzu(1)) # [1]  
print(fuege_hinzu(2)) # [1, 2] - Unerwartetes Verhalten!
```

Lösung: Verwende None als Default und erstelle neue Objekte im Funktionskörper:

```
def fuege_hinzu(element, liste=None):  
    if liste is None:  
        liste = []  
    liste.append(element)  
    return liste
```

[!WARNING] Fehler 5: Falsche Parameterreihenfolge

```
def berechne_kraft(masse, beschleunigung=9.81):
    return masse * beschleunigung

# Falsch:
kraft = berechne_kraft(beschleunigung=5, 10) # SyntaxError!
```

Datei: lessons/V11-GPTs-LLMs-KI/02_praxis.ipynb

V11 – Python-Funktionen Teil 2: Praxis

Lernziele

Nach diesem Notebook kannst du ... - Funktionen mit **Default-Parametern** schreiben, - **Keyword-Argumente** gezielt einsetzen, - ***args** und ****kwargs** für flexible Funktionen nutzen, - einen sauberen **Docstring** verfassen.

Zeitbudget

Ca. 40 Minuten.

TL;DR

Funktionen in Python können mehr als nur feste Parameter: Default-Werte sparen Tipparbeit, ***args**/****kwargs** erlauben beliebig viele Argumente.

So gehst du vor

- Führe jede Zelle aus (Shift+Enter).
- Ändere Werte, probiere aus.
- Zellen mit # TODO: sind zum Selbst-Ausfüllen.

Voraussetzungen

- V10: Funktionen Teil 1 (def, return, Parameter, Docstring-Basics).

Schritt 1 – Kurze Wiederholung: def und return

Eine Funktion bekommt **Eingaben** (Parameter) und liefert mit **return** eine **Ausgabe**.

```
def addiere(a, b):
    return a + b

print(addiere(3, 4))
print(addiere(10, -2))
```

Schritt 2 – Default-Parameter

Ein Parameter darf einen **Standardwert** bekommen. Dann muss man ihn beim Aufruf nicht mehr angeben.

```
def begruesse(name, gruss="Hallo"):
    return f"{gruss}, {name}!"

print(begruesse("Anna"))
print(begruesse("Ben", gruss="Servus"))
```

Jetzt du

Vervollständige die Funktion `berechne_drehzahl` so, dass sie standardmäßig mit **Riemenübersetzung** = 1.0 rechnet. Formel: `drehzahl_ab = drehzahl_an * uebersetzung`.

```
# TODO: Ergänze den Default-Parameter so, dass ohne Angabe die Drehzahl unverändert bleibt.
def berechne_drehzahl(drehzahl_an, uebersetzung=0):    # <-- Default ändern!
    return drehzahl_an * uebersetzung

n_ab = berechne_drehzahl(1500)           # Ziel: 1500
n_ab2 = berechne_drehzahl(1500, 0.5)     # Ziel: 750
print(n_ab, n_ab2)

# Selbstkontrolle
try:
    assert berechne_drehzahl(1500) == 1500, "Ohne Übersetzung muss die Drehzahl gleich bleiben."
    assert berechne_drehzahl(1500, 0.5) == 750.0, "Mit 0.5 muss die Drehzahl halbiert sein."
    print(" Default-Parameter sitzt!")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except (NameError, TypeError):
    print(" Funktion noch nicht korrekt definiert - hast du den Default-Wert gesetzt?")
```

Schritt 3 – Keyword-Argumente

Du darfst Parameter beim Aufruf **mit Namen** übergeben. Das macht den Code lesbarer und die Reihenfolge egal.

```
def rechteck_flaeche(breite, hoehe):
    return breite * hoehe

# Alle drei Aufrufe ergeben 50:
print(rechteck_flaeche(5, 10))
print(rechteck_flaeche(breite=5, hoehe=10))
print(rechteck_flaeche(hoehe=10, breite=5))
```

Schritt 4 – `*args`: beliebig viele positionale Argumente

Mit `*args` darf die Funktion **beliebig viele** einzelne Werte bekommen. Innerhalb der Funktion ist `args` ein Tupel.

```
def summe(*zahlen):
    gesamt = 0
    for z in zahlen:
        gesamt += z
    return gesamt

print(summe(1, 2, 3))
print(summe(10, 20, 30, 40, 50))
print(summe())           # keine Argumente → 0
```

Jetzt du

Schreibe `mittelwert(*werte)`: gibt den arithmetischen Mittelwert zurück. Falls keine Werte übergeben werden, soll die Funktion 0 zurückgeben (einfachste Annahme).

```
# TODO: Implementiere mittelwert mit *args.
def mittelwert(*werte):
    if len(werte) == 0:
        return 0
    # TODO: Summe / Anzahl - bisher liefert die Funktion nur die Summe zurück.
    return sum(werte)

print(mittelwert(2, 4, 6))    # Ziel: 4.0
print(mittelwert())          # Ziel: 0

# Selbstkontrolle
try:
    assert mittelwert(2, 4, 6) == 4, f"Erwartet 4, bekommen {mittelwert(2, 4, 6)}"
    assert mittelwert(10, 20) == 15, f"Erwartet 15, bekommen {mittelwert(10, 20)}"
    assert mittelwert() == 0, "Ohne Werte soll 0 zurückkommen."
    print(" *args verstanden!")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except (NameError, TypeError):
    print(" Funktion mittelwert noch nicht korrekt definiert.")
```

Schritt 5 – `**kwargs`: beliebig viele benannte Argumente

Mit `**kwargs` darf die Funktion **beliebig viele Schlüssel=Wert-Paare** bekommen. Innerhalb der Funktion ist `kwargs` ein Dictionary.

```
def bauteil_info(**eigenschaften):
    for schluessel, wert in eigenschaften.items():
```

```

        print(f"{schluessel}: {wert}")

bauteil_info(name="Zahnrad", material="Stahl", zaehne=42)
print("---")
bauteil_info(farbe="blau")

```

Schritt 6 – Alles zusammen + Docstring

Reihenfolge: erst normale Parameter, dann `*args`, dann benannte Parameter, dann `**kwargs`. Ein **Docstring** (direkt unter `def`) dokumentiert, was die Funktion tut.

```

def log_meldung(level, *teile, trenner=" ", **extras):
    """Erzeugt eine Log-Zeile.

    Args:
        level: Log-Level, z. B. "INFO" oder "ERROR".
        *teile: beliebig viele Textbausteine.
        trenner: Trennzeichen zwischen den Teilen (Default " ").
        **extras: zusätzliche Schlüssel=Wert-Infos, die angehängt werden.

    Returns:
        Die fertig formatierte Log-Zeile als String.
    """
    nachricht = trenner.join(str(t) for t in teile)
    zusatz = " ".join(f"{k}={v}" for k, v in extras.items())
    if zusatz:
        return f"[{level}] {nachricht} | {zusatz}"
    return f"[{level}] {nachricht}"

print(log_meldung("INFO", "Motor", "gestartet"))
print(log_meldung("ERROR", "Sensor", "defekt", trenner="-", id=7, bauteil="S3"))

```

Mini-Challenge

Schreibe eine Funktion `max_wert(*werte)`, die den größten übergebenen Wert zurückgibt. Wenn keine Werte übergeben werden, soll sie `None` zurückgeben.

```

# TODO: Implementiere max_wert mit *args. Aktuell gibt die Funktion nur None zurück.
def max_wert(*werte):
    if len(werte) == 0:
        return None
    # TODO: größten Wert bestimmen und zurückgeben
    return None

ergebnis = max_wert(3, 7, 2, 9, 4)

```

```

ergebnis2 = max_wert()
print(ergebnis, ergebnis2)

try:
    assert max_wert(3, 7, 2, 9, 4) == 9
    assert max_wert(-1, -5, -3) == -1
    assert max_wert() is None
    print(" Stark - du kannst *args anwenden!")
except AssertionError:
    print(" Noch nicht ganz. Tipp: max() hat ein default=-Argument, oder prüfe mit len(wert)
except (NameError, TypeError):
    print(" Funktion max_wert noch nicht korrekt definiert.")

```

Zusammenfassung

- **Default-Parameter** (param=wert) machen Aufrufe bequemer.
- **Keyword-Argumente** (name=...) machen Code lesbarer.
- ***args** → beliebig viele positionale Werte (Tupel).
- ****kwargs** → beliebig viele benannte Werte (Dict).
- **Docstring** dokumentiert Zweck, Parameter und Rückgabe.

Nächster Schritt

Weiter mit 03_aufgaben.ipynb.

Datei: lessons/V12-Prompt-Engineering-Best-Practices/02_praxis.ipynb

V12 – Praxis: Prompt-Vorlagen mit f-Strings & String-Methoden

Lernziele

Nach diesem Notebook kannst du ... - **f-Strings** (Wiederholung aus V02) sicher in mehrzeiligen Prompt-Templates verwenden, - die vier wichtigsten String-Methoden **.strip()**, **.upper()**, **.lower()**, **.replace()** gezielt einsetzen, - Eingaben des Nutzers vor dem Einbau in einen Prompt **säubern** und **normieren**, - eine wiederverwendbare **Prompt-Vorlage** aus Bausteinen zusammensetzen.

Zeitbudget

Ca. 25 Minuten.

TL;DR

Wir bauen Schritt für Schritt Prompt-Vorlagen. f-Strings liefern das Gerüst, String-Methoden bereiten die Eingaben so auf, dass Groß-/Kleinschreibung, überflüssige Leerzeichen und Tippfehler den Prompt nicht zerstören.

Voraussetzungen

- V02 (f-Strings), V03 (Datentypen), V04 (Vergleiche & Logik), 00_python_recap.ipynb.

So gehst du vor

- Führe jede Code-Zelle aus (Shift+Enter).
- Ändere Werte, probiere aus.
- Zellen mit # TODO: sind zum Selbst-Ausfüllen – danach die Selbstkontroll-Zelle laufen lassen.

Schritt 1 – f-Strings schnell aufgefrischt

Ein **f-String** beginnt mit `f"..."`. Alles in geschweiften Klammern ist ein Python-Ausdruck, der beim Bauen des Strings ausgewertet wird.

```
maschine = "CNC-Fraese 01"
temperatur = 42.567

print(f"Maschine: {maschine}")
print(f"Temperatur: {temperatur:.1f} Grad C")
print(f"Kurzform: {maschine[:3].upper()} | {temperatur:>6.2f}")
```

Mehrzeilige f-Strings mit `"""..."""`

Für Prompt-Vorlagen ist ein **mehrzeiliger** f-String ideal: Zeilenumbrüche und Einrückungen bleiben so, wie du sie schreibst.

```
rolle = "Fertigungsingenieur"
aufgabe = "Erkläre Vorschubgeschwindigkeit"
format_wunsch = "3 Sätze, einfache Sprache"

prompt = f"""Rolle: {rolle}
Aufgabe: {aufgabe}
Format: {format_wunsch}"""

print(prompt)
```

Fill-in-Blank 1 – Prompt-Template selbst bauen

Baue einen mehrzeiligen f-String mit **genau vier** Zeilen, die so aussehen:

```
Rolle: <rolle>
Kontext: <kontext>
Aufgabe: <aufgabe>
Format: <format_wunsch>
```

und speichere das Ergebnis in der Variable `prompt`.

```
rolle = "Python-Tutor"
kontext = "Anfaenger-Kurs, 1. Semester"
aufgabe = "Erklaere f-Strings"
format_wunsch = "2 Saetze + 1 Beispiel"

# TODO: mehrzeiligen f-String bauen (siehe Muster oben)
prompt = "" # <-- hier anpassen

erwartet = (
    "Rolle: Python-Tutor\n"
    "Kontext: Anfaenger-Kurs, 1. Semester\n"
    "Aufgabe: Erkläre f-Strings\n"
    "Format: 2 Saetze + 1 Beispiel"
)
try:
    assert prompt == erwartet, f"Prompt stimmt nicht. Bekommen:\n{prompt}"
    print("OK:\n" + prompt)
except AssertionError as e:
    print(" Noch nicht richtig:", e)
```

Schritt 2 – `.strip()`: Leerzeichen und Umbrüche entfernen

`text.strip()` entfernt am Anfang und am Ende Leerzeichen, Tabs und Zeilenumbrüche. Sehr nützlich, wenn Eingaben aus einer Textdatei oder aus `input()` kommen – dort schleppt man gerne unsichtbare Zeichen mit.

```
eingabe = "   Erkläre Vorschubgeschwindigkeit \n"
sauber = eingabe.strip()
print(repr(eingabe))
print(repr(sauber))
```

Schritt 3 – `.upper()` und `.lower()`: Groß-/Kleinschreibung

`text.upper()` macht alles groß, `text.lower()` alles klein. Die Methoden verändern den Original-String nicht, sondern geben einen neuen String zurück (Strings sind in Python *immutable*, siehe V03). `.lower()` ist der klassische Trick, um Eingaben unempfindlich gegen Groß-/Kleinschreibung zu vergleichen.

```
begriff = "SpindelDrehZAHL"
print(begriff.upper()) # SPINDELDREHZAHL
print(begriff.lower()) # spindeldrehzahl
```

```
# Vergleich unempfindlich gegen Schreibweise:
nutzer_eingabe = "  SpindelDrehzahl  "
if nutzer_eingabe.strip().lower() == "spindeldrehzahl":
    print("Treffer")
else:
    print("Kein Treffer")
```

Fill-in-Blank 2 – Eingabe normieren

Gegeben ist die rohe Eingabe `rohe_eingabe`. Erzeuge daraus den bereinigten Schlüssel `normiert`, so dass er

- keine führenden/nachfolgenden Leerzeichen enthält,
- komplett in Kleinbuchstaben geschrieben ist.

Erwartetes Ergebnis: `"vorschubgeschwindigkeit"`.

```
rohe_eingabe = "  Vorschubgeschwindigkeit  \n"
```

```
# TODO: normiert aus rohe_eingabe bauen - tipp: .strip() und .lower() verketteten
normiert = rohe_eingabe # <-- hier anpassen
```

```
try:
    assert normiert == "vorschubgeschwindigkeit", f"Erwartet 'vorschubgeschwindigkeit', erhalten {normiert}"
    print("OK:", repr(normiert))
except AssertionError as e:
    print(" Noch nicht richtig:", e)
```

Schritt 4 – `.replace()`: Platzhalter füllen

`text.replace(alt, neu)` ersetzt **jede** Vorkommnis von `alt` durch `neu`. Das ist die einfachste Art, **Platzhalter** in einem Template zu füllen, ohne gleich ein f-String- Konstrukt zu bauen – ideal, wenn die Vorlage aus einer Datei kommt.

```
vorlage = "Du bist {ROLLE}. Erkläre {BEGRIFF} in {SAETZE} Saetzen."
gefüllt = (
    vorlage
    .replace("{ROLLE}", "Fertigungsingenieur")
    .replace("{BEGRIFF}", "Vorschubgeschwindigkeit")
    .replace("{SAETZE}", "3")
)
print(gefüllt)
```

Fill-in-Blank 3 – Datenblatt-Prompt befüllen

Fülle die Platzhalter `{WERKSTOFF}` und `{ZWECK}` in der Vorlage mit den Werten aus den Variablen und speichere das Ergebnis in `prompt`.

```

vorlage = "Fasse das Datenblatt zu {WERKSTOFF} zusammen. Zielgruppe: {ZWECK}."
werkstoff = "S235JR"
zweck = "Konstruktions-Entwurf einer Rahmen-Schweissbaugruppe"

# TODO: prompt mit replace() befüllen
prompt = vorlage # <-- hier anpassen

erwartet = (
    "Fasse das Datenblatt zu S235JR zusammen. "
    "Zielgruppe: Konstruktions-Entwurf einer Rahmen-Schweissbaugruppe."
)
try:
    assert prompt == erwartet, f"Erhalten: {prompt}"
    print("OK:", prompt)
except AssertionError as e:
    print(" Noch nicht richtig:", e)

```

Schritt 5 – Alles kombiniert: Prompt-Factory

Jetzt bauen wir eine **Mini-Funktion**, die aus drei Bausteinen einen sauberen, normierten Prompt macht. Das ist exakt das Muster, das du in den Aufgaben brauchst.

```

def baue_prompt(rolle, aufgabe, format_wunsch):
    rolle = rolle.strip()
    aufgabe = aufgabe.strip()
    format_wunsch = format_wunsch.strip()
    return (
        f"Du bist {rolle}.\n"
        f"Aufgabe: {aufgabe}\n"
        f"Format: {format_wunsch}"
    )

print(baue_prompt(
    " Python-Tutor ",
    "Erkläre f-Strings\n",
    " 2 Sätze + 1 Beispiel ",
))

```

Mini-Challenge

Schreibe eine Funktion `normiere_begriff(text)`, die einen technischen Begriff so bereinigt, dass er als **eindeutiger Schlüssel** dienen kann:

- führende/nachfolgende Leerzeichen entfernen (`.strip()`),
- komplett klein schreiben (`.lower()`),
- Bindestriche durch Unterstriche ersetzen (`.replace("-", "_")`).

Beispiel: `normiere_begriff(" CNC-Fraese ")` soll `"cnc_fraese"` liefern.

```
def normiere_begriff(text):
    # TODO: strip, lower, replace anwenden
    return text

try:
    assert normiere_begriff(" CNC-Fraese ") == "cnc_fraese"
    assert normiere_begriff("Vorschub-Geschwindigkeit") == "vorschub_geschwindigkeit"
    assert normiere_begriff("\tWerkzeug-Halter\n") == "werkzeug_halter"
    print(" Mini-Challenge geloest.")
except AssertionError as e:
    print(" Noch nicht ganz:", e)
except (NameError, TypeError) as e:
    print(" Funktion noch nicht korrekt definiert:", e)
```

Zusammenfassung

- **f-Strings** (`f"...{x}..."`) bauen Text mit Variablen – auch mehrzeilig mit `"""..."""`.
- `.strip()` räumt Eingaben auf (Leerzeichen, `\n`, `\t`).
- `.lower()` / `.upper()` vereinheitlichen Groß-/Kleinschreibung.
- `.replace(alt, neu)` füllt Platzhalter oder korrigiert Zeichen.
- Diese vier Werkzeuge reichen, um Prompt-Vorlagen **robust** aus Nutzer-Eingaben zu bauen.

Nächster Schritt

Weiter mit `03_aufgaben.ipynb` – fünf echte Aufgaben aus dem Fertigungs-Alltag.

Datei: `lessons/V13-Betriebssysteme-Rechnerarchitektur-Teil1/02_praxis.ipynb`

V13 – Rechnerarchitektur: Praxis – Daten visualisieren mit matplotlib

Lernziele

Nach diesem Notebook kannst du ... - einfache **Balkendiagramme** mit `matplotlib.pyplot.bar()` erstellen, - deine Plots mit **Titel** und **Achsenbeschriftungen** versehen, - **Messreihen** aus dem Maschinenbau (z. B. Werkzeugverschleiß) sichtbar machen, - Balken **einfärben** und X-Labels **drehen**, damit lange Namen lesbar bleiben.

Zeitbudget

Ca. 30 Minuten.

TL;DR

matplotlib ist die Standard-Bibliothek in Python, um Daten grafisch darzustellen. Mit `plt.bar(kategorien, werte)` zeichnest du ein Balkendiagramm; `plt.title`, `plt.xlabel`, `plt.ylabel` und `plt.show` vervollständigen den Plot.

Voraussetzungen

- V13 Theorie (01_theorie.ipynb) – wenigstens überflogen
- Python-Grundlagen aus V01–V08 (Listen, `for`, Funktionen)
- Paket **matplotlib** (siehe `requirements.txt`)

1. Warum überhaupt Plots?

Im Maschinenbau fallen ständig **Messreihen** an: Temperaturen an einem Lager, Verschleiß eines Werkzeugs nach jeder Schicht, Ausschussraten einer Fertigungszelle. Als reine Zahlenkolonne in einer Tabelle sind diese Werte für Menschen schwer zu überblicken – erst ein Diagramm macht Trends, Ausreißer und Muster unmittelbar sichtbar.

Ein **Balkendiagramm** vergleicht Werte zwischen **diskreten Kategorien** (z. B. fünf verschiedene Werkzeuge, drei Maschinen, vier Schichten). Es beantwortet die Frage: „*Welche Kategorie hat den höchsten/niedrigsten Wert?*“ schneller als jede Tabelle.

Die Python-Bibliothek **matplotlib** liefert genau dafür die nötigen Funktionen. Wir nutzen das Submodul `matplotlib.pyplot` und importieren es nach Konvention als `plt`.

```
# Standard-Import von matplotlib. 'plt' ist der übliche Kurzname.
import matplotlib.pyplot as plt

# Wir sagen matplotlib, dass Plots inline im Notebook erscheinen sollen
# (bei nbconvert und in VS Code ist das ohnehin der Standard).
print("matplotlib ist bereit:", plt.__name__)
```

2. Der erste Plot

Die wichtigste Funktion für uns ist `plt.bar(x, hoehen)`. Sie bekommt zwei Listen: die **Positionen** der Balken auf der X-Achse und die **Höhen** der Balken auf der Y-Achse.

Am einfachsten startest du mit Zahlen `[1, 2, 3]` als Positionen und irgendwelchen Messwerten als Höhen. Mit `plt.show()` sagst du matplotlib:

„Jetzt das Diagramm anzeigen.“ Ohne `plt.show()` wird in Skripten manchmal gar nichts dargestellt.

```
import matplotlib.pyplot as plt

# Einfachster Balkendiagramm-Aufruf: drei Positionen, drei Höhen.
plt.figure()
plt.bar([1, 2, 3], [10, 15, 12])
plt.show()
```

Du siehst drei Balken unterschiedlicher Höhe. Die X-Achse zeigt 1, 2, 3, die Y-Achse die Werte. Das Diagramm ist **korrekt**, aber es fehlt jede inhaltliche Einordnung: Was bedeuten die Zahlen? In welcher Einheit? Genau das regeln **Titel** und **Achsenbeschriftungen** im nächsten Abschnitt.

3. Achsen beschriften – Pflicht, nicht Kür

Ein Diagramm ohne Beschriftung ist in einer Ingenieur-Präsentation wertlos. Drei Funktionen reichen, um es aussagekräftig zu machen:

- `plt.title("...")` setzt den **Titel** des Plots.
- `plt.xlabel("...")` beschriftet die **X-Achse**.
- `plt.ylabel("...")` beschriftet die **Y-Achse** (hier nennst du immer die **Einheit**, z. B. "Verschleiß in mm").

Die Aufrufe müssen **vor** `plt.show()` stehen, sonst werden sie dem nächsten Diagramm zugeordnet.

```
import matplotlib.pyplot as plt

werkzeuge = ["W1", "W2", "W3"]
standzeiten = [120, 95, 150] # Standzeit in Minuten bis Verschleißgrenze

plt.figure()
plt.bar(werkzeuge, standzeiten)
plt.title("Standzeit pro Werkzeug")
plt.xlabel("Werkzeug")
plt.ylabel("Standzeit in Minuten")
plt.show()
```

4. Mit echten Maschinenbau-Daten arbeiten

Stell dir vor, du hast den **Werkzeugverschleiß** von fünf CNC-Maschinen nach einer 8-Stunden-Schicht gemessen. Du bekommst zwei parallele Listen: die **Maschinen-Namen** und den **gemessenen Verschleiß** in Millimetern.

Das Muster ist immer dasselbe:

1. Zwei Listen vorbereiten – eine für **Kategorien** (X-Achse), eine für **Werte** (Y-Achse).
2. `plt.bar(kategorien, werte)` aufrufen.
3. Titel und Achsen beschriften.
4. `plt.show()` – fertig.

```
import matplotlib.pyplot as plt

maschinen = ["CNC-A", "CNC-B", "CNC-C", "CNC-D", "CNC-E"]
verschleiss = [0.12, 0.18, 0.09, 0.22, 0.15] # in mm pro Schicht

plt.figure()
plt.bar(maschinen, verschleiss)
plt.title("Werkzeugverschleiß pro CNC-Maschine (eine Schicht)")
plt.xlabel("Maschine")
plt.ylabel("Verschleiß in mm")
plt.show()

print("Höchster Verschleiß:", max(verschleiss), "mm")
print("Niedrigster Verschleiß:", min(verschleiss), "mm")
```

5. Farbe und gedrehte X-Labels

Zwei kleine Verbesserungen, die du in fast jedem Report brauchst:

- Mit `color="..."` färbst du alle Balken einheitlich ein ("steelblue", "orange", "#2e7d32" ...).
- Lange Kategorie-Namen überlappen schnell. `plt.xticks(rotation=45)` dreht die X-Labels um 45°, dann sind sie wieder lesbar.

Das reicht für 95 % aller Standard-Plots.

```
import matplotlib.pyplot as plt

schichten = ["Frühschicht", "Spätschicht", "Nachtschicht", "Wochenend-Sonder"]
ausschuss_prozent = [1.2, 2.8, 3.5, 4.1]

plt.figure()
plt.bar(schichten, ausschuss_prozent, color="steelblue")
plt.title("Ausschussrate nach Schicht")
plt.xlabel("Schicht")
plt.ylabel("Ausschuss in %")
plt.xticks(rotation=45)
plt.tight_layout() # sorgt dafür, dass gedrehte Labels nicht abgeschnitten werden
plt.show()

import matplotlib.pyplot as plt
```



```
# Kurze Variante mit farbigen Balken und zweiter Farbe zum Vergleich.
plt.figure()
plt.bar(["vorher", "nachher"], [4.1, 1.6], color=["firebrick", "seagreen"])
plt.title("Ausschuss vor und nach Wartung")
plt.ylabel("Ausschuss in %")
plt.show()
```

Fill-in #1 – Titel ergänzen

Im folgenden Plot fehlt der Titel. Ergänze in der Zeile mit `# TODO` den passenden Titel-String „Lagertemperatur je Messpunkt“. Der Selbstcheck liest den Titel mit `plt.gca().get_title()` aus.

```
import matplotlib.pyplot as plt

messpunkte = ["MP1", "MP2", "MP3", "MP4"]
temperaturen = [62, 71, 58, 68] # in °C

plt.figure()
plt.bar(messpunkte, temperaturen, color="steelblue")

# TODO: Ersetze den leeren String durch den Titel "Lagertemperatur je Messpunkt".
plt.title("")

plt.xlabel("Messpunkt")
plt.ylabel("Temperatur in °C")
titel_gesetzt = plt.gca().get_title()
plt.show()

# Selbstkontrolle
try:
    assert titel_gesetzt == "Lagertemperatur je Messpunkt", \
        f"Titel stimmt noch nicht, aktuell: '{titel_gesetzt}'"
    print(" Titel ist korrekt gesetzt.")
except AssertionError as e:
    print(" Noch nicht richtig:", e)
except NameError as e:
    print(" Variable fehlt:", e)
```

Fill-in #2 – X-Achse beschriften

Dem nächsten Plot fehlt die X-Achsen-Beschriftung. Ergänze den passenden Aufruf `plt.xlabel("Baugruppe")`. Der Selbstcheck fragt `plt.gca().get_xlabel()` ab.

```
import matplotlib.pyplot as plt
```

```

baugruppen = ["Rahmen", "Antrieb", "Steuerung", "Hydraulik"]
fehler_pro_100 = [3, 7, 2, 5]

plt.figure()
plt.bar(baugruppen, fehler_pro_100, color="orange")
plt.title("Fehler pro 100 montierte Baugruppen")

# TODO: Ergänze hier den fehlenden xlabel-Aufruf (Text: "Baugruppe").
plt.xlabel("")

plt.ylabel("Fehler pro 100 Stück")
xlabel_gesetzt = plt.gca().get_xlabel()
plt.show()

# Selbstkontrolle
try:
    assert xlabel_gesetzt == "Baugruppe", \
        f"xlabel stimmt noch nicht, aktuell: '{xlabel_gesetzt}'"
    print(" X-Achse ist beschriftet.")
except AssertionError as e:
    print(" Noch nicht richtig:", e)
except NameError as e:
    print(" Variable fehlt:", e)

```

Fill-in #3 – Balkenhöhen eintragen

Du willst die Produktionsstückzahlen der letzten vier Quartale plotten. Die Kategorien stehen schon da, aber die Liste `stueckzahlen` ist noch leer. Ergänze sie mit den Werten [1200, 1350, 1500, 1420] (in dieser Reihenfolge). Der Selbstcheck prüft die **Summe** der Werte.

```

import matplotlib.pyplot as plt

quartale = ["Q1", "Q2", "Q3", "Q4"]

# TODO: Trage die vier Stückzahlen in dieser Reihenfolge ein: 1200, 1350, 1500, 1420
stueckzahlen = []

if len(stueckzahlen) == len(quartale):
    plt.figure()
    plt.bar(quartale, stueckzahlen, color="seagreen")
    plt.title("Produktion pro Quartal")
    plt.xlabel("Quartal")
    plt.ylabel("Stückzahl")
    plt.show()
else:

```

```

        print("Noch keine Werte eingetragen - Plot wird erst nach TODO erzeugt.")

# Selbstkontrolle
try:
    assert len(stueckzahlen) == 4, f"Liste muss 4 Werte haben, hat aber {len(stueckzahlen)}"
    assert sum(stueckzahlen) == 5470, f"Summe erwartet 5470, bekommen: {sum(stueckzahlen)}"
    print(" Stückzahlen sind korrekt eingetragen.")
except AssertionError as e:
    print(" Noch nicht richtig:", e)
except NameError as e:
    print(" Variable fehlt:", e)

```

Mini-Challenge

Erstelle **einen eigenen Balkenplot** mit **mindestens vier** selbst gewählten Werten aus dem Maschinenbau-Kontext (z. B. Lauflistung verschiedener Lager, Taktzeiten verschiedener Stationen, Energieverbrauch verschiedener Antriebe). **Alles beschriften**: Titel, X-Achse, Y-Achse – und `plt.show()` nicht vergessen.

Der Selbstcheck prüft, ob Titel, X-Label und Y-Label **alle nicht leer** sind.

```

import matplotlib.pyplot as plt

# TODO: Definiere eigene Kategorien und Werte (mind. 4) und beschrifte alles.
kategorien = ["A", "B", "C", "D"]
werte = [0, 0, 0, 0] # TODO: sinnvolle Werte einsetzen

plt.figure()
plt.bar(kategorien, werte, color="steelblue")
plt.title("") # TODO: Titel
plt.xlabel("") # TODO: X-Achse
plt.ylabel("") # TODO: Y-Achse (mit Einheit!)
titel = plt.gca().get_title()
xl = plt.gca().get_xlabel()
yl = plt.gca().get_ylabel()
plt.show()

# Selbstkontrolle
try:
    assert titel != "", "Titel fehlt noch."
    assert xl != "", "X-Label fehlt noch."
    assert yl != "", "Y-Label fehlt noch."
    assert sum(werte) > 0, "Setze mindestens einen Wert > 0."
    print(" Mini-Challenge gelöst: Plot ist vollständig beschriftet.")
except AssertionError as e:
    print(" Noch nicht ganz:", e)
except NameError as e:

```

```
print(" Variable fehlt:", e)
```

Zusammenfassung

- `plt.bar(kategorien, werte)` zeichnet ein Balkendiagramm.
- **Immer** mit `plt.title`, `plt.xlabel`, `plt.ylabel` beschriften – Y-Achse mit Einheit.
- `color="..."` färbt alle Balken, `plt.xticks(rotation=45)` dreht lange Labels.
- `plt.show()` gehört ans Ende jeder Plot-Zelle; `plt.figure()` startet bei Bedarf ein frisches Diagramm.

Nächster Schritt

Weiter mit 03_aufgaben.ipynb – dort wendest du das Gelernte auf echte Maschinendaten an.

Datei: lessons/V14-Betriebssysteme-Rechnerarchitektur-Teil2/02_praxis.ipynb

V14 – Betriebssysteme Teil 2: Praxis (os-Modul)

Lernziele

Nach diesem Notebook kannst du ... - mit `os.listdir(pfad)` den Inhalt eines Ordners als Liste auslesen, - mit `os.path.isfile(pfad)` und `os.path.isdir(pfad)` prüfen, ob ein Eintrag eine Datei oder ein Verzeichnis ist, - mit `os.path.getsize(pfad)` die Größe einer Datei in Bytes ermitteln, - mit `os.path.join(basis, name)` plattformunabhängig Pfade zusammenbauen, - mit `os.makedirs(pfad, exist_ok=True)` einen Ordner anlegen und mit `shutil.rmtree(pfad)` rekursiv wieder entfernen.

Zeitbudget

Ca. 25 Minuten.

TL;DR

Wir legen uns im Notebook-Verzeichnis einen temporären Ordner `temp_messdaten/` mit ein paar Dummy-Messdateien an und üben mit dem `os`-Modul alle Operationen, die du später in den Aufgaben brauchst. Am Ende räumen wir den Ordner wieder auf, damit dein Arbeitsverzeichnis sauber bleibt.

Voraussetzungen

- 01_theorie.ipynb — insbesondere Abschnitt 5 (Dateisystem)
- Python-Basics: Listen, Schleifen, f-Strings

0. Was ist das `os`-Modul?

Das Modul `os` gehört zur Python-Standardbibliothek und liefert den plattformunabhängigen Zugang zu Betriebssystem-Funktionen. Egal ob du unter Windows, Linux oder macOS arbeitest: `os.listdir` funktioniert überall gleich und liefert dir einen Ordner-Inhalt als Liste von Strings.

Der wichtigste Untermodul-Bereich ist `os.path`. Er enthält Hilfsfunktionen, die Pfade als Strings analysieren: existiert der Pfad überhaupt? Ist es eine Datei? Ein Verzeichnis? Wie groß ist sie? Wir nutzen in diesem Notebook fünf dieser Funktionen — mehr brauchst du für die Aufgaben nicht.

Vorbereitung: aktuelles Arbeitsverzeichnis

Bevor wir anfangen, vergewissern wir uns, in welchem Ordner das Notebook gerade läuft. `os.getcwd()` („get current working directory“) liefert den Pfad zurück. Relative Pfade wie `"temp_messdaten"` werden Python immer relativ zu *diesem* Ordner aufgelöst.

```
import os

print("Arbeitsverzeichnis:", os.getcwd())
```

Setup: Temp-Ordner mit Dummy-Dateien anlegen

Damit das Notebook auf jedem Rechner läuft — unabhängig davon, was sonst so in deinem Arbeitsverzeichnis herumliegt — erzeugen wir uns einen eigenen **temporären Ordner** mit ein paar Dummy-Dateien. Die folgende Zelle nutzt `os.makedirs(pfad, exist_ok=True)`, um den Ordner sicher anzulegen: Existiert er bereits, passiert nichts; existiert er nicht, wird er erzeugt. Anschließend schreiben wir fünf kleine Dateien hinein und einen Unterordner `archiv/`.

[!NOTE] Das Argument `newline="\n"` beim Öffnen sorgt dafür, dass die Dateigrößen auf Windows und Unix identisch sind — sonst würde Windows automatisch `\r\n` einfügen, und die Asserts weiter unten würden brechen.

```
import os

temp_ordner = "temp_messdaten"
os.makedirs(temp_ordner, exist_ok=True)

# Simulierte Mess-Dateien anlegen
dateien = {
```

```

"messung_01.csv": "Zeit,Drehzahl\n0,1200\n1,1210\n2,1205\n",
"messung_02.csv": "Zeit,Drehzahl\n0,1190\n1,1202\n",
"messung_03.csv": "Zeit,Drehzahl\n0,1220\n1,1225\n2,1219\n3,1230\n",
"kalibrierung.txt": "Offset: 0.02\nSteigung: 1.001\n",
"logbuch.log": "08:00 gestartet\n08:05 Warnung\n",
}
for name, inhalt in dateien.items():
    # newline="\n" erzwingt Unix-Zeileneenden unabhaengig vom Betriebssystem
    with open(os.path.join(temp_ordner, name), "w", encoding="utf-8", newline="\n") as f:
        f.write(inhalt)

# Zusaetzlich einen Unterordner anlegen, damit wir isdir pruefen koennen
os.makedirs(os.path.join(temp_ordner, "archiv"), exist_ok=True)

print(f"Ordner '{temp_ordner}' vorbereitet mit {len(dateien)} Dateien + 1 Unterordner.")

```

1. os.listdir — den Ordner-Inhalt auflisten

Die Funktion `os.listdir(pfad)` liefert dir den Inhalt des angegebenen Ordners als Liste von Strings zurück. Jeder String ist der Name eines Eintrags — *ohne* den Pfad davor und *ohne* Unterscheidung, ob es sich um eine Datei oder einen Unterordner handelt.

Die Reihenfolge der Einträge ist **nicht garantiert**. Auf manchen Systemen erhältst du sie alphabetisch sortiert, auf anderen in der Reihenfolge, in der sie ins Dateisystem aufgenommen wurden. Wenn du deterministische Reihenfolge brauchst, sortiere selbst: `sorted(os.listdir(pfad))`.

[!WARNING] `os.listdir` liefert **nur Namen**, keine vollständigen Pfade. Wenn du später auf diese Einträge zugreifen willst (z. B. mit `os.path.getsize`), musst du den Ordnernamen davor setzen — dafür ist `os.path.join` da.

```

import os

eintraege = os.listdir("temp_messdaten")

print(f"'temp_messdaten' enthaelt {len(eintraege)} Eintraege:")
for name in sorted(eintraege):
    print(" -", name)

```

2. os.path.isfile und os.path.isdir — Datei oder Ordner?

Da `listdir` keinen Hinweis darauf liefert, ob ein Eintrag Datei oder Ordner ist, brauchen wir zwei Prüffunktionen. `os.path.isfile(pfad)` gibt `True` zurück, wenn der Pfad auf eine existierende Datei zeigt. `os.path.isdir(pfad)` gibt

True zurück, wenn er auf ein existierendes Verzeichnis zeigt. Beide liefern False, wenn der Pfad nicht existiert (sie werfen keine Exception).

Wichtig: beide Funktionen brauchen den **vollen Pfad** relativ zu `os.getcwd()`, nicht nur den Namen aus `listdir`. Wir bauen den Pfad also aus Ordnername und Eintragsname zusammen.

```
import os

basis = "temp_messdaten"

for name in sorted(os.listdir(basis)):
    voller_pfad = os.path.join(basis, name)
    if os.path.isfile(voller_pfad):
        kennzeichen = "DATEI"
    elif os.path.isdir(voller_pfad):
        kennzeichen = "ORDNER"
    else:
        kennzeichen = "???"
    print(f" {kennzeichen:6s} {voller_pfad}")
```

Fill-in 1: nur die Dateien

Fülle unten den TODO-Platzhalter so, dass die Variable `nur_dateien` **ausschließlich die Dateinamen** (also nicht den Unterordner `archiv`) aus `temp_messdaten` enthält. Nutze `os.listdir`, `os.path.join` und `os.path.isfile`.

Erwartet: Eine Liste mit 5 Einträgen, sortiert: `['kalibrierung.txt', 'logbuch.log', 'messung_01.csv', 'messung_02.csv', 'messung_03.csv']`.

```
import os

basis = "temp_messdaten"

# TODO: Baue eine Liste der Einträge, die wirklich Dateien sind.
nur_dateien = []

print("Dateien:", sorted(nur_dateien))

# Selbstkontrolle
try:
    erwartet = ["kalibrierung.txt", "logbuch.log", "messung_01.csv", "messung_02.csv", "messung_03.csv"]
    assert sorted(nur_dateien) == erwartet, f"Erwartet {erwartet}, bekommen: {sorted(nur_dateien)}"
    print(" Richtig!")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
```

```
print(" Variable `nur_dateien` fehlt noch.")
```

3. `os.path.getsize` — wie groß ist eine Datei?

`os.path.getsize(pfad)` liefert die Größe einer Datei in **Byte** zurück. Für Ordner ist der Rückgabewert plattformabhängig und meist wenig aussagekräftig — wir prüfen deswegen mit `os.path.isfile` vorher, bevor wir die Größe abfragen.

Wenn der Pfad nicht existiert, wirft `getsize` einen `FileNotFoundError`. Das ist bewusst so: Eine nicht existierende Datei hat keine Größe, und jeder andere Rückgabewert (z. B. 0) wäre irreführend.

```
import os

basis = "temp_messdaten"

for name in sorted(os.listdir(basis)):
    voller_pfad = os.path.join(basis, name)
    if os.path.isfile(voller_pfad):
        groesse = os.path.getsize(voller_pfad)
        print(f" {name:20s} {groesse:4d} Byte")
```

Fill-in 2: die größte Datei finden

Ermittle unten den **Namen** der größten Datei im Ordner `temp_messdaten` und weise ihn der Variable `groesste_datei` zu. Die Dateigröße in Byte soll in `groesste_byte` landen.

Erwartet: `groesste_datei == "messung_03.csv"` mit `groesste_byte == 42`.

```
import os

basis = "temp_messdaten"

groesste_datei = ""
groesste_byte = 0

# TODO: iteriere ueber die Eintraege, merke dir Name und Byte-Zahl der groessten Datei

print(f"Groesste Datei: {groesste_datei} ({groesste_byte} Byte)")

# Selbstkontrolle
try:
    assert groesste_datei == "messung_03.csv", f"Erwartet 'messung_03.csv', bekommen: '{groesste_datei}'"
    assert groesste_byte == 42, f"Erwartet 42 Byte, bekommen: {groesste_byte}"
    print(" Richtig!")
except AssertionError as e:
```



```

    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Eine Variable fehlt noch.")

```

4. `os.path.join` — Pfade sauber zusammenbauen

Du hast `os.path.join` oben bereits zweimal angewendet. Warum ist diese Funktion so wichtig — könntest du nicht einfach `basis + "/" + name` schreiben? Die Antwort: unter Windows ist der Pfad-Trenner `\` (Backslash), unter Linux und macOS `/`. `os.path.join` erkennt die Plattform und setzt automatisch den richtigen Trenner ein. Dein Code läuft damit auf allen Betriebssystemen ohne Anpassung.

Die Funktion nimmt beliebig viele Argumente und fügt sie mit dem plattform-spezifischen Trenner aneinander: `os.path.join("a", "b", "c")` ergibt unter Linux `"a/b/c"`, unter Windows `"a\\b\\c"`. Doppelte Trenner werden dabei vermieden — auch wenn du selbst einen einfügst.

[TIP] Gib niemals Pfade mit fest einprogrammierten Slashes weiter. `os.path.join` ist eine Zeile mehr, aber spart dir Stunden Debugging beim ersten Windows-Benutzer, der dein Skript ausführt.

```

import os

p1 = os.path.join("temp_messdaten", "messung_01.csv")
p2 = os.path.join("temp_messdaten", "archiv", "altlast.csv")
p3 = os.path.join("temp_messdaten", "logbuch.log")

print("p1:", p1)
print("p2:", p2)
print("p3:", p3, "existiert?", os.path.isfile(p3))

```

5. Mini-Challenge: summiere alle Dateigrößen

Zum Abschluss eine kleine Übung, die alle bisherigen Bausteine kombiniert. Summiere die Dateigrößen aller Einträge in `temp_messdaten`, die **Dateien** (nicht Ordner!) sind, und weise das Ergebnis der Variable `summe_bytes` zu.

Erwartet: 164 Byte (= 35 + 28 + 42 + 29 + 30).

```

import os

basis = "temp_messdaten"

# TODO: iteriere ueber die Eintraege, summiere Dateigroessen
summe_bytes = 0

print("Gesamt:", summe_bytes, "Byte")

```

```

try:
    # erwartete Summe der oben angelegten Dateien: 35+28+42+29+30 = 164
    assert summe_bytes == 164, f"Erwartet 164 Byte, bekommen: {summe_bytes}"
    print(" Richtig!")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" `summe_bytes` fehlt noch.")

```

6. Aufräumen

Um dein Arbeitsverzeichnis sauber zu hinterlassen, entfernen wir den Temp-Ordner am Ende wieder. Das Modul `shutil` aus der Standardbibliothek enthält dafür die Funktion `shutil.rmtree(pfad)`, die einen Ordner *mit allen Unterordnern und Dateien* rekursiv löscht.

[!WARNING] `shutil.rmtree` fragt nicht nach! Es löscht **unwiderruflich** alles unterhalb des angegebenen Pfades. Immer eine Sekunde nachdenken, bevor du den Aufruf ausführst, und ausschließlich auf Ordner anwenden, die du selbst angelegt hast.

Wir prüfen mit `os.path.exists` vorher, ob der Ordner überhaupt noch existiert — dann kannst du diese Zelle gefahrlos auch mehrfach ausführen.

```

import os
import shutil

ziel = "temp_messdaten"
if os.path.exists(ziel):
    shutil.rmtree(ziel)
    print(f"Ordner '{ziel}' wurde entfernt.")
else:
    print(f"Ordner '{ziel}' existiert nicht (mehr) - nichts zu tun.")

```

Zusammenfassung

- `os.listdir(pfad)` liefert den Ordnerinhalt als Liste von Namen (ohne Pfad-Präfix).
- `os.path.isfile(pfad)` und `os.path.isdir(pfad)` unterscheiden Dateien und Verzeichnisse.
- `os.path.getsize(pfad)` gibt die Dateigröße in Byte zurück.
- `os.path.join(basis, name)` setzt Pfade plattformunabhängig zusammen.
- `os.makedirs(pfad, exist_ok=True)` legt Ordner inklusive fehlender Zwischenebenen an.
- `shutil.rmtree(pfad)` löscht rekursiv — mit Bedacht einsetzen!

Nächster Schritt

Weiter mit 03_aufgaben.ipynb — fünf Übungsaufgaben rund um Dateien, Ordner und Ingenieurs-Datenformate.

Datei: lessons/V15-Netzwerktechnik-Grundlagen-Protokolle-Teil1/02_praxis.ipynb

V15 – Netzwerktechnik: Praxis

Lernziele

Nach diesem Notebook kannst du ... - typische **String-Methoden** (`.split`, `.startswith`, `.isdigit`, `.strip`) anwenden, um Netzwerkadressen und URLs zu zerlegen, - eine **CSV-Datei** zeilenweise einlesen und die Anzahl der Datenzeilen bestimmen, - eine **Port-Prüffunktion** bauen, die Gültigkeit über Zeichenart und Zahlenbereich abfragt.

Zeitbudget

Ca. 30 Minuten.

TL;DR

Statt echte Netzwerk-Verbindungen aufzubauen, arbeiten wir **offline** mit vorgegebenen Strings und einer vorhandenen CSV-Datei. Das reicht, um die typischen Parser-Handgriffe für Netzwerkdaten einzuüben.

Voraussetzungen

- 00_python_recap.ipynb
- 01_theorie.ipynb

1. Eine Adresse zerlegen mit `.split`

Die Methode `.split(trennzeichen)` zerlegt einen String an jedem Auftreten des Trennzeichens und liefert eine Liste von Teilstrings. Für die Zerlegung von `host:port` ist genau das ideal. Ohne Argument wird an Whitespace getrennt.

```
adresse = "192.168.10.200:4840"
teile = adresse.split(":")
print(teile)
print("Host:", teile[0])
print("Port:", teile[1])
```

1.1 Eine IP in vier Oktette zerlegen

Dieselbe Idee, nur diesmal mit Punkt als Trennzeichen. Danach wandeln wir jedes Teil-String per `int(...)` in eine Ganzzahl um.

```
ip = "10.0.5.42"
oktette = [int(x) for x in ip.split(".")]
print(oktette)
print("Erstes Oktett:", oktette[0])
```

2. Präfixe erkennen mit `.startswith`

Um eine URL einzuordnen, willst du wissen, ob sie mit `http://` oder `https://` beginnt. `.startswith("http")` liefert einen Wahrheitswert zurück.

```
urls = ["https://beispiel.de", "opc.tcp://192.168.10.200:4840", "ftp://archiv"]
for url in urls:
    ist_web = url.startswith("http")
    print(f"{url:40s} → Web? {ist_web}")
```

3. Ziffernprüfung mit `.isdigit`

Für einen Port-String brauchen wir Zweierlei: Er muss nur aus Ziffern bestehen, und die resultierende Zahl muss im Bereich 1..65535 liegen. Den ersten Teil erledigt `.isdigit()` in einem Schritt.

```
kandidaten = ["80", "4840", "abc", "70000", "-5", "0"]
for k in kandidaten:
    print(f"{k}.isdigit() = {k.isdigit()}")
```

[!WARNING] `.isdigit()` ist bei negativen Zahlen `False`, weil das Minus-Zeichen keine Ziffer ist. Das ist in unserem Fall genau das gewünschte Verhalten (ein Port darf nicht negativ sein), kann in anderen Kontexten aber überraschen.

4. Weißraum entfernen mit `.strip`

Eingelesene Zeilen aus Dateien enden oft auf `\n` oder enthalten unschöne Leerzeichen am Rand. `.strip()` entfernt Whitespace am Anfang **und** Ende.

```
rohzeile = "  CNC-01  \n"
sauber = rohzeile.strip()
print(f"Roh:      '{rohzeile}'")
print(f"Sauber:   '{sauber}'")
```

5. Fill-in-Blank 1 – Host und Port trennen

Zerlege die Adresse in Host (String) und Port (Integer). Nutze `.split` und `int(...)`.

```

adresse = "scada.halle.a:502"
host = ""      # TODO: richtigen Wert setzen
port = 0       # TODO: als int speichern

# Selbstkontrolle
try:
    assert host == "scada.halle.a", f"host falsch: {host}"
    assert port == 502, f"port falsch: {port}"
    assert isinstance(port, int), "port muss ein int sein"
    print(" Host und Port korrekt zerlegt.")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" `host` oder `port` fehlt noch.")

```

6. Fill-in-Blank 2 – Protokoll aus URL ableiten

Setze `ist_sicher` auf `True`, wenn die URL mit `https` beginnt.

```

url = "https://mes.halle.a/api/status"
ist_sicher = False # TODO: per startswith setzen

# Selbstkontrolle
try:
    assert ist_sicher is True, "https-URL sollte als sicher erkannt werden"
    print(" Protokoll richtig erkannt.")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Die Variable `ist_sicher` existiert noch nicht.")

```

7. Fill-in-Blank 3 – Zeile säubern

Entferne Whitespace am Rand und prüfe, ob das Ergebnis exakt `"Modbus TCP"` lautet.

```

rohzeile = "  Modbus TCP\n"
sauber = rohzeile # TODO: .strip() anwenden

# Selbstkontrolle
try:
    assert sauber == "Modbus TCP", f"Nicht sauber: '{sauber}'"
    print(" Whitespace korrekt entfernt.")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Die Variable `sauber` existiert noch nicht.")

```

8. CSV-Datei zeilenweise einlesen

Eine CSV-Datei ist schlicht eine Textdatei mit einer Zeile pro Datensatz. Wir können sie mit `open()` und `with`-Block zeilenweise durchgehen. Die erste Zeile ist üblicherweise die Kopfzeile mit Spaltennamen – sie zählt nicht zu den Daten.

```
with open("netzwerk_pakete.csv", encoding="utf-8") as f:
    zeilen = f.readlines()

print(f"Gesamtanzahl Zeilen (inkl. Kopfzeile): {len(zeilen)}")
print(f"Anzahl Datensätze: {len(zeilen) - 1}")
print(f"Kopfzeile: {zeilen[0].strip()}")
print(f"Erster Datensatz: {zeilen[1].strip()}")
```

8.1 Ein einzelnes Feld extrahieren

Jede Datenzeile lässt sich an `,` aufteilen. Laut Kopfzeile ist das fünfte Feld (Index 4) der **Port**. Hier ein Beispiel für die erste Datenzeile.

```
erste_daten = zeilen[1].strip()
felder = erste_daten.split(",")
print("Felder:", felder)
print("Port aus erster Zeile:", felder[4])
```

9. Mini-Challenge – `ist_valide_port`

Schreibe eine Funktion `ist_valide_port(port_str)`, die genau dann `True` zurückgibt, wenn

- der String `port_str` **nur Ziffern** enthält **und**
- die resultierende Zahl im Bereich $1 \leq \text{port} \leq 65535$ liegt.

In allen anderen Fällen (leer, Buchstaben, zu groß, zu klein, Null) soll sie `False` zurückgeben.

```
def ist_valide_port(port_str):
    # TODO: Prüfung implementieren
    return False

# Selbstkontrolle
try:
    assert ist_valide_port("80") is True
    assert ist_valide_port("65535") is True
    assert ist_valide_port("abc") is False
    assert ist_valide_port("70000") is False
    assert ist_valide_port("0") is False
    assert ist_valide_port("") is False
    print(" Challenge gelöst.")
except AssertionError as e:
```

```

    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Die Funktion `ist_valide_port` existiert noch nicht.")

```

Zusammenfassung

- `.split`, `.startswith`, `.isdigit` und `.strip` decken den Großteil der String-Arbeit bei Netzwerkdaten ab.
- CSV-Dateien lassen sich mit `open()` und `readlines()` zeilenweise einlesen; die erste Zeile ist meist eine Kopfzeile.
- Gültige Ports lassen sich über eine Kombination aus Zeichenprüfung (`.isdigit`) und Bereichsprüfung (`1..65535`) bestimmen.

Nächster Schritt

Weiter mit 03_aufgaben.ipynb.

Datei: lessons/V16-Netzwerktechnik-Grundlagen-Protokolle-Teil2/02_praxis.ipynb

V16 – Netzwerktechnik Teil 2: Praxis

Lernziele

Nach diesem Notebook kannst du ... - `sensoren_daten.csv` mit Pandas einlesen und mit `head`, `shape`, `describe` inspizieren, - einzelne Spalten selektieren und aggregieren (`mean`, `max`, `count`), - Zeilen mit einer **Boolean-Maske** filtern, - Gruppen mit `groupby` bilden und pro Gruppe einen Mittelwert berechnen, - eine Series mit `.plot()` visualisieren.

Zeitbudget

Ca. 30 Minuten.

TL;DR

Ein durchgehendes Tutorial auf den 15 Sensor-Messungen: einlesen, verstehen, filtern, gruppieren, plotten. Drei Fill-in-Blanks mit Selbstkontrolle und eine kleine Abschluss-Challenge.

Voraussetzungen

- 00_python_recap.ipynb
- 01_theorie.ipynb

1. DataFrame laden und reinschauen

Beginn wie immer: `import pandas as pd`, dann `pd.read_csv(pfad)`. Mit `df.shape` siehst du sofort Zeilen-/Spalten-Zahl, mit `df.head()` die ersten fünf Zeilen.

```
import pandas as pd

df = pd.read_csv("sensoren_daten.csv")
print("Form:", df.shape)
df.head()
```

1.1 Numerische Übersicht mit `describe`

`describe()` zeigt pro numerischer Spalte die wichtigsten Statistiken. Achte auf `max` vs. `75 %` – ein großer Abstand deutet auf Ausreißer hin.

```
df.describe()
```

2. Spalten und Aggregationen

Mit `df["spalte"]` bekommst du die Spalte als Series und kannst direkt aggregieren.

```
print("Mittlere Temperatur:", round(df["Temperatur_C"].mean(), 2))
print("Maximum Vibration: ", df["Vibration_mm_s"].max())
print("Anzahl Messungen: ", df.shape[0])
```

2.1 Fill-in-Blank 1 – Maximum Temperatur

Speichere den maximalen Temperatur-Wert aus `df["Temperatur_C"]` in der Variablen `max_temp`.

```
# TODO: Maximum der Spalte Temperatur_C bestimmen
max_temp = 0.0

print("max_temp =", max_temp)

# Selbstkontrolle
try:
    assert max_temp == 105.2, f"Erwartet 105.2, bekommen {max_temp}"
    print(" Fill-in 1 gelöst.")
except AssertionError as e:
    print(f" {e}")
except NameError:
    print(" Variable `max_temp` fehlt.")
```


3. Filtern mit Boolean-Masken

`df[df["spalte"] bedingung]` liefert nur die Zeilen, die die Bedingung erfüllen.
Beispiel: alle Sensoren mit Status Kritisch.

```
kritisch = df[df["Status"] == "Kritisch"]
print(f"{len(kritisch)} kritische Sensoren:")
kritisch[["Sensor_ID", "Sensor_Name", "Vibration_mm_s"]]
```

3.1 Fill-in-Blank 2 – Anzahl heißer Sensoren

Zähle mit einer Boolean-Maske die Sensoren, deren Temperatur_C größer oder gleich 80 ist, und speichere die Anzahl in `anzahl_heiss`.

```
# TODO: Maske bauen, Zeilen zählen
anzahl_heiss = 0

print("anzahl_heiss =", anzahl_heiss)

# Selbstkontrolle
# Temperaturen >= 80: S013 (105.2), S004 (92.3), S010 (88.7) → 3
try:
    assert anzahl_heiss == 3, f"Erwartet 3, bekommen {anzahl_heiss}"
    print(" Fill-in 2 gelöst.")
except AssertionError as e:
    print(f" {e}")
except NameError:
    print(" Variable `anzahl_heiss` fehlt.")
```

4. Gruppieren mit groupby

Wir wollen pro Sensor-Typ (Temperatur/Vibration/Druck) den Mittelwert von Temperatur_C sehen.

```
mittel_pro_typ = df.groupby("Typ")["Temperatur_C"].mean()
print(mittel_pro_typ)
```

4.1 Fill-in-Blank 3 – Typ mit höchstem Mittelwert

Nutze `.idxmax()` auf der obigen Series, um den Namen des Sensor-Typs mit der höchsten mittleren Temperatur zu bekommen. Speichere das Ergebnis als String in `bester_typ`.

```
# TODO: mittel_pro_typ.idxmax() aufrufen
bester_typ = ""

print("bester_typ =", bester_typ)

# Selbstkontrolle
try:
```

```

    assert bester_typ == "Temperatur", f"Erwartet 'Temperatur', bekommen '{bester_typ}'"
    print(" Fill-in 3 gelöst.")
except AssertionError as e:
    print(f" {e}")
except NameError:
    print(" Variable `bester_typ` fehlt.")

```

5. Plot einer Series

Mit `.plot()` bekommst du einen Schnell-Plot. Für einen Balken-Vergleich der Gruppen setzen wir `kind="bar"`.

```

import matplotlib.pyplot as plt

mittel_pro_typ.plot(kind="bar", title="Mittlere Temperatur je Sensor-Typ")
plt.ylabel("°C")
plt.show()

```

6. Mini-Challenge – Sortierter Überblick

Berechne den Mittelwert der Spalte `Vibration_mm_s` pro Typ, sortiere die resultierende Series **absteigend** und speichere den **Typ mit der höchsten mittleren Vibration** in `top_vibration_typ`.

Tipp: `.sort_values(ascending=False)` auf der Series, danach `.index[0]` – oder kürzer `.idxmax()`.

```

# TODO: groupby nach Typ, Mittelwert Vibration_mm_s, idxmax
top_vibration_typ = ""

```

```

print("top_vibration_typ =", top_vibration_typ)

```

```

# Selbstkontrolle

```

```

# Vibration_mm_s-Mittel: Temperatur=0.222, Vibration=3.612, Druck=0.09 → Vibration

```

```

try:

```

```

    assert top_vibration_typ == "Vibration", f"Erwartet 'Vibration', bekommen '{top_vibration_typ}'"
    print(" Mini-Challenge gelöst.")
except AssertionError as e:
    print(f" {e}")
except NameError:
    print(" Variable `top_vibration_typ` fehlt.")

```

Zusammenfassung

- CSV-Dateien lädst du mit `pd.read_csv(pfad)` in einen DataFrame.
- `describe()` liefert numerische Statistik auf einen Blick.
- Mit Boolean-Masken filterst du Zeilen (`df[df["spalte"] op wert]`).

- `groupby(...)[\"ziel\"].aggregation()` ist das Standard-Werkzeug für Fragen „pro Gruppe“.
- `.idxmax()` liefert den **Index** der Zeile mit dem Maximum – das ist oft der gesuchte Name.
- `.plot()` ist der schnellste Weg zu einer ersten Visualisierung.

Nächster Schritt

Weiter mit 03_aufgaben.ipynb – fünf Übungsaufgaben auf echten Testdaten.

Datei: lessons/V17-Kryptografie-Teil1/02_praxis.ipynb

V17 – Kryptografie: Praxis

Lernziele

Nach diesem Notebook kannst du ... - mit `ord()` und `chr()` flüssig zwischen Zeichen und ASCII-Code wechseln, - eine Hilfsfunktion `caesar_zeichen(z, k)` für einen einzelnen Großbuchstaben implementieren, - die Funktion `caesar_text(text, k)` zeichenweise auf einen ganzen String anwenden, - Caesar durch Umkehren des Schlüssels wieder entschlüsseln, - einen einfachen **Brute-Force-Angriff** per Schleife programmieren.

Zeitbudget

Ca. 20 Minuten.

TL;DR

Wir bauen Caesar Schritt für Schritt zusammen: zuerst `ord/chr`, dann die Positionsformel, dann die Zeichen- und Text-Funktion, zum Schluss ein kleiner Knack-Angriff.

Voraussetzungen

- 00_python_recap.ipynb
- 01_theorie.ipynb

1. `ord()` und `chr()` sicher einsetzen

`ord(\"A\")` liefert 65, `chr(65)` liefert \"A\". Die beiden Funktionen sind zueinander invers, solange man im ASCII-Bereich bleibt. Für V17 interessiert uns nur der Bereich der **Großbuchstaben**, also 65 bis 90.

```

print(ord("A"), ord("Z"))      # 65 90
print(chr(65), chr(90))        # A Z
print(chr(ord("H")))           # H - rundtrip
print(ord(chr(75)))            # 75 - rundtrip

```

Kleine ASCII-Tabelle

Lass dir einmal alle Großbuchstaben mit zugehörigem Code ausgeben. So siehst du, wie sauber der Bereich A–Z auf 65–90 abgebildet ist.

```

for code in range(65, 91):
    print(f"{code:3d} -> {chr(code)}")

```

Fill-in 1 – ASCII-Code vom Buchstaben M

Setze die Variable `code_m` auf den ASCII-Code des Buchstabens M. Nutze `ord()`.

```

# TODO: code_m = ord("M")
code_m = 0
print("code_m =", code_m)

try:
    assert code_m == 77, f"Erwartet 77, bekommen {code_m}"
    print(" ord('M') == 77.")
except AssertionError as e:
    print(f" {e}")
except NameError:
    print(" Die Variable `code_m` existiert noch nicht.")

```

2. Vom Zeichen zur Alphabet-Position

Die Caesar-Formel arbeitet nicht mit ASCII-Codes, sondern mit **Positionen** 0–25. Zwei Schritte reichen:

1. **Zeichen** → **Position**: `pos = ord(z) - 65`
2. **Position** → **Zeichen**: `z = chr(pos + 65)`

Mit diesen beiden Umrechnungen hast du eine sichere Brücke zwischen Buchstaben und Zahlen.

```

for z in "AHMZ":
    pos = ord(z) - 65
    zurueck = chr(pos + 65)
    print(f"{z} -> Position {pos:2d} -> {zurueck}")

```

3. caesar_zeichen(z, k) – ein einzelner Buchstabe

Wir bauen jetzt eine kleine Funktion, die einen **einzelnen** Großbuchstaben `z` um `k` Schritte verschiebt. Nicht-Buchstaben (Leerzeichen, Ziffern, Punkte) lassen wir unverändert – das macht die spätere Text-Funktion viel angenehmer.

```

def caesar_zeichen(z, k):
    if "A" <= z <= "Z":
        pos = ord(z) - 65
        neu = (pos + k) % 26
        return chr(neu + 65)
    return z

print(caesar_zeichen("A", 3))    # D
print(caesar_zeichen("H", 3))    # K
print(caesar_zeichen("Z", 1))    # A - Umlauf
print(caesar_zeichen(" ", 3))    # ' ' unverändert
print(caesar_zeichen("!", 5))    # '!' unverändert

```

Fill-in 2 – Eigene Variante mit Schlüssel 7

Rufe `caesar_zeichen` für den Buchstaben "T" mit Schlüssel 7 auf und speichere das Ergebnis in `zeichen_7`. Erwartet: "A" (Position $19 + 7 = 26$, $\% 26 = 0 \rightarrow A$).

```

# TODO: Rufe caesar_zeichen("T", 7) auf
zeichen_7 = ""
print("zeichen_7 =", zeichen_7)

try:
    assert zeichen_7 == "A", f"Erwartet 'A', bekommen '{zeichen_7}'"
    print(" T um 7 verschoben landet bei A.")
except AssertionError as e:
    print(f" {e}")
except NameError:
    print(" Die Variable `zeichen_7` existiert noch nicht.")

```

4. caesar_text(text, k) – ein ganzer String

Jetzt legen wir ein Schleifchen um `caesar_zeichen`, damit wir einen beliebigen Text auf einen Rutsch verschlüsseln können. Weil `caesar_zeichen` Nicht-Buchstaben unverändert lässt, funktioniert die Text-Funktion auch für Sätze mit Leerzeichen und Satzzeichen.

```

def caesar_text(text, k):
    ergebnis = ""
    for z in text:
        ergebnis += caesar_zeichen(z, k)
    return ergebnis

print(caesar_text("HALLO WELT", 3))    # KDOOR ZHOW
print(caesar_text("VERSCHLUESSELUNG IST WICHTIG", 4))
print(caesar_text("ABCXYZ", 1))        # BCDYZA

```

Fill-in 3 – Verschlüssele "MASCHINE" mit Schlüssel 5

Berechne `caesar_text("MASCHINE", 5)` und speichere das Ergebnis in `geheim_maschine`. Erwartet: "RFXHMNSJ".

```
# TODO: caesar_text("MASCHINE", 5)
geheim_maschine = ""
print("geheim_maschine =", geheim_maschine)

try:
    assert geheim_maschine == "RFXHMNSJ", f"Erwartet 'RFXHMNSJ', bekommen '{geheim_maschine}'"
    print(" Verschlüsselung passt.")
except AssertionError as e:
    print(f" {e}")
except NameError:
    print(" Die Variable `geheim_maschine` existiert noch nicht.")
```

5. Entschlüsseln – einfach das Vorzeichen drehen

Weil Caesar eine reine Verschiebung ist, reicht es, mit `-k` erneut zu verschlüsseln, um den Klartext zu erhalten. Alternativ kannst du $(26 - k) \% 26$ als „Rück-Schlüssel“ verwenden – beide Wege führen zum selben Ergebnis.

```
geheim = caesar_text("ANGRIFF BEI MORGENGRAUEN", 7)
print("Geheim:", geheim)
print("Klar (-k):", caesar_text(geheim, -7))
print("Klar (26-k) % 26:", caesar_text(geheim, (26 - 7) % 26))
```

6. Mini-Challenge – Brute-Force eines fremden Geheimtexts

Jemand hat dir folgenden Geheimtext zugesteckt – du kennst den Schlüssel nicht:

KDOOR YRP JHKHLP HQ GLHQVW

Nee, so ergibt das nichts. Aber du weißt, dass es sich um Caesar handelt. Dafür gibt es **genau 25 mögliche Schlüssel**, und du probierst sie einfach alle aus. Der einzige lesbare Text unter den 25 Ergebnissen ist der Klartext.

Führe die nächste Zelle aus und suche in der Ausgabe die Zeile, die sich wie deutscher Text liest.

```
geheim = "KDOOR YRP JHKHLP HQ GLHQVW"
for k in range(1, 26):
    versuch = caesar_text(geheim, -k)
    print(f"k = {k:2d}: {versuch}")
```

Tip: Die lesbare Zeile enthält die Worte HALLO und DIENST – prüfe den zugehörigen Schlüssel. Genau so händisch gehen Angreifer

bei Caesar vor – 25 Sekunden reichen.

Zusammenfassung

- `caesar_zeichen(z, k)` verschiebt einen Großbuchstaben um `k` Schritte mit `% 26`.
- `caesar_text(text, k)` wendet das zeichenweise auf einen String an.
- Entschlüsseln = `caesar_text(geheim, -k)`.
- Brute-Force = alle 25 Schlüssel ausprobieren und den lesbaren Text auswählen.

Nächster Schritt

Weiter mit `03_aufgaben.ipynb` – fünf Übungsaufgaben zur Selbstkontrolle.

Datei: `lessons/V18-Kryptografie-Teil2/02_praxis.ipynb`

V18 – Kryptografie Teil 2: Praxis mit `hashlib`

Lernziele

Nach diesem Notebook kannst du ... - mit `hashlib` MD5- und SHA-256-Hashes von Strings berechnen und als Hexadezimal-String ausgeben, - erklären, warum vor dem Hashen immer ein `.encode("utf-8")` nötig ist, - eine einfache Passwort-Vergleichs-Funktion implementieren, - einen Mini-Wörterbuch-Angriff auf einen gegebenen SHA-256-Hash durchführen.

Zeitbudget

Ca. 25 Minuten.

TL;DR

`hashlib.sha256(b"...").hexdigest()` liefert dir den SHA-256-Hash als 64 Zeichen langen Hexadezimal-String. Der Input muss vom Typ `bytes` sein – deshalb immer vorher `.encode("utf-8")` auf Strings anwenden. Zum Vergleichen zweier Hashes reicht ein schlichtes `==`.

Voraussetzungen

- `00_python_recap.ipynb` (bytes vs. str, Listen-Lookup)
- `01_theorie.ipynb` (Hash-Eigenschaften, Wörterbuch-Angriff)

1. hashlib importieren

Das Modul `hashlib` ist Teil der Python-Standardbibliothek – du brauchst nichts nachzuinstallieren. Es enthält unter anderem die Hash-Funktionen `md5`, `sha1`, `sha256`, `sha512` und weitere.

```
import hashlib

print("Verfügbare garantierte Algorithmen:")
print(sorted(hashlib.algorithms_guaranteed))
```

2. Der erste MD5-Hash

Der einfachste Aufruf sieht so aus: Du übergibst `hashlib.md5(...)` die Eingabe als `bytes`-Objekt und hängst `.hexdigest()` an. Das Ergebnis ist ein **Hexadezimal-String** – bei MD5 immer exakt 32 Zeichen lang, was 128 Bit entspricht.

```
import hashlib

hash_md5 = hashlib.md5(b"hallo").hexdigest()
print(hash_md5)
print("Länge:", len(hash_md5), "Hex-Zeichen")
```

Der erwartete Output ist `598d4c200461b81522a3328565c25f7c`. Jede Python-Installation auf jedem Betriebssystem liefert denselben Wert – daran erkennst du die **deterministische** Eigenschaft aus der Theorie.

3. SHA-256 – der moderne Standard

Der Aufruf ist völlig analog, nur dass du `hashlib.sha256` statt `hashlib.md5` verwendest. Der Hexadezimal-Output ist dafür **doppelt so lang** (64 Zeichen = 256 Bit).

```
import hashlib

hash_sha = hashlib.sha256(b"test").hexdigest()
print(hash_sha)
print("Länge:", len(hash_sha), "Hex-Zeichen")
```

Für `b"test"` ist der SHA-256-Hash `9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a`. Im Gegensatz zum 32-Zeichen-MD5-Wert ist das deutlich mehr Sicherheits-Spielraum gegen Brute-Force.

4. `.encode()` nicht vergessen

Ein häufiger Anfänger-Fehler: Man übergibt einen normalen `str` statt `bytes`. Python wirft dann einen `TypeError`, denn eine Hash-Funktion weiß nicht,

welche Kodierung du möchtest. Führe das nächste Beispiel aus, um den Fehler zu sehen – der `try/except`-Block fängt ihn ab.

```
import hashlib
try:
    hashlib.sha256("test").hexdigest()    # str, nicht bytes!
except TypeError as e:
    print("TypeError:", e)
```

Die korrekte Form ist `hashlib.sha256("test".encode("utf-8")).hexdigest()`. Die beiden folgenden Zeilen sind äquivalent.

```
import hashlib

h1 = hashlib.sha256(b"test").hexdigest()
h2 = hashlib.sha256("test".encode("utf-8")).hexdigest()
print(h1)
print(h2)
print("gleich?", h1 == h2)
```

5. Lawineneffekt live

Wir ändern die Eingabe um **ein einziges Zeichen** und beobachten, wie sich der Hash komplett umstellt.

```
import hashlib

for wort in ["passwort", "passwort!", "Passwort", "passwort "]:
    h = hashlib.sha256(wort.encode("utf-8")).hexdigest()
    print(f"{wort!r:15s} -> {h}")
```

Obwohl die Eingaben sehr ähnlich aussehen, haben die Hashes optisch **nichts** gemeinsam. Das ist der Lawineneffekt aus dem Theorie-Notebook in Aktion.

6. Fill-in: MD5 von „geheim“

Baue den MD5-Hash des Strings "geheim" und speichere das Ergebnis als Hex-String in `ergebnis`.

```
import hashlib
# TODO: Berechne den MD5-Hash von "geheim" (als bytes!) und weise ihn ergebnis zu.
ergebnis = ""    # hier anpassen

# Selbstkontrolle
try:
    erwartet = hashlib.md5(b"geheim").hexdigest()
    assert ergebnis == erwartet, f"Erwartet {erwartet}, bekommen {ergebnis!r}"
    print(" Richtig!")
except AssertionError as e:
```

```

        print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Die Variable `ergebnis` existiert noch nicht.")

```

7. Passwort-Check-Funktion

Ein realistischer Anwendungsfall: Der Server hat in der Datenbank den SHA-256-Hash eines Passworts gespeichert. Beim Login gibt der Nutzer seine Eingabe ein, und wir müssen prüfen, ob sie zum gespeicherten Hash passt.

Die Funktion ist nur drei Zeilen lang: Eingabe hashen, vergleichen, Boolean zurückgeben.

```

import hashlib

def passwort_korrekt(eingabe: str, gespeicherter_hash: str) -> bool:
    hash_eingabe = hashlib.sha256(eingabe.encode("utf-8")).hexdigest()
    return hash_eingabe == gespeicherter_hash

# Annahme: In der Datenbank steht der SHA-256-Hash von "s3cret!"
gespeicherter_hash = hashlib.sha256(b"s3cret!").hexdigest()

print(passwort_korrekt("s3cret!", gespeicherter_hash))    # True
print(passwort_korrekt("falsch", gespeicherter_hash))    # False
print(passwort_korrekt("S3cret!", gespeicherter_hash))    # False (Gross-/Kleinschreibung!)

```

8. Fill-in: Passwort-Check

Implementiere die Funktion `meine_pruefung(eingabe, hash_wert)`, die genau wie oben `True` liefert, wenn der SHA-256-Hash der Eingabe dem `hash_wert` entspricht.

```

import hashlib

def meine_pruefung(eingabe, hash_wert):
    # TODO: Hashe die eingabe mit SHA-256 und vergleiche mit hash_wert
    return False

# Selbstkontrolle
try:
    h = hashlib.sha256(b"robot").hexdigest()
    assert meine_pruefung("robot", h) is True, "korrekte Eingabe sollte True liefern"
    assert meine_pruefung("falsch", h) is False, "falsche Eingabe sollte False liefern"
    print(" Richtig!")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Die Funktion `meine_pruefung` existiert noch nicht.")

```

9. Fill-in: Hash-Länge prüfen

Speichere in der Variablen `laenge_sha256` die Anzahl der **Hex-Zeichen** eines SHA-256-Hashes. Tipp: Berechne einen beliebigen SHA-256-Hash und nimm `len(...)`.

```
import hashlib

# TODO: Ermittle die Hex-Länge eines SHA-256-Hashes
laenge_sha256 = 0 # hier anpassen

# Selbstkontrolle
try:
    assert laenge_sha256 == 64, f"SHA-256 hat 64 Hex-Zeichen, du hast: {laenge_sha256}"
    print(" Richtig - 64 Hex-Zeichen = 256 Bit.")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Die Variable `laenge_sha256` existiert noch nicht.")
```

10. Mini-Challenge: Wörterbuch-Angriff

Gegeben ist der folgende SHA-256-Hash. Er ist der Hash eines extrem unsicheren Passworts. Nutze die Wortliste `["test", "1234", "admin", "passwort"]` und finde heraus, welcher Eintrag passt. Speichere dein Ergebnis in der Variablen `knackpasswort`.

```
import hashlib

ziel_hash = hashlib.sha256(b"admin").hexdigest() # Der zu knackende Hash
wortliste = ["test", "1234", "admin", "passwort"]

# TODO: Hashe jedes Wort aus der Wortliste und vergleiche mit ziel_hash.
#       Sobald du eine Übereinstimmung findest, weise das Wort an knackpasswort zu.
knackpasswort = None

# Selbstkontrolle
try:
    assert knackpasswort == "admin", f"Erwartet 'admin', bekommen: {knackpasswort!r}"
    print(" Richtig - Passwort geknackt. Genau das ist ein Wörterbuch-Angriff.")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Die Variable `knackpasswort` existiert noch nicht.")
```

Zusammenfassung

Aufgabe	Idiom
MD5 eines Strings	<code>hashlib.md5(s.encode("utf-8")).hexdigest()</code>
SHA-256 eines Strings	<code>hashlib.sha256(s.encode("utf-8")).hexdigest()</code>
Hashes vergleichen	einfach mit <code>==</code>
Passwort prüfen	Eingabe hashen, mit gespeichertem Hash vergleichen
Wörterbuch-Angriff	Wortliste durchlaufen, hashen, vergleichen

Nächster Schritt

Jetzt bist du für die fünf Übungsaufgaben in `03_aufgaben.ipynb` bereit.

Datei: `lessons/V19-Datenbanken-Teil1/02_praxis.ipynb`

V19 – Datenbanken Teil 1: Praxis

Lernziele

Nach diesem Notebook kannst du ... - eine SQLite-Verbindung mit `sqlite3.connect(":memory:")` im `with`-Block öffnen, - eine Tabelle mit `CREATE TABLE`, Einträge mit `INSERT`, Abfragen mit `SELECT` und Änderungen mit `UPDATE/DELETE` ausführen, - Werte mit **?-Parameter-Binding** sicher übergeben, - eine **CSV-Datei** mit `csv.DictReader` zeilenweise in eine SQLite-Tabelle überführen.

Zeitbudget

Ca. 25 Minuten.

TL;DR

Wir arbeiten mit einer In-Memory-Datenbank, bauen Schritt für Schritt die **Produkte**-Tabelle auf, üben alle CRUD-Operationen und importieren am Ende die sechs Zeilen aus `testdaten/produkte.csv`. Drei Fill-in-Blanks bauen dir unterwegs Muskelgedächtnis auf.

Voraussetzungen

- `01_theorie.ipynb`.

1. Verbindung aufbauen

Wir beginnen mit einer reinen **In-Memory-Datenbank**. Der Aufruf `sqlite3.connect(":memory:")` erzeugt eine leere Datenbank im RAM; mit `conn.cursor()` holen wir uns den Cursor, mit dem alle SQL-Befehle abgesetzt werden.

```
import sqlite3

conn = sqlite3.connect(":memory:")
cur = conn.cursor()
print("Verbindung offen:", conn)
print("Cursor:", cur)
```

2. Tabelle anlegen

Wir erzeugen die Tabelle `Maschinen` mit ID, Name, Typ und einem Anschaffungsjahr. Der Primärschlüssel ist die `Maschinen_ID`.

```
cur.execute('''
CREATE TABLE Maschinen (
    Maschinen_ID INTEGER PRIMARY KEY,
    Name          TEXT    NOT NULL,
    Typ           TEXT    NOT NULL,
    Baujahr       INTEGER CHECK (Baujahr >= 1900)
)
''')
print("Tabelle Maschinen angelegt.")
```

Zur Kontrolle: SQLite kennt `sqlite_master`

SQLite speichert sein Schema in einer internen Tabelle `sqlite_master`. Eine Abfrage gegen sie zeigt, welche Tabellen gerade existieren.

```
for row in cur.execute("SELECT name FROM sqlite_master WHERE type='table'"):
    print("Tabelle:", row[0])
```

3. INSERT mit Parameter-Binding

Jetzt fügen wir drei Maschinen ein. Wichtig: **immer** `?`-Platzhalter und Tupel als zweites Argument.

```
cur.execute("INSERT INTO Maschinen VALUES (?, ?, ?, ?)", (1, "Drehmaschine D-7", "Drehen", 2010))
cur.execute("INSERT INTO Maschinen VALUES (?, ?, ?, ?)", (2, "Fraesmaschine F-12", "Fraesen", 2015))
cur.execute("INSERT INTO Maschinen VALUES (?, ?, ?, ?)", (3, "3D-Drucker P-30", "Additiv", 2018))

print("Eingefuegte Zeilen insgesamt:", cur.execute("SELECT COUNT(*) FROM Maschinen").fetchone()[0])
```

4. SELECT – lesen, filtern, sortieren

Die Grundform `SELECT ... FROM ...` liefert alle Zeilen. Mit `WHERE`, `ORDER BY` und `LIMIT` schränkst du das Ergebnis ein.

```
print("Alle Maschinen:")
for pid, name, typ, baujahr in cur.execute("SELECT * FROM Maschinen"):
    print(f"  #{pid} {name:<22} Typ={typ:<8} Baujahr={baujahr}")

print("\nNur Maschinen ab 2018, sortiert nach Baujahr:")
for name, bj in cur.execute("SELECT Name, Baujahr FROM Maschinen WHERE Baujahr >= ? ORDER BY Baujahr"):
    print(f"  {bj} {name}")
```

Fill-in-Blank 1 — Älteste Maschine

Hol die **älteste** Maschine (kleinstes Baujahr) mit einer einzigen Query. Nutze `ORDER BY` und `LIMIT`. Speichere den Namen in `aeltester_name`.

```
aeltester_name = None
# TODO: Query absetzen, fetchone() verwenden, Name extrahieren

try:
    assert aeltester_name == "Drehmaschine D-7", f"Erwartet 'Drehmaschine D-7', bekommen: {aeltester_name}"
    print("  SELECT + ORDER BY + LIMIT sitzt.")
except AssertionError as e:
    print(f"  Noch nicht ganz: {e}")
except NameError:
    print("  Die Variable `aeltester_name` existiert noch nicht.")
```

5. UPDATE – Werte ändern

Der 3D-Drucker wird umbenannt, weil die Firma einen neuen Naming-Standard einführt. Denk an die `WHERE`-Klausel!

```
cur.execute("UPDATE Maschinen SET Name = ? WHERE Maschinen_ID = ?", ("3D-Drucker PRO-30", 3))
for row in cur.execute("SELECT * FROM Maschinen WHERE Maschinen_ID = ?", (3,)):
    print("Nach UPDATE:", row)
```

6. DELETE – Zeilen löschen

Die alte Drehmaschine D-7 wird außer Dienst gestellt und aus der Tabelle entfernt.

```
cur.execute("DELETE FROM Maschinen WHERE Maschinen_ID = ?", (1,))
print("Verbleibende Zeilen:", cur.execute("SELECT COUNT(*) FROM Maschinen").fetchone()[0])
for row in cur.execute("SELECT * FROM Maschinen"):
    print(" ", row)
```

Fill-in-Blank 2 — Sicheres INSERT

Füge eine neue Maschine mit Maschinen_ID = 4, Name "Laserschneider L-5", Typ "Laser", Baujahr 2024 ein. Nutze Parameter-Binding.

```
neuer_name = "Laserschneider L-5"
# TODO: cur.execute(...) mit Parameter-Binding aufrufen

try:
    row = cur.execute("SELECT Name, Typ, Baujahr FROM Maschinen WHERE Maschinen_ID = ?", (4,))
    assert row is not None, "Maschine 4 nicht gefunden - INSERT fehlt"
    assert row == ("Laserschneider L-5", "Laser", 2024), f"Falsche Werte: {row}"
    print(" INSERT mit Parameter-Binding korrekt.")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
```

7. Warnung: Niemals String-Konkatenation!

Zur Einprägung zeigen wir noch einmal, wie **falsch** aussieht, was du nicht tun sollst. Der Code ist bewusst nur als String, er wird nicht ausgeführt.

```
boeser_code = '''
name = input("Name? ")
# UNSICHER - niemals so schreiben!
cur.execute(f"SELECT * FROM Maschinen WHERE Name = '{name}'")
'''

print(boeser_code)
print("-> bei Eingabe '\" OR '1'='1\" bekommt der Angreifer ALLE Zeilen.")
```

8. CSV-Import – produkte.csv in SQLite

Jetzt kombinieren wir die Module `csv` und `sqlite3`. Wir legen eine neue Tabelle `Produkte` an und fügen zeilenweise alle sechs Produkte aus `testdaten/produkte.csv` ein.

```
import csv

cur.execute('''
CREATE TABLE Produkte (
    Produkt_ID          INTEGER PRIMARY KEY,
    Produktname         TEXT    NOT NULL,
    Produktionszeit_Minuten INTEGER,
    Material_pro_Stueck_kg REAL
)
''')

with open("testdaten/produkte.csv", encoding="utf-8") as f:
    reader = csv.DictReader(f)
```

```

for zeile in reader:
    cur.execute(
        "INSERT INTO Produkte VALUES (?, ?, ?, ?)",
        (
            int(zeile["Produkt_ID"]),
            zeile["Produktname"],
            int(zeile["Produktionszeit_Minuten"]),
            float(zeile["Material_pro_Stueck_kg"]),
        ),
    )

print("Importierte Produkte:", cur.execute("SELECT COUNT(*) FROM Produkte").fetchone()[0])
for row in cur.execute("SELECT * FROM Produkte ORDER BY Produkt_ID"):
    print(" ", row)

```

Fill-in-Blank 3 — Filter-Query

Liste alle Produkte auf, deren **Material_pro_Stueck_kg** größer als 2.0 ist. Speichere das Ergebnis als Liste von Tupeln in `schwere_produkte` (nur Spalten `Produktname`, `Material_pro_Stueck_kg`).

```

schwere_produkte = []
# TODO: SELECT mit WHERE, fetchall()

try:
    erwartete_namen = {"Welle W-18", "Flansch F-90", "Gehaeuse G-55"}
    tatsaechliche_namen = {name for name, _masse in schwere_produkte}
    assert tatsaechliche_namen == erwartete_namen, (
        f"Erwartet {erwartete_namen}, bekommen: {tatsaechliche_namen}"
    )
    assert all(masse > 2.0 for _name, masse in schwere_produkte), "Alle Massen muessen > 2.0 sein"
    print(" Filter-Query korrekt.")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Die Variable `schwere_produkte` existiert noch nicht.")

```

9. Aufräumen

Ein letztes `conn.close()` schließt die Verbindung. Bei In-Memory-Datenbanken ist das kosmetisch – die Daten sind ohnehin flüchtig.

```

conn.close()
print("Verbindung geschlossen.")

```


Zusammenfassung

- `sqlite3.connect(":memory:")` liefert eine blitzschnelle Sandbox für Übungen.
- CREATE, INSERT, SELECT, UPDATE, DELETE decken das komplette CRUD ab.
- ?-Parameter sind Pflicht – String-Konkatenation ist ein Sicherheits-Risiko.
- CSV-Import ist ein einfacher Dreier-Schritt: Tabelle anlegen → zeilenweise lesen → INSERT mit Typ-Konvertierung.

Nächster Schritt

Weiter mit 03_aufgaben.ipynb – dort übst du CRUD an fünf Aufgaben auf der Produkte-Tabelle.

Datei: lessons/V20-Datenbanken-Teil2/02_praxis.ipynb

V20 – Datenbanken Teil 2: Praxis

Lernziele

Nach diesem Notebook kannst du ... - eine kleine deterministische In-Memory-DB mit `setup_db()` aufbauen, - INNER JOIN, LEFT JOIN, GROUP BY, HAVING und eine einfache Subquery praktisch anwenden, - erkennen, wann INNER JOIN und wann LEFT JOIN das richtige Werkzeug ist.

Zeitbudget

Ca. 25 Minuten.

TL;DR

Wir legen eine kleine, feste Maschinen- und Wartungs-Datenbank an und üben die Theorie Schritt für Schritt an konkreten Abfragen. Drei Fill-in-Blanks bauen dir unterwegs Muskelgedächtnis auf.

Voraussetzungen

- 01_theorie.ipynb.

1. `setup_db()` – unsere feste Datenbasis

Damit alle Ergebnisse reproduzierbar bleiben, legen wir die gesamte Datenbasis in einer einzigen Funktion `setup_db()` an. Sie öffnet eine frische In-Memory-DB, legt die Tabellen `Maschinen` und `Wartungen` an und füllt sie mit **denselben**

festen Zeilen – ohne random, ohne externe Dateien. Jede Zelle, die mit conn = setup_db() beginnt, startet dadurch aus exakt demselben Zustand.

```
import sqlite3

def setup_db() -> sqlite3.Connection:
    """Legt eine frische In-Memory-DB mit deterministischen Daten an."""
    conn = sqlite3.connect(":memory:")
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE Maschinen (
            Maschinen_ID INTEGER PRIMARY KEY,
            Name          TEXT NOT NULL,
            Typ           TEXT NOT NULL,
            Baujahr       INTEGER NOT NULL
        )
    """)
    cur.execute("""
        CREATE TABLE Wartungen (
            Wartung_ID    INTEGER PRIMARY KEY,
            Maschinen_ID  INTEGER NOT NULL,
            Wartungstyp   TEXT NOT NULL,
            Datum         TEXT NOT NULL,
            Kosten        REAL NOT NULL,
            FOREIGN KEY (Maschinen_ID) REFERENCES Maschinen(Maschinen_ID)
        )
    """)
    maschinen = [
        (1, "CNC-Fräse 01", "Fräse", 2018),
        (2, "CNC-Fräse 02", "Fräse", 2020),
        (3, "Drehmaschine 01", "Drehbank", 2016),
        (4, "Drehmaschine 02", "Drehbank", 2021),
        (5, "Roboter-Arm R1", "Roboter", 2019),
    ]
    wartungen = [
        (1, 1, "Inspektion", "2024-01-10", 150.00),
        (2, 1, "Reparatur", "2024-03-05", 1200.00),
        (3, 2, "Inspektion", "2024-02-14", 180.00),
        (4, 3, "Ölwechsel", "2024-01-20", 90.00),
        (5, 3, "Reparatur", "2024-04-12", 2500.00),
        (6, 3, "Inspektion", "2024-05-18", 200.00),
        (7, 4, "Inspektion", "2024-02-28", 160.00),
        (8, 5, "Kalibrierung", "2024-03-22", 450.00),
    ]
    cur.executemany("INSERT INTO Maschinen VALUES (?, ?, ?, ?)", maschinen)
    cur.executemany("INSERT INTO Wartungen VALUES (?, ?, ?, ?, ?)", wartungen)
```

```

        conn.commit()
    return conn

# Kurztest
conn = setup_db()
cur = conn.cursor()
cur.execute("SELECT COUNT(*) FROM Maschinen")
print("Maschinen:", cur.fetchone()[0])
cur.execute("SELECT COUNT(*) FROM Wartungen")
print("Wartungen:", cur.fetchone()[0])
conn.close()

```

2. INNER JOIN – Wartung + Maschine zusammenführen

Wir fragen zu jeder Wartung den passenden Maschinennamen ab. Das klassische JOIN-Muster: FROM Maschinen M INNER JOIN Wartungen W ON M.Maschinen_ID = W.Maschinen_ID.

```

conn = setup_db()
cur = conn.cursor()
cur.execute("""
    SELECT M.Name, W.Wartungstyp, W.Kosten
    FROM Maschinen M
    INNER JOIN Wartungen W ON M.Maschinen_ID = W.Maschinen_ID
    ORDER BY M.Name, W.Datum
""")
for zeile in cur.fetchall():
    print(zeile)
conn.close()

```

3. LEFT JOIN – auch Maschinen ohne Wartungen anzeigen

Zum Vergleich fügen wir testweise eine sechste Maschine **Roboter-Arm R2** ein, die noch **keine** Wartung hatte. Mit INNER JOIN würde sie unsichtbar bleiben; mit LEFT JOIN erscheint sie mit NULL-Werten auf der rechten Seite.

```

conn = setup_db()
cur = conn.cursor()
cur.execute("INSERT INTO Maschinen VALUES (?, ?, ?, ?)",
            (6, "Roboter-Arm R2", "Roboter", 2023))

cur.execute("""
    SELECT M.Name, W.Wartungstyp, W.Kosten
    FROM Maschinen M
    LEFT JOIN Wartungen W ON M.Maschinen_ID = W.Maschinen_ID
    ORDER BY M.Name
""")

```

```

for zeile in cur.fetchall():
    print(zeile)
conn.close()

```

Beachte die letzte Zeile: ("Roboter-Arm R2", None, None). Genau das sucht man in Aufgabe 5 heraus.

4. GROUP BY + SUM – Top-Kostenverursacher

Wer hat uns am meisten gekostet? Wir gruppieren die Wartungen pro Maschine und addieren die Kosten. ORDER BY `gesamt` DESC LIMIT 1 liefert den Spitzenreiter.

```

conn = setup_db()
cur = conn.cursor()
cur.execute("""
    SELECT M.Name, SUM(W.Kosten) AS gesamt
    FROM Maschinen M
    INNER JOIN Wartungen W ON M.Maschinen_ID = W.Maschinen_ID
    GROUP BY M.Maschinen_ID
    ORDER BY gesamt DESC
""")
for zeile in cur.fetchall():
    print(f"{zeile[0]:20s} {zeile[1]:>8.2f} €")
conn.close()

```

5. GROUP BY + COUNT – Wartungstypen zählen

Welche Art Wartung kommt am häufigsten vor? Wir gruppieren direkt nach Wartungstyp.

```

conn = setup_db()
cur = conn.cursor()
cur.execute("""
    SELECT Wartungstyp, COUNT(*) AS anzahl
    FROM Wartungen
    GROUP BY Wartungstyp
    ORDER BY anzahl DESC, Wartungstyp
""")
print(cur.fetchall())
conn.close()

```

6. HAVING – nur teure Maschinen

Wir wollen jetzt nur die Maschinen sehen, deren **summierte** Wartungskosten 1000 € übersteigen. Das ist ein Filter **auf Gruppenebene** und muss deshalb mit HAVING formuliert werden:

```

conn = setup_db()
cur = conn.cursor()
cur.execute("""
    SELECT M.Name, SUM(W.Kosten) AS gesamt
    FROM Maschinen M
    INNER JOIN Wartungen W ON M.Maschinen_ID = W.Maschinen_ID
    GROUP BY M.Maschinen_ID
    HAVING gesamt > 1000
    ORDER BY gesamt DESC
""")
print(cur.fetchall())
conn.close()

```

7. Subquery – über dem Durchschnitt

Mit einer skalaren Subquery können wir nach Maschinen fragen, deren Gesamtkosten **über dem Durchschnitt** aller Gesamtkosten liegen.

```

conn = setup_db()
cur = conn.cursor()
cur.execute("""
    SELECT Name FROM Maschinen
    WHERE Maschinen_ID IN (
        SELECT Maschinen_ID FROM Wartungen
        GROUP BY Maschinen_ID
        HAVING SUM(Kosten) > (SELECT AVG(Kosten) FROM Wartungen)
    )
""")
print(cur.fetchall())
conn.close()

```

8. Fill-in-Blank 1 – COUNT

Ergänze die Abfrage so, dass `anzahl` auf die Gesamtzahl der Einträge in Wartungen gesetzt wird (erwarteter Wert: 8).

```

conn = setup_db()
cur = conn.cursor()

# TODO: führe die passende SQL-Abfrage aus
cur.execute("SELECT 0") # ersetzen!
anzahl = cur.fetchone()[0]
conn.close()

try:
    assert anzahl == 8, f"Erwartet 8, bekommen: {anzahl}"
    print(" Fill-in 1 gelöst.")

```

```

except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Variable `anzahl` fehlt.")

```

9. Fill-in-Blank 2 – SUM + GROUP BY

Fülle die Lücken so, dass `ergebnis` eine Liste [(Wartungstyp, Summe der Kosten)] enthält, sortiert **alphabetisch** nach Wartungstyp.

```

conn = setup_db()
cur = conn.cursor()

# TODO: passende GROUP-BY-Abfrage
cur.execute("SELECT 'TODO', 0 WHERE 0") # ersetzen!
ergebnis = cur.fetchall()
conn.close()

try:
    assert ergebnis == [
        ("Inspektion", 690.0),
        ("Kalibrierung", 450.0),
        ("Reparatur", 3700.0),
        ("Ölwechsel", 90.0),
    ], f"Ergebnis stimmt noch nicht: {ergebnis}"
    print(" Fill-in 2 gelöst.")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Variable `ergebnis` fehlt.")

```

10. Fill-in-Blank 3 – INNER JOIN + MAX

Schreibe eine Abfrage, die den Namen der Maschine und die Kosten der **teuersten einzelnen Wartung** zurückliefert. Das Ergebnis soll in `teuerste` stehen und ein Tupel (Name, Kosten) sein.

```

conn = setup_db()
cur = conn.cursor()

# TODO: JOIN + ORDER BY Kosten DESC LIMIT 1
cur.execute("SELECT 'TODO', 0.0") # ersetzen!
teuerste = cur.fetchone()
conn.close()

try:
    assert teuerste == ("Drehmaschine 01", 2500.0), f"Erwartet ('Drehmaschine 01', 2500.0),

```

```

    print(" Fill-in 3 gelöst.")
except AssertionError as e:
    print(f" Noch nicht ganz: {e}")
except NameError:
    print(" Variable `teuerste` fehlt.")

```

Zusammenfassung

- `setup_db()` liefert uns eine reproduzierbare Ausgangslage.
- `INNER JOIN` und `LEFT JOIN` unterscheiden sich nur darin, ob Zeilen ohne Partner wegfallen.
- `GROUP BY` + Aggregatfunktion + `ORDER BY` ist das Standard-Muster für Kennzahlen.
- `HAVING` filtert **Gruppen**, Subqueries erlauben Zwischenergebnisse als Bausteine.

Nächster Schritt

Weiter mit 03_aufgaben.ipynb.

Zusätzliche Python-Dateien aus den Kapiteln V01–V20

Datei: lessons/V04-Boolsche-Algebra-Teil2/p2.py

```

# Beispiel: Notenbewertung
punkte = int(input("Erreichte Punktzahl (0-100): "))

if punkte >= 90:
    note = "Sehr gut (1)"
else:
    if punkte >= 80:
        note = "Gut (2)"
    else:
        if punkte >= 70:
            note = "Befriedigend (3)"
        else:
            if punkte >= 60:
                note = "Ausreichend (4)"
            else:
                note = "Nicht bestanden (5)"

print(f"Deine Note: {note}")

```

```

# Beispiel: Notenbewertung
punkte = int(input("Erreichte Punktzahl (0-100): "))

if punkte >= 90:
    note = "Sehr gut (1)"
elif punkte >= 80:
    note = "Gut (2)"
elif punkte >= 70:
    note = "Befriedigend (3)"
elif punkte >= 60:
    note = "Ausreichend (4)"
else:
    note = "Nicht bestanden (5)"

print(f"Deine Note: {note}")

```

Datei: lessons/V05-Programm-Ablauf-Plaene-Teil1/p1.py

```

# Dein Code hier
drehzahl = int(input("Aktuelle Spindeldrehzahl (U/min): "))

if drehzahl > 5000:
    print(" Error: Viel zu Hohe Drehzahl!!!! Abbruch.")
else:
    if drehzahl > 3000:
        print(" WARNUNG: Hohe Drehzahl! Werkzeugverschleiß prüfen.")
    else:
        print("alles Super")

```

Datei: lessons/V06-Programm-Ablauf-Plaene-Teil2/is-prime.py

```

# BEGIN IstPrimzahl
#   EINGABE(n)
#   IF n < 2 THEN
#       AUSGABE("Keine Primzahl")
#   ELSE
#       ist_prim ← WAHR
#       i ← 2
#       WHILE i * i ≤ n DO
#           IF n MOD i = 0 THEN
#               ist_prim ← FALSCH

```



```

#             ENDIF
#             i ← i + 1
#         ENDWHILE
#         IF ist_prim THEN
#             AUSGABE("Primzahl")
#         ELSE
#             AUSGABE("Keine Primzahl")
#         ENDIF
#     ENDIF
# END IstPrimzahl

# prime_candidate = int(input("Gib eine Zahl ein: "))

for prime_candidate in range(10000000000,100000000000):
    if prime_candidate < 2:
        print("keine Primzahl!")
    else:
        ist_prim = True
        aktueller_teiler = 2
        while aktueller_teiler * aktueller_teiler <= prime_candidate:
            if prime_candidate % aktueller_teiler == 0:
                ist_prim = False
                break
            aktueller_teiler += 1
        if ist_prim:
            print(f"{prime_candidate} ist eine Primzahl!!")
        # else:
        #     print(f"{prime_candidate} ist keine Primzahl.")

```

Datei: lessons/V06-Programm-Ablauf-Plaene-Teil2/listen-beispiel.py

```

# messwerte = [27,2,5,8,11,13]
# messwerte2 = ["W", 27, True]
# # messwerte3 = []
# # i = -2
# # print(messwerte[i])

# messwerte3 = [messwerte, messwerte2]
# # print(messwerte3)

# # messwerte3.append(["r","t"])
# # print(messwerte3)
# summe = 0

```

```

# # while summe > 10:
# #     print("Hallo")
# #     summe += 1

# for index, messwert in enumerate(messwerte2):
#     print(value)
#     summe += i # summe = summe +i

# print(f"summe: {summe}")

r = list(range(10))
print(r)

```

Datei: lessons/V06-Programm-Ablauf-Plaene-Teil2/p1.py

```

#Drehen-Sequenz (U/min): 200 400 600 800 1000 1200 1400 1600 1800 2000

print("Drehen-Sequenz (U/min):", end="")

for i in range(3000,-1,-500):
    print(i,end=" ")

```

Datei: lessons/V06-Programm-Ablauf-Plaene-Teil2/p2.py

```

# Anzahl der Getriebestufen: 3
# Übersetzung Stufe 1: 2.5
# Übersetzung Stufe 2: 1.8
# Übersetzung Stufe 3: 3.0
#
# Gesamt-Übersetzung: 13.50
# Eingangsdrehzahl: 1500 U/min
# Ausgangsdrehzahl: 111.11 U/min

anzahl_getriebe_stufen = int(input("Anzahl der Getriebestufen: "))

# if getriebe_stufen >= 1:
#     stufe1 = input("Übersetzung Stufe 1: ")

```

```

#     print(stufe1)
# if getriebe_stufen >= 2:
#     stufe2 = input("Übersetzung Stufe 2: ")
#     print(stufe2)
stufen = []
for i in range(anzahl_getriebe_stufen):
    temp = float(input(f"Übersetzung Stufe {i+1}: "))
    if True:
        pass
    else:
        stufen.append(temp)
print("-----")

gesamt_uebersetzung = 1
for i in stufen:
    gesamt_uebersetzung = gesamt_uebersetzung * i

print(stufen)

```

Datei: lessons/V07-Software-Engineering-KISS-DRY-SRP/p3.py

```

#
# Anziehdrehmoment-Tabelle (trocken, verzinkt)
#
# Gewinde | 4.6 [Nm] | 8.8 [Nm] | 10.9 [Nm]
# | | |
# M3 | 0.5 | 1.0 | 1.3
# M4 | 1.2 | 2.4 | 3.0
# M5 | 2.4 | 4.7 | 5.9
# M6 | 3.9 | 7.8 | 9.8
# M8 | 9.4 | 18.8 | 23.5
# M10 | 18.3 | 36.6 | 45.8

nenndurchmesser_schrauben = [3, 4, 5, 6, 8, 10]
# (Festigkeitsklasse: Zugfestigkeit in MPa)
festigkeitsklassen = [
    {"klasse": "4.6", "rm": 400},
    {"klasse": "8.8", "rm": 800},
    {"klasse": "10.9", "rm": 1000}
]

```

```

print("
print(" Anziehdrehmoment-Tabelle (trocken, verzinkt)")
print("
print("Gewinde | 4.6 [Nm] | 8.8 [Nm] | 10.9 [Nm]")
print(" | | | ")

for durchmesser in nenndurchmesser_schrauben:
    print(f" M{durchmesser} |", end="")
    for fk in festigkeitsklassen:
        # Formel: Spannungsquerschnitt  $A_s = 0.8 * d^2/4$ 
        import math
        spannungsquerschnitt = 0.8 * (math.pi * (durchmesser ** 2)) / 4
        # Formel: Vorspannkraft  $F = 0.7 * A_s * R_m$ 
        vorspannkraft = 0.7 * spannungsquerschnitt * fk["rm"]
        # Formel: Anziehdrehmoment  $M_a = 0.2 * d * F$ 
        anziehdrehmoment = 0.2 * durchmesser * vorspannkraft
        print(f" {anziehdrehmoment:6.1f} |", end="")
    print()

```

Datei: lessons/V08-Listen-und-Datenstrukturen-Teil1/calculator.py

```

# Einfache Ausgabe, um das Programm zu erklaren.
print("Der tolle Taschenrechner!")
print("Lass uns zwei Zahlen addieren")

# Zeige die moeglichen Operatoren an.
print("Operatoren:")
print("- addition/a")
print("- subtraktion/s")
print("- multiplikation/m")
print("- division/d")
print("- exit/e")
# Hier wird die Nutzereingabe fuer den Operator gespeichert.
operator = ""
# while True bedeutet: Schleife laeuft endlos, bis ein break kommt.
while True:
    # Wiederhole den gesamten Ablauf, bis der Nutzer beendet.
    operator = input("Gib deine Rechenoperation an:")
    # if prueft eine Bedingung; wenn sie stimmt, wird der Block ausgefuehrt.
    if operator == "exit" or operator == "e":
        break
    # if/elif/else waehlen genau einen der drei Zweige.
    if operator in ["multiplikation", "m", "muli", "*"]:

```

```

        # Multiplikation startet bei 1, damit das Ergebnis korrekt bleibt.
        ergebnis = 1
    # elif ist der zweite Zweig, falls das if nicht passt.
    elif operator == "division" or operator == "d":
        # Bei Division brauchen wir zuerst den Zaehler.
        ergebnis = float(input("Gib den Zähler der Division an:"))
    # else gilt, wenn weder if noch elif zutreffen.
    else:
        # Addition und Subtraktion starten bei 0.
        ergebnis = 0
    # Zaehler fuer die Eingaben (1. Zahl, 2. Zahl, ...).
    i = 1
    # while True bedeutet: Zahlen werden so lange abgefragt, bis abgebrochen wird.
    while True:
        # Frage fortlaufend Zahlen ab.
        zahl = float(input(f"Gib die {i} Zahl ein:"))
        # 0 ist hier das Abbruchsignal fuer die Zahlenliste.
        # if prueft hier, ob die Eingabe das Abbruchsignal ist.
        if zahl == 0:
            break
        # if prueft hier den gewaehlten Operator.
        if operator == "addition" or operator == "a" :
            # Addiere die eingegebene Zahl.
            ergebnis += zahl
        # if prueft hier den gewaehlten Operator.
        if operator == "subtraktion" or operator == "s":
            # Subtrahiere die eingegebene Zahl.
            ergebnis -= zahl
        # if prueft hier den gewaehlten Operator.
        if operator == "multiplikation" or operator == "m":
            # Multipliziere mit der eingegebenen Zahl.
            ergebnis *= zahl
        # if prueft hier den gewaehlten Operator.
        if operator == "division" or operator == "d":
            # Teile durch die eingegebene Zahl.
            ergebnis /= zahl
        i+=1
    # Ausgabe des berechneten Ergebnisses.
    print(f"Das Ergebnis ist: {ergebnis}")
    print()

```

Datei: lessons/V08-Listen-und-Datenstrukturen-Teil1/listen.py

```
liste2 = [3.0,4.0,5.0,6.0,7.0,89.0,123.0]

liste = ["sdfsdf", 242,'s', True, 234.2]
index = 3 + 4
index += 1
# print(len(l))
# print(l)
# l.append(4.3)
# print(len(l))
# print(l)
if len(liste2) > index:
    print(liste2[index])
else:
    print("Index ist zu groß")

# print(liste2[9])

if 1.0 in liste2:
    print("element ist enthalten")
else:
    print("element ist nicht enthalten")

# index = l.index(5.0)
# print(index)
```

Datei: lessons/V09-Listen-und-Datenstrukturen-Teil2/p2.py

```
datei = open("lessons/V09-Listen-und-Datenstrukturen-Teil2/maschine_01.log")
zeilen = datei.readlines()
print(f"Anzahl Zeilen: {len(zeilen)}")
anzahlAlarm = 0
anzahlError = 0

for zeile in zeilen:
    datum = zeile.split("|")[0]
    print(datum)
    anzahlAlarm += zeile.count("ALARM")
    anzahlError += zeile.count("ERROR")

print(f"Anzahl Alarm: {anzahlAlarm}")
```

```
print(f"Anzahl Error: {anzahlError}")
```

Datei: lessons/V09-Listen-und-Datenstrukturen-Teil2/try__catch.py

```
# eingabe = input("eingabe Zahl: ")

# zahl = int(eingabe)

try:
    zahl = int(input("Gib eine Zahl ein: "))
    ergebnis = 100 / zahl
    print(f"Ergebnis: {ergebnis}")
except ValueError as e:
    print("Fehler: Keine gültige Zahl eingegeben!")
except ZeroDivisionError as e:
    print("Fehler: Division durch Null ist nicht erlaubt!")
finally:
    print("wird immer ausgegeben")
```

Datei: lessons/V16-Netzwerktechnik-Grundlagen-Protokolle-Teil2/generate__datasets.py

```
import csv
import random
from datetime import datetime, timedelta

# Generate sensoren_daten_erweitert.csv with >300 rows
print("Generating sensoren_daten_erweitert.csv...")
machines = ['CNC-01', 'CNC-02', 'Presse-01', 'Presse-02', 'Drehbank-01', 'Fraese-01']
sensor_types = ['Temperatur', 'Drehzahl', 'Druck', 'Schwingung']
units = {'Temperatur': '°C', 'Drehzahl': 'RPM', 'Druck': 'bar', 'Schwingung': 'mm/s'}
status_options = ['Normal', 'Warnung', 'Kritisch']

base_time = datetime(2024, 1, 15, 8, 0, 0)
rows = []

sensor_id = 1
for hour in range(24): # 24 hours
    for minute in range(0, 60, 5): # Every 5 minutes
        timestamp = base_time + timedelta(hours=hour, minutes=minute)
        for machine in machines:
            for sensor_type in sensor_types:
                # Generate realistic values based on sensor type
```

```

        if sensor_type == 'Temperatur':
            value = random.uniform(60, 110)
        elif sensor_type == 'Drehzahl':
            value = random.randint(1500, 3500)
        elif sensor_type == 'Druck':
            value = random.uniform(100, 200)
        else: # Schwingung
            value = random.uniform(0.3, 1.5)

        # Occasionally mark as warning or critical
        rand_val = random.random()
        if rand_val < 0.05:
            status = 'Kritisch'
        elif rand_val < 0.15:
            status = 'Warnung'
        else:
            status = 'Normal'

        rows.append({
            'Sensor_ID': f'S{sensor_id:04d}',
            'Maschine': machine,
            'Typ': sensor_type,
            'Wert': round(value, 1),
            'Einheit': units[sensor_type],
            'Timestamp': timestamp.strftime('%Y-%m-%d %H:%M:%S'),
            'Status': status
        })
        sensor_id += 1

# Write to CSV
with open('sensoren_daten_erweitert.csv', 'w', newline='', encoding='utf-8') as f:
    writer = csv.DictWriter(f, fieldnames=['Sensor_ID', 'Maschine', 'Typ', 'Wert', 'Einheit'])
    writer.writeheader()
    writer.writerows(rows)

print(f" Created sensoren_daten_erweitert.csv with {len(rows)} rows")

# Generate produktion_auftrage.csv with >300 rows
print("\nGenerating produktion_auftrage.csv...")
machines_prod = ['CNC-01', 'CNC-02', 'CNC-03', 'Presse-01', 'Presse-02', 'Drehbank-01', 'Drehbank-02']
bauteile = ['Welle-A', 'Flansch-B', 'Gehaeuse-C', 'Bolzen-D', 'Platte-E', 'Buchse-F']
status_prod = ['Abgeschlossen', 'Abgeschlossen', 'Abgeschlossen', 'Abgeschlossen', 'Verzoegert']

base_date = datetime(2024, 1, 1)
prod_rows = []

```



```

auftrag_id = 1
for day in range(45): # 45 days
    date = base_date + timedelta(days=day)
    # Each day, 7-10 orders
    num_orders = random.randint(7, 10)
    for _ in range(num_orders):
        machine = random.choice(machines_prod)
        bauteil = random.choice(bauteile)
        zielmenge = random.randint(50, 500)

        # Calculate production with realistic scrap
        status = random.choice(status_prod)
        if status == 'Verzoegert':
            produziert = int(zielmenge * random.uniform(0.75, 0.92))
            ausschuss = int(produziert * random.uniform(0.05, 0.12))
        else:
            produziert = int(zielmenge * random.uniform(0.92, 0.99))
            ausschuss = int(produziert * random.uniform(0.01, 0.05))

        # Cycle time depends on part type
        if 'Welle' in bauteil or 'Bolzen' in bauteil:
            zykluszeit = random.uniform(10, 15)
        elif 'Gehaeuse' in bauteil:
            zykluszeit = random.uniform(20, 30)
        else:
            zykluszeit = random.uniform(5, 12)

        prod_rows.append({
            'Auftrag_ID': f'A{auftrag_id:04d}',
            'Maschine': machine,
            'Bauteil': bauteil,
            'Zielmenge': zielmenge,
            'Produziert': produziert,
            'Ausschuss': ausschuss,
            'Zykluszeit_s': round(zykluszeit, 1),
            'Datum': date.strftime('%Y-%m-%d'),
            'Status': status
        })
    auftrag_id += 1

# Write to CSV
with open('produktion_auftrage.csv', 'w', newline='', encoding='utf-8') as f:
    writer = csv.DictWriter(f, fieldnames=['Auftrag_ID', 'Maschine', 'Bauteil', 'Zielmenge',
    writer.writeheader()
    writer.writerows(prod_rows)

```

```
print(f" Created produktion_auftrage.csv with {len(prod_rows)} rows")
print("\nDataset generation complete!")
```
